

Terminal Web Retro ESP32

1.0

Generado por Doxygen 1.9.8

1 Índice de estructuras de datos	1
1.1 Estructuras de datos	1
2 Índice de archivos	3
2.1 Lista de archivos	3
3 Documentación de estructuras de datos	5
3.1 Referencia de la estructura fan_schedule_t	5
3.1.1 Descripción detallada	5
3.1.2 Documentación de campos	5
3.1.2.1 end_hour	5
3.1.2.2 min_temp	5
3.1.2.3 start_hour	6
3.2 Referencia de la estructura gpio_command_t	6
3.2.1 Documentación de campos	6
3.2.1.1 command	6
3.2.1.2 command_id	6
3.2.1.3 response	6
3.3 Referencia de la estructura ntc_data_t	6
3.3.1 Descripción detallada	7
3.3.2 Documentación de campos	7
3.3.2.1 raw_adc_value	7
3.3.2.2 resistance	7
3.3.2.3 temperature_c	7
3.4 Referencia de la estructura pir_event_t	7
3.4.1 Descripción detallada	7
3.4.2 Documentación de campos	8
3.4.2.1 motion	8
3.5 Referencia de la estructura registro_activo_t	8
3.5.1 Descripción detallada	8
3.5.2 Documentación de campos	8
3.5.2.1 activo	8
3.5.2.2 velocidad	8
3.6 Referencia de la estructura session_t	9
3.6.1 Documentación de campos	9
3.6.1.1 authenticated	9
3.6.1.2 ip	9
3.6.1.3 last_activity	9
3.7 Referencia de la estructura terminal_context_t	9
3.7.1 Documentación de campos	9
3.7.1.1 gpio_command_queue	9
3.7.1.2 gpio_response_queue	9
3.8 Referencia de la estructura webserver_context_t	10

3.8.1 Documentación de campos	10
3.8.1.1 command_id_counter	10
3.8.1.2 command_id_mutex	10
3.8.1.3 gpio_command_queue	10
3.8.1.4 gpio_response_queue	10
3.8.1.5 server	10
3.8.1.6 session_mutex	10
3.8.1.7 sessions	10
4 Documentación de archivos	11
4.1 Referencia del archivo front/script.js	11
4.2 Referencia del archivo main/fan_control.c	11
4.2.1 Descripción detallada	12
4.2.2 Hardware	13
4.2.3 Características	13
4.2.4 Modos de operación	13
4.2.5 Documentación de «define»	14
4.2.5.1 FAN_AUTO_TEMP_TASK_PRIORITY	14
4.2.5.2 FAN_AUTO_TEMP_TASK_STACK_SIZE	15
4.2.5.3 FAN_AUTO_TEMP_UPDATE_INTERVAL_MS	15
4.2.5.4 FAN_SCHEDULE_TASK_PRIORITY	15
4.2.5.5 FAN_SCHEDULE_TASK_STACK_SIZE	15
4.2.5.6 FAN_SCHEDULE_UPDATE_INTERVAL_MS	15
4.2.6 Documentación de funciones	15
4.2.6.1 calculate_temp_percent()	15
4.2.6.2 fan_auto_temp_task()	15
4.2.6.3 fan_get_current_percent()	15
4.2.6.4 fan_get_mode()	15
4.2.6.5 fan_init()	16
4.2.6.6 fan_schedule_task()	16
4.2.6.7 fan_set_manual_percent()	16
4.2.6.8 fan_set_mode()	16
4.2.6.9 fan_set_schedule()	16
4.2.6.10 fan_start_auto_temp_task()	17
4.2.6.11 fan_start_schedule_task()	17
4.2.6.12 fan_update_auto_temp()	17
4.2.6.13 fan_update_from_registros()	17
4.2.6.14 fan_update_schedule()	17
4.2.6.15 set_fan_pwm()	18
4.2.7 Documentación de variables	18
4.2.7.1 current_mode	18
4.2.7.2 current_pwm_percent	18

4.2.7.3 fan_schedule	18
4.2.7.4 manual_percent	18
4.2.7.5 TAG	18
4.3 Referencia del archivo main/fan_control.h	18
4.3.1 Descripción detallada	19
4.3.2 Hardware	20
4.3.3 Características	20
4.3.4 Modos de operación	20
4.3.5 Documentación de «define»	21
4.3.5.1 FAN_DUTY_RES	21
4.3.5.2 FAN_FREQUENCY	21
4.3.5.3 FAN_LEDC_CHANNEL	21
4.3.5.4 FAN_LEDC_TIMER	21
4.3.5.5 FAN_PWM_PIN	21
4.3.5.6 FAN_TEMP_HYSTERESIS	21
4.3.5.7 FAN_TEMP_MAX_C	21
4.3.5.8 FAN_TEMP_MIN_C	21
4.3.6 Documentación de enumeraciones	21
4.3.6.1 fan_mode_t	21
4.3.7 Documentación de funciones	22
4.3.7.1 fan_get_current_percent()	22
4.3.7.2 fan_get_mode()	22
4.3.7.3 fan_init()	22
4.3.7.4 fan_set_manual_percent()	22
4.3.7.5 fan_set_mode()	22
4.3.7.6 fan_set_schedule()	23
4.3.7.7 fan_start_auto_temp_task()	23
4.3.7.8 fan_start_schedule_task()	23
4.3.7.9 fan_update_auto_temp()	24
4.3.7.10 fan_update_from_registros()	24
4.3.7.11 fan_update_schedule()	24
4.4 fan_control.h	24
4.5 Referencia del archivo main/main.c	25
4.5.1 Descripción detallada	26
4.5.2 Arquitectura del sistema	26
4.5.3 Tareas FreeRTOS	26
4.5.4 Colas FreeRTOS	26
4.5.5 Semáforos FreeRTOS	27
4.5.6 Documentación de funciones	28
4.5.6.1 app_main()	28
4.5.7 Documentación de variables	28
4.5.7.1 TAG	28

4.6 Referencia del archivo main/ntc_sensor.c	28
4.6.1 Descripción detallada	29
4.6.2 Funcionalidades	30
4.6.3 Hardware	30
4.6.4 Características	30
4.6.5 Documentación de «define»	31
4.6.5.1 ADC_MAX_VALUE	31
4.6.5.2 ADC_MIN_VALUE	31
4.6.5.3 ADC_STABILIZATION_DELAY_MS	31
4.6.5.4 NTC_TASK_PRIORITY	31
4.6.5.5 NTC_TASK_STACK_SIZE	31
4.6.5.6 RESISTANCE_MAX_OHMS	32
4.6.5.7 SENSOR_READ_INTERVAL_MS	32
4.6.5.8 TEMP_ERROR_VALUE	32
4.6.6 Documentación de funciones	32
4.6.6.1 adc_calibration_init()	32
4.6.6.2 ntc_get_current_temperature()	32
4.6.6.3 ntc_read_temperature()	32
4.6.6.4 ntc_reading_task()	33
4.6.6.5 ntc_sensor_init()	33
4.6.6.6 ntc_start_reading_task()	33
4.6.7 Documentación de variables	33
4.6.7.1 adc1_cali_handle	33
4.6.7.2 adc1_handle	33
4.6.7.3 current_ntc_data	33
4.6.7.4 data_mutex	33
4.6.7.5 data_ready	33
4.6.7.6 TAG	34
4.7 Referencia del archivo main/ntc_sensor.h	34
4.7.1 Descripción detallada	34
4.7.2 Hardware	34
4.7.3 Características	35
4.7.4 Documentación de «define»	35
4.7.4.1 ADC_UNIT	35
4.7.4.2 B_COEFFICIENT	35
4.7.4.3 NOMINAL_RESISTANCE	35
4.7.4.4 NOMINAL_TEMPERATURE	35
4.7.4.5 NTC_PIN	35
4.7.4.6 SERIES_RESISTOR	35
4.7.5 Documentación de funciones	36
4.7.5.1 ntc_get_current_temperature()	36
4.7.5.2 ntc_read_temperature()	36

4.7.5.3 ntc_sensor_init()	36
4.7.5.4 ntc_start_reading_task()	36
4.8 ntc_sensor.h	36
4.9 Referencia del archivo main/pir_driver.c	37
4.9.1 Descripción detallada	37
4.9.2 Hardware	38
4.9.3 Características	38
4.9.4 Arquitectura del sistema	38
4.9.5 Uso en el sistema	38
4.9.6 Documentación de funciones	39
4.9.6.1 pir_init()	39
4.9.6.2 pir_is_motion_active()	40
4.9.6.3 pir_isr_handler()	40
4.9.7 Documentación de variables	41
4.9.7.1 s_pir_evt_queue	41
4.9.7.2 s_pir_gpio	41
4.9.7.3 s_pir_initialized	41
4.9.7.4 TAG	41
4.10 Referencia del archivo main/pir_driver.h	41
4.10.1 Descripción detallada	41
4.10.2 Hardware	42
4.10.3 Características	42
4.10.4 Uso en el sistema	42
4.10.5 Documentación de «define»	43
4.10.5.1 PIR_GPIO_PIN	43
4.10.6 Documentación de funciones	43
4.10.6.1 pir_init()	43
4.10.6.2 pir_is_motion_active()	44
4.11 pir_driver.h	44
4.12 Referencia del archivo main/registros.c	45
4.12.1 Descripción detallada	46
4.12.2 Funcionalidades	46
4.12.3 Uso en el sistema	46
4.12.4 Formato de registro	46
4.12.5 Documentación de funciones	47
4.12.5.1 agregar_registro()	47
4.12.5.2 crear_archivo_si_no_existe()	47
4.12.5.3 get_time_ms()	47
4.12.5.4 leer_registros_json()	47
4.12.5.5 normalizar_dia()	48
4.12.5.6 verificar_registro_activo()	48
4.12.6 Documentación de variables	48

4.12.6.1 TAG	48
4.13 Referencia del archivo main/registros.h	48
4.13.1 Descripción detallada	49
4.13.2 Uso en el sistema	49
4.13.3 Formato de registro	49
4.13.4 Documentación de funciones	50
4.13.4.1 agregar_registro()	50
4.13.4.2 crear_archivo_si_no_existe()	50
4.13.4.3 leer_registros_json()	50
4.13.4.4 verificar_registro_activo()	51
4.14 registros.h	51
4.15 Referencia del archivo main/rgb_led.c	51
4.15.1 Descripción detallada	52
4.15.2 Hardware	52
4.15.3 Características	52
4.15.4 Documentación de «define»	53
4.15.4.1 GPIO_GREEN	53
4.15.4.2 LEDC_CHANNEL_G	53
4.15.4.3 LEDC_DUTY_RES	53
4.15.4.4 LEDC_FREQUENCY	53
4.15.4.5 LEDC_MODE	53
4.15.4.6 LEDC_TIMER	53
4.15.5 Documentación de funciones	54
4.15.5.1 rgb_led_init()	54
4.15.5.2 rgb_set_green_percent()	54
4.15.6 Documentación de variables	54
4.15.6.1 rgb_led_initialized	54
4.15.6.2 TAG	54
4.16 Referencia del archivo main/rgb_led.h	54
4.16.1 Descripción detallada	55
4.16.2 Hardware	55
4.16.3 Características	55
4.16.4 Documentación de funciones	55
4.16.4.1 rgb_led_init()	55
4.16.4.2 rgb_set_green_percent()	56
4.17 rgb_led.h	56
4.18 Referencia del archivo main/terminal_commands.c	57
4.18.1 Descripción detallada	57
4.18.2 Funcionalidades	57
4.18.3 Arquitectura del sistema	57
4.18.4 Documentación de funciones	59
4.18.4.1 process_terminal_command()	59

4.18.4.2 terminal_command_task()	59
4.18.5 Documentación de variables	60
4.18.5.1 TAG	60
4.19 Referencia del archivo main/terminal_commands.h	60
4.19.1 Descripción detallada	61
4.19.2 Funcionalidades	61
4.19.3 Arquitectura del sistema	61
4.19.4 Documentación de funciones	62
4.19.4.1 process_terminal_command()	62
4.19.4.2 terminal_command_task()	63
4.20 terminal_commands.h	64
4.21 Referencia del archivo main/time_sync.c	65
4.21.1 Descripción detallada	66
4.21.2 Funcionalidades	66
4.21.3 Características	66
4.21.4 Documentación de funciones	67
4.21.4.1 cargar_hora_desde_nvs()	67
4.21.4.2 establecer_hora_manual()	67
4.21.4.3 guardar_hora_en_nvs()	67
4.21.4.4 hora_sincronizada()	67
4.21.4.5 obtener_dia_actual()	68
4.21.4.6 obtener_hora_actual()	69
4.21.4.7 sntp_sync_time_cb()	69
4.21.4.8 time_sync_init()	69
4.21.5 Documentación de variables	69
4.21.5.1 dias_semana	69
4.21.5.2 NVS_KEY_TIME	69
4.21.5.3 NVS_NAMESPACE	69
4.21.5.4 sntp_initialized	70
4.21.5.5 TAG	70
4.21.5.6 time_synced	70
4.22 Referencia del archivo main/time_sync.h	70
4.22.1 Descripción detallada	70
4.22.2 Características	70
4.22.3 Documentación de funciones	71
4.22.3.1 establecer_hora_manual()	71
4.22.3.2 hora_sincronizada()	71
4.22.3.3 obtener_dia_actual()	72
4.22.3.4 obtener_hora_actual()	72
4.22.3.5 time_sync_init()	72
4.23 time_sync.h	72
4.24 Referencia del archivo main/web_server.c	73

4.24.1 Descripción detallada	75
4.24.1.1 Sistema de archivos SPIFFS	75
4.24.1.2 Servidor HTTP con múltiples rutas	76
4.24.1.3 Sistema de autenticación y sesiones	76
4.24.1.4 Procesamiento de comandos	76
4.24.1.5 Gestión de temperatura	76
4.24.1.6 Gestión de registros de horarios	76
4.24.2 Documentación de «define»	78
4.24.2.1 CHECK_AUTH	78
4.24.2.2 CHECK_ESP_OK	79
4.24.2.3 CMD_RESPONSE_MAX_ATTEMPTS	79
4.24.2.4 CMD_RESPONSE_TIMEOUT_MS	79
4.24.2.5 GPIO_QUEUE_SIZE	79
4.24.2.6 MAX_SESSIONS	79
4.24.2.7 REDIRECT	79
4.24.2.8 SEND_JSON_RESPONSE	79
4.24.2.9 SEND_JSON_RESPONSE_STR	79
4.24.2.10 SESSION_TIMEOUT_MS	80
4.24.2.11 VALID_PASS	80
4.24.2.12 VALID_USER	80
4.24.3 Documentación de funciones	80
4.24.3.1 cmd_get_handler()	80
4.24.3.2 dashboard_get_handler()	80
4.24.3.3 extract_form_param()	80
4.24.3.4 fan_diagnostic_get_handler()	80
4.24.3.5 fan_manual_post_handler()	80
4.24.3.6 fan_mode_post_handler()	81
4.24.3.7 fan_status_get_handler()	81
4.24.3.8 favicon_get_handler()	81
4.24.3.9 find_or_create_session()	81
4.24.3.10 get_client_ip()	82
4.24.3.11 get_time_ms()	82
4.24.3.12 gpio_command_task_wrapper()	82
4.24.3.13 init_spiffs()	82
4.24.3.14 is_authenticated()	82
4.24.3.15 login_post_handler()	83
4.24.3.16 logout_get_handler()	83
4.24.3.17 pir_status_get_handler()	83
4.24.3.18 register_http_routes()	83
4.24.3.19 registros_get_handler()	84
4.24.3.20 registros_post_handler()	84
4.24.3.21 root_get_handler()	84

4.24.3.22 script_get_handler()	84
4.24.3.23 send_file_from_spiffs()	84
4.24.3.24 session_management_task()	84
4.24.3.25 slider_get_handler()	85
4.24.3.26 start_webserver()	85
4.24.3.27 str_to_lower()	85
4.24.3.28 style_get_handler()	86
4.24.3.29 temperature_get_handler()	86
4.24.3.30 terminal_get_handler()	86
4.24.3.31 time_set_post_handler()	86
4.24.3.32 url_decode()	86
4.24.3.33 verify_spiffs_files()	86
4.24.4 Documentación de variables	86
4.24.4.1 TAG	86
4.25 Referencia del archivo main/web_server.h	86
4.25.1 Descripción detallada	87
4.25.2 Rutas disponibles	87
4.25.2.1 Páginas web	87
4.25.2.2 Sistema de autenticación y sesiones	87
4.25.2.3 API REST	87
4.25.3 Documentación de funciones	89
4.25.3.1 start_webserver()	89
4.26 web_server.h	89
4.27 Referencia del archivo main/wifi_app.c	89
4.27.1 Descripción detallada	90
4.27.2 Funcionalidades	90
4.27.3 Uso en el sistema	90
4.27.4 Documentación de funciones	91
4.27.4.1 wifi_event_handler()	91
4.27.4.2 wifi_init_softap()	92
4.27.4.3 wifi_is_connected()	92
4.27.5 Documentación de variables	92
4.27.5.1 s_retry_num	92
4.27.5.2 TAG	92
4.27.5.3 wifi_connected	92
4.28 Referencia del archivo main/wifi_app.h	92
4.28.1 Descripción detallada	93
4.28.2 Configuración Access Point (SoftAP)	93
4.28.3 Configuración Station (STA)	93
4.28.4 Documentación de «define»	94
4.28.4.1 ESP_WIFI_CHANNEL	94
4.28.4.2 ESP_WIFI_PASS	94

4.28.4.3 ESP_WIFI_SSID	94
4.28.4.4 MAX_STA_CONN	94
4.28.4.5 WIFI_MAX_RETRY	94
4.28.4.6 WIFI_STA_PASS	94
4.28.4.7 WIFI_STA_SSID	94
4.28.5 Documentación de funciones	94
4.28.5.1 wifi_init_softap()	94
4.28.5.2 wifi_is_connected()	95
4.29 wifi_app.h	95
Index	97

Chapter 1

Índice de estructuras de datos

1.1 Estructuras de datos

Lista de estructuras con breves descripciones:

fan_schedule_t	Estructura para el modo programado (legacy, actualmente se usan registros)	5
gpio_command_t		6
ntc_data_t	Estructura que contiene los datos del sensor NTC	6
pir_event_t	Estructura de evento del sensor PIR	7
registro_activo_t	Estructura para representar un registro activo	8
session_t		9
terminal_context_t		9
webserver_context_t		10

Chapter 2

Índice de archivos

2.1 Lista de archivos

Lista de todos los archivos con breves descripciones:

front/ script.js	11
main/ fan_control.c	
Implementación del controlador de ventilador mediante PWM	11
main/ fan_control.h	
Controlador de ventilador mediante PWM para ESP32	18
main/ main.c	
Punto de entrada principal de la aplicación ESP32 - Terminal Web Retro	25
main/ ntc_sensor.c	
Implementación del driver para sensor de temperatura NTC	28
main/ ntc_sensor.h	
Driver para sensor de temperatura NTC (Negative Temperature Coefficient)	34
main/ pir_driver.c	
Implementación del driver para sensor PIR	37
main/ pir_driver.h	
Driver para sensor PIR (Passive Infrared Sensor)	41
main/ registros.c	
Implementación de la gestión de registros de horarios	45
main/ registros.h	
Gestión de registros de horarios del ventilador	48
main/ rgb_led.c	
Implementación del controlador de LED RGB	51
main/ rgb_led.h	
Controlador de LED RGB mediante PWM	54
main/ terminal_commands.c	
Implementación del procesamiento de comandos de terminal	57
main/ terminal_commands.h	
Procesamiento de comandos de terminal web	60
main/ time_sync.c	
Implementación de la sincronización de tiempo mediante SNTP	65
main/ time_sync.h	
Sincronización de tiempo mediante SNTP	70
main/ web_server.c	
Implementación del servidor web HTTP del ESP32	73
main/ web_server.h	
Servidor web HTTP del ESP32	86

main/ wifi_app.c	
Implementación del módulo de WiFi en modo AP+STA	89
main/ wifi_app.h	
Configuración y gestión de WiFi en modo AP+STA	92

Chapter 3

Documentación de estructuras de datos

3.1 Referencia de la estructura fan_schedule_t

Estructura para el modo programado (legacy, actualmente se usan registros)

```
#include <fan_control.h>
```

Campos de datos

- uint8_t [start_hour](#)
- uint8_t [end_hour](#)
- float [min_temp](#)

3.1.1 Descripción detallada

Estructura para el modo programado (legacy, actualmente se usan registros)

Nota

Esta estructura se mantiene por compatibilidad, pero el modo SCHEDULE ahora funciona basado en registros guardados en SPIFFS

3.1.2 Documentación de campos

3.1.2.1 end_hour

```
uint8_t fan_schedule_t::end_hour
```

3.1.2.2 min_temp

```
float fan_schedule_t::min_temp
```

3.1.2.3 start_hour

```
uint8_t fan_schedule_t::start_hour
```

La documentación de esta estructura está generada del siguiente archivo:

- main/[fan_control.h](#)

3.2 Referencia de la estructura gpio_command_t

```
#include <terminal_commands.h>
```

Campos de datos

- uint32_t [command_id](#)
- char [command](#) [100]
- char [response](#) [512]

3.2.1 Documentación de campos

3.2.1.1 command

```
char gpio_command_t::command[100]
```

3.2.1.2 command_id

```
uint32_t gpio_command_t::command_id
```

3.2.1.3 response

```
char gpio_command_t::response[512]
```

La documentación de esta estructura está generada del siguiente archivo:

- main/[terminal_commands.h](#)

3.3 Referencia de la estructura ntc_data_t

Estructura que contiene los datos del sensor NTC.

```
#include <ntc_sensor.h>
```

Campos de datos

- float `temperature_c`
- float `resistance`
- int `raw_adc_value`

3.3.1 Descripción detallada

Estructura que contiene los datos del sensor NTC.

3.3.2 Documentación de campos

3.3.2.1 `raw_adc_value`

```
int ntc_data_t::raw_adc_value
```

3.3.2.2 `resistance`

```
float ntc_data_t::resistance
```

3.3.2.3 `temperature_c`

```
float ntc_data_t::temperature_c
```

La documentación de esta estructura está generada del siguiente archivo:

- main/`ntc_sensor.h`

3.4 Referencia de la estructura `pir_event_t`

Estructura de evento del sensor PIR.

```
#include <pir_driver.h>
```

Campos de datos

- bool `motion`
true si hay movimiento detectado, false si no hay movimiento

3.4.1 Descripción detallada

Estructura de evento del sensor PIR.

Esta estructura se envía a través de la cola de eventos cuando se detecta un cambio en el estado del sensor PIR (movimiento detectado o no detectado).

3.4.2 Documentación de campos

3.4.2.1 motion

```
bool pir_event_t::motion
```

true si hay movimiento detectado, false si no hay movimiento

La documentación de esta estructura está generada del siguiente archivo:

- [main/pir_driver.h](#)

3.5 Referencia de la estructura registro_activo_t

Estructura para representar un registro activo.

```
#include <registros.h>
```

Campos de datos

- int [velocidad](#)
- bool [activo](#)

3.5.1 Descripción detallada

Estructura para representar un registro activo.

3.5.2 Documentación de campos

3.5.2.1 activo

```
bool registro_activo_t::activo
```

3.5.2.2 velocidad

```
int registro_activo_t::velocidad
```

La documentación de esta estructura está generada del siguiente archivo:

- [main/registros.h](#)

3.6 Referencia de la estructura `session_t`

Campos de datos

- `char ip[16]`
- `int64_t last_activity`
- `bool authenticated`

3.6.1 Documentación de campos

3.6.1.1 `authenticated`

```
bool session_t::authenticated
```

3.6.1.2 `ip`

```
char session_t::ip[16]
```

3.6.1.3 `last_activity`

```
int64_t session_t::last_activity
```

La documentación de esta estructura está generada del siguiente archivo:

- `main/web_server.c`

3.7 Referencia de la estructura `terminal_context_t`

Campos de datos

- `QueueHandle_t gpio_command_queue`
- `QueueHandle_t gpio_response_queue`

3.7.1 Documentación de campos

3.7.1.1 `gpio_command_queue`

```
QueueHandle_t terminal_context_t::gpio_command_queue
```

3.7.1.2 `gpio_response_queue`

```
QueueHandle_t terminal_context_t::gpio_response_queue
```

La documentación de esta estructura está generada del siguiente archivo:

- `main/terminal_commands.c`

3.8 Referencia de la estructura `webserver_context_t`

Campos de datos

- `httpd_handle_t` [server](#)
- `QueueHandle_t` [gpio_command_queue](#)
- `QueueHandle_t` [gpio_response_queue](#)
- `SemaphoreHandle_t` [session_mutex](#)
- `uint32_t` [command_id_counter](#)
- `SemaphoreHandle_t` [command_id_mutex](#)
- [session_t](#) `sessions` [[MAX_SESSIONS](#)]

3.8.1 Documentación de campos

3.8.1.1 `command_id_counter`

```
uint32_t webserver_context_t::command_id_counter
```

3.8.1.2 `command_id_mutex`

```
SemaphoreHandle_t webserver_context_t::command_id_mutex
```

3.8.1.3 `gpio_command_queue`

```
QueueHandle_t webserver_context_t::gpio_command_queue
```

3.8.1.4 `gpio_response_queue`

```
QueueHandle_t webserver_context_t::gpio_response_queue
```

3.8.1.5 `server`

```
httpd_handle_t webserver_context_t::server
```

3.8.1.6 `session_mutex`

```
SemaphoreHandle_t webserver_context_t::session_mutex
```

3.8.1.7 `sessions`

```
session\_t webserver_context_t::sessions[MAX\_SESSIONS]
```

La documentación de esta estructura está generada del siguiente archivo:

- `main/web_server.c`

Chapter 4

Documentación de archivos

4.1 Referencia del archivo front/script.js

4.2 Referencia del archivo main/fan_control.c

Implementación del controlador de ventilador mediante PWM.

```
#include "fan_control.h"
#include "ntc_sensor.h"
#include "registros.h"
#include "time_sync.h"
#include "pir_driver.h"
#include "esp_log.h"
#include "driver/ledc.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include <math.h>
#include <string.h>
```

defines

- `#define FAN_AUTO_TEMP_TASK_STACK_SIZE 4096`
- `#define FAN_AUTO_TEMP_TASK_PRIORITY 5`
- `#define FAN_AUTO_TEMP_UPDATE_INTERVAL_MS 1000`
- `#define FAN_SCHEDULE_TASK_STACK_SIZE 4096`
- `#define FAN_SCHEDULE_TASK_PRIORITY 5`
- `#define FAN_SCHEDULE_UPDATE_INTERVAL_MS 10000`

Funciones

- static void `set_fan_pwm` (uint8_t percent)
Establece el ciclo de trabajo PWM del ventilador.
- static uint8_t `calculate_temp_percent` (float temp_c)
Calcula el porcentaje de velocidad basado en la temperatura.
- void `fan_init` (void)
Inicializa el PWM para el control del ventilador.
- void `fan_set_mode` (fan_mode_t mode)
Establece el modo de operación del ventilador.
- fan_mode_t `fan_get_mode` (void)
Obtiene el modo de operación actual.
- void `fan_set_manual_percent` (uint8_t percent)
Establece el porcentaje de velocidad en modo manual.
- void `fan_set_schedule` (const fan_schedule_t *schedule)
Establece la configuración para el modo programado (legacy)
- void `fan_update_auto_temp` (float current_temp)
Actualiza la velocidad del ventilador en modo automático por temperatura.
- void `fan_update_schedule` (float current_temp, uint8_t current_hour)
Actualiza la velocidad del ventilador en modo programado (legacy)
- void `fan_update_from_registros` (void)
Actualiza el ventilador basado en registros activos.
- uint8_t `fan_get_current_percent` (void)
Obtiene el porcentaje de velocidad PWM actual.
- static void `fan_auto_temp_task` (void *pvParameters)
Tarea que lee la temperatura periódicamente y actualiza el ventilador cuando está en modo FAN_MODE_AUTO ↔ TEMP.
- void `fan_start_auto_temp_task` (void)
Inicia la tarea de control automático por temperatura.
- static void `fan_schedule_task` (void *pvParameters)
Tarea que verifica periódicamente los registros y actualiza el ventilador cuando está en modo FAN_MODE ↔ SCHEDULE.
- void `fan_start_schedule_task` (void)
Inicia la tarea de control por horarios (registros).

Variables

- static const char * `TAG` = "FAN_CONTROL"
- static fan_mode_t `current_mode` = FAN_MODE_OFF
- static uint8_t `manual_percent` = 0
- static uint8_t `current_pwm_percent` = 0
- static fan_schedule_t `fan_schedule`

4.2.1 Descripción detallada

Implementación del controlador de ventilador mediante PWM.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del módulo de control del ventilador mediante PWM. Este módulo gestiona el control de velocidad del ventilador en diferentes modos de operación: OFF, MANUAL, AUTO_TEMP (automático por temperatura) y SCHEDULE (control por horarios basado en registros).

4.2.2 Hardware

- Ventilador: Controlado mediante PWM en GPIO 26 (LEDC Channel 2, Timer 1)
- Sensor PIR: El ventilador solo funciona si detecta presencia (excepto modo MANUAL)
- Sensor NTC: Utilizado para control automático por temperatura

4.2.3 Características

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 25kHz (adecuada para motores)
- Múltiples modos de operación con tareas independientes
- Integración con sistema de registros para control por horarios
- Verificación de presencia mediante sensor PIR (excepto modo MANUAL)

4.2.4 Modos de operación

- FAN_MODE_OFF: Ventilador apagado (0% PWM)
- FAN_MODE_MANUAL: Control manual por porcentaje (0-100%), ignora PIR
- FAN_MODE_AUTO_TEMP: Control automático basado en temperatura (15-25°C)
- FAN_MODE_SCHEDULE: Control por horarios usando registros guardados

ARCHIVO: [fan_control.c](#)

RESUMEN: Implementación del módulo de control del ventilador mediante PWM. Este módulo gestiona el control de velocidad del ventilador en diferentes modos de operación: OFF, MANUAL, AUTO_TEMP (automático por temperatura) y SCHEDULE (control por horarios basado en registros).

Hardware:

- Ventilador: Controlado mediante PWM en GPIO 26 (LEDC Channel 2, Timer 1)
- Sensor PIR: El ventilador solo funciona si detecta presencia (excepto modo MANUAL)
- Sensor NTC: Utilizado para control automático por temperatura

Características:

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 25kHz (adecuada para motores)
- Múltiples modos de operación con tareas independientes
- Integración con sistema de registros para control por horarios
- Verificación de presencia mediante sensor PIR (excepto modo MANUAL)

Modos de operación:

- FAN_MODE_OFF: Ventilador apagado (0% PWM)
 - FAN_MODE_MANUAL: Control manual por porcentaje (0-100%), ignora PIR
 - FAN_MODE_AUTO_TEMP: Control automático basado en temperatura (15-25°C)
 - FAN_MODE_SCHEDULE: Control por horarios usando registros guardados
-

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 48 a 60 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 62 a 75 Sección 3: VARIABLES GLOBALES Y ESTADO se encuentra en las líneas 77 a 85 Sección 4: FUNCIONES INTERNAS DE CONTROL se encuentra en las líneas 87 a 95 Sección 5: FUNCIONES PÚBLICAS DE CONTROL se encuentra en las líneas 97 a 269 Sección 6: TAREA DE CONTROL AUTOMÁTICO POR TEMPERATURA se encuentra en las líneas 271 a 330

Sección 7: TAREA DE CONTROL POR HORARIOS (REGISTROS) se encuentra en las líneas 332 a 387

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

1. fan_auto_temp_task (Sección 6)

- Nombre: "fan_auto_temp"
- Stack: 4096 bytes
- Prioridad: 5 (alta)
- Función: Lee la temperatura periódicamente y actualiza el ventilador cuando está en modo FAN_↔MODE_AUTO_TEMP
- Propósito: Control automático del ventilador basado en temperatura
- Estado: Loop infinito, actualiza cada 1 segundo
- Flujo:
 - (a) Verifica que el modo sea AUTO_TEMP
 - (b) Lee temperatura del sensor NTC
 - (c) Calcula porcentaje de velocidad basado en temperatura
 - (d) Actualiza PWM del ventilador

2. fan_schedule_task (Sección 7)

- Nombre: "fan_schedule"
- Stack: 4096 bytes
- Prioridad: 5 (alta)
- Función: Verifica periódicamente los registros y actualiza el ventilador cuando está en modo FAN_↔MODE_SCHEDULE
- Propósito: Control del ventilador basado en horarios guardados
- Estado: Loop infinito, verifica cada 10 segundos
- Flujo:
 - (a) Verifica que el modo sea SCHEDULE
 - (b) Obtiene día y hora actual
 - (c) Busca registro activo en registros.json
 - (d) Actualiza velocidad del ventilador según registro encontrado

=== COLAS (QUEUES) ===

Ninguna en este módulo. Las colas se gestionan en otros módulos.

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. El acceso a variables globales es thread-safe porque las funciones de control se llaman desde tareas específicas.

4.2.5 Documentación de «define»

4.2.5.1 FAN_AUTO_TEMP_TASK_PRIORITY

```
#define FAN_AUTO_TEMP_TASK_PRIORITY 5
```

4.2.5.2 FAN_AUTO_TEMP_TASK_STACK_SIZE

```
#define FAN_AUTO_TEMP_TASK_STACK_SIZE 4096
```

4.2.5.3 FAN_AUTO_TEMP_UPDATE_INTERVAL_MS

```
#define FAN_AUTO_TEMP_UPDATE_INTERVAL_MS 1000
```

4.2.5.4 FAN_SCHEDULE_TASK_PRIORITY

```
#define FAN_SCHEDULE_TASK_PRIORITY 5
```

4.2.5.5 FAN_SCHEDULE_TASK_STACK_SIZE

```
#define FAN_SCHEDULE_TASK_STACK_SIZE 4096
```

4.2.5.6 FAN_SCHEDULE_UPDATE_INTERVAL_MS

```
#define FAN_SCHEDULE_UPDATE_INTERVAL_MS 10000
```

4.2.6 Documentación de funciones**4.2.6.1 calculate_temp_percent()**

```
static uint8_t calculate_temp_percent (
    float temp_c ) [static]
```

Calcula el porcentaje de velocidad basado en la temperatura.

Parámetros

<i>temp_c</i>	Temperatura actual en Celsius.
---------------	--------------------------------

Devuelve

Porcentaje de 0 a 100.

4.2.6.2 fan_auto_temp_task()

```
static void fan_auto_temp_task (
    void * pvParameters ) [static]
```

Tarea que lee la temperatura periódicamente y actualiza el ventilador cuando está en modo FAN_MODE_AUTO↵_TEMP.

Esta tarea se ejecuta de forma independiente y monitorea la temperatura cada segundo para ajustar automáticamente la velocidad del ventilador.

4.2.6.3 fan_get_current_percent()

```
uint8_t fan_get_current_percent (
    void )
```

Obtiene el porcentaje de velocidad PWM actual.

Devuelve

Porcentaje de velocidad actual (0-100)

4.2.6.4 fan_get_mode()

```
fan_mode_t fan_get_mode (
    void )
```

Obtiene el modo de operación actual.

Devuelve

Modo actual del ventilador

4.2.6.5 fan_init()

```
void fan_init (
    void )
```

Inicializa el PWM para el control del ventilador.

Configura el timer LEDC y el canal PWM para controlar el ventilador. Esta función debe llamarse una vez durante la inicialización del sistema antes de usar cualquier otra función de este módulo.

Proceso de inicialización:

1. Configura el timer LEDC con resolución de 8 bits y frecuencia de 25kHz
2. Configura el canal LEDC para el GPIO del ventilador
3. Inicia el ventilador en modo OFF (0% PWM)

4.2.6.6 fan_schedule_task()

```
static void fan_schedule_task (
    void * pvParameters ) [static]
```

Tarea que verifica periódicamente los registros y actualiza el ventilador cuando está en modo FAN_MODE_↔ SCHEDULE.

Esta tarea se ejecuta de forma independiente y verifica los registros cada minuto para activar el ventilador cuando coincide el día y hora con algún registro.

4.2.6.7 fan_set_manual_percent()

```
void fan_set_manual_percent (
    uint8_t percent )
```

Establece el porcentaje de velocidad en modo manual.

Solo tiene efecto si el ventilador está en modo FAN_MODE_MANUAL. En modo MANUAL, el ventilador ignora el sensor PIR y funciona directamente.

Parámetros

<i>percent</i>	Porcentaje de velocidad (0-100)
----------------	---------------------------------

4.2.6.8 fan_set_mode()

```
void fan_set_mode (
    fan_mode_t mode )
```

Establece el modo de operación del ventilador.

Cambia el modo de operación del ventilador y aplica la configuración correspondiente. Al cambiar de modo, se actualiza inmediatamente la velocidad del ventilador.

Parámetros

<i>mode</i>	Modo a establecer (FAN_MODE_OFF, FAN_MODE_MANUAL, FAN_MODE_AUTO_TEMP, FAN_MODE_SCHEDULE)
-------------	--

4.2.6.9 fan_set_schedule()

```
void fan_set_schedule (
    const fan_schedule_t * schedule )
```

Establece la configuración para el modo programado (legacy)

Nota

Esta función se mantiene por compatibilidad, pero el modo SCHEDULE ahora funciona basado en registros guardados en SPIFFS

Parámetros

<i>schedule</i>	Configuración de horario y temperatura mínima
-----------------	---

4.2.6.10 fan_start_auto_temp_task()

```
void fan_start_auto_temp_task (
    void )
```

Inicia la tarea de control automático por temperatura.

Esta función crea una tarea de FreeRTOS que monitorea la temperatura y actualiza el ventilador automáticamente cuando está en modo AUTO_TEMP.

4.2.6.11 fan_start_schedule_task()

```
void fan_start_schedule_task (
    void )
```

Inicia la tarea de control por horarios (registros).

Inicia la tarea de control por horarios (registros)

Esta función crea una tarea de FreeRTOS que verifica periódicamente los registros guardados y activa el ventilador cuando coincide el día y hora actual con algún registro. Cuando está en modo FAN_MODE_SCHEDULE, el ventilador solo funciona según los registros.

4.2.6.12 fan_update_auto_temp()

```
void fan_update_auto_temp (
    float current_temp )
```

Actualiza la velocidad del ventilador en modo automático por temperatura.

Calcula el porcentaje de velocidad basado en la temperatura actual usando un mapeo lineal entre FAN_TEMP_MIN_C y FAN_TEMP_MAX_C. Solo tiene efecto si el ventilador está en modo FAN_MODE_AUTO_TEMP.

Parámetros

<i>current_temp</i>	Temperatura actual en Celsius
---------------------	-------------------------------

4.2.6.13 fan_update_from_registros()

```
void fan_update_from_registros (
    void )
```

Actualiza el ventilador basado en registros activos.

Verifica si hay un registro activo para el momento actual y ajusta la velocidad. Solo funciona cuando el ventilador está en modo FAN_MODE_SCHEDULE. Durante el modo horario, el ventilador solo funciona según los registros, ignorando otros modos (manual, temperatura).

4.2.6.14 fan_update_schedule()

```
void fan_update_schedule (
    float current_temp,
    uint8_t current_hour )
```

Actualiza la velocidad del ventilador en modo programado (legacy)

Nota

Esta función se mantiene por compatibilidad, pero el modo SCHEDULE ahora funciona basado en registros. Usar `fan_update_from_registros()`.

Parámetros

<code>current_temp</code>	Temperatura actual en Celsius
<code>current_hour</code>	Hora actual (0-23)

4.2.6.15 set_fan_pwm()

```
static void set_fan_pwm (
    uint8_t percent ) [static]
```

Establece el ciclo de trabajo PWM del ventilador.

Parámetros

<code>percent</code>	Porcentaje de 0 a 100.
----------------------	------------------------

Nota

En modo MANUAL, el ventilador funciona independientemente del PIR. En otros modos, el ventilador solo se activará si el sensor PIR detecta presencia. Si no hay presencia, el ventilador se apagará automáticamente.

4.2.7 Documentación de variables**4.2.7.1 current_mode**

```
fan_mode_t current_mode = FAN_MODE_OFF [static]
```

4.2.7.2 current_pwm_percent

```
uint8_t current_pwm_percent = 0 [static]
```

4.2.7.3 fan_schedule

```
fan_schedule_t fan_schedule [static]
```

Valor inicial:

```
= {
    .start_hour = 10,
    .end_hour = 16,
    .min_temp = FAN_TEMP_MIN_C
}
```

4.2.7.4 manual_percent

```
uint8_t manual_percent = 0 [static]
```

4.2.7.5 TAG

```
const char* TAG = "FAN_CONTROL" [static]
```

4.3 Referencia del archivo main/fan_control.h

Controlador de ventilador mediante PWM para ESP32.

```
#include <stdint.h>
#include <stdbool.h>
```

Estructuras de datos

- struct `fan_schedule_t`

Estructura para el modo programado (legacy, actualmente se usan registros)

defines

- #define `FAN_PWM_PIN` 26
- #define `FAN_LEDC_TIMER` LEDC_TIMER_1
- #define `FAN_LEDC_CHANNEL` LEDC_CHANNEL_2
- #define `FAN_DUTY_RES` LEDC_TIMER_8_BIT
- #define `FAN_FREQUENCY` (25000)
- #define `FAN_TEMP_MIN_C` 15.0f
- #define `FAN_TEMP_MAX_C` 25.0f
- #define `FAN_TEMP_HYSTERESIS` 1.0f

Enumeraciones

- enum `fan_mode_t` { `FAN_MODE_OFF` , `FAN_MODE_MANUAL` , `FAN_MODE_AUTO_TEMP` , `FAN_MODE_SCHEDULE` }

Modos de operación del ventilador.

Funciones

- void `fan_init` (void)
Inicializa el PWM para el control del ventilador.
- void `fan_set_mode` (`fan_mode_t` mode)
Establece el modo de operación del ventilador.
- `fan_mode_t` `fan_get_mode` (void)
Obtiene el modo de operación actual.
- void `fan_set_manual_percent` (uint8_t percent)
Establece el porcentaje de velocidad en modo manual.
- void `fan_set_schedule` (const `fan_schedule_t` *schedule)
Establece la configuración para el modo programado (legacy)
- void `fan_update_auto_temp` (float current_temp)
Actualiza la velocidad del ventilador en modo automático por temperatura.
- void `fan_update_schedule` (float current_temp, uint8_t current_hour)
Actualiza la velocidad del ventilador en modo programado (legacy)
- uint8_t `fan_get_current_percent` (void)
Obtiene el porcentaje de velocidad PWM actual.
- void `fan_start_auto_temp_task` (void)
Inicia la tarea de control automático por temperatura.
- void `fan_start_schedule_task` (void)
Inicia la tarea de control por horarios (registros)
- void `fan_update_from_registros` (void)
Actualiza el ventilador basado en registros activos.

4.3.1 Descripción detallada

Controlador de ventilador mediante PWM para ESP32.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para el módulo de control del ventilador mediante PWM. Este módulo gestiona el control de velocidad del ventilador en diferentes modos de operación: OFF, MANUAL, AUTO_TEMP (automático por temperatura) y SCHEDULE (control por horarios basado en registros).

4.3.2 Hardware

- Ventilador: Controlado mediante PWM en GPIO 26 (LEDC Channel 2, Timer 1)
- Sensor PIR: El ventilador solo funciona si detecta presencia (excepto modo MANUAL)
- Sensor NTC: Utilizado para control automático por temperatura

4.3.3 Características

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 25kHz (adecuada para motores)
- Múltiples modos de operación con tareas independientes
- Integración con sistema de registros para control por horarios
- Verificación de presencia mediante sensor PIR (excepto modo MANUAL)

4.3.4 Modos de operación

- FAN_MODE_OFF: Ventilador apagado (0% PWM)
- FAN_MODE_MANUAL: Control manual por porcentaje (0-100%), ignora PIR
- FAN_MODE_AUTO_TEMP: Control automático basado en temperatura (15-25°C)
- FAN_MODE_SCHEDULE: Control por horarios usando registros guardados

ARCHIVO: [fan_control.h](#)

RESUMEN: Header file para el módulo de control del ventilador mediante PWM. Este módulo gestiona el control de velocidad del ventilador en diferentes modos de operación: OFF, MANUAL, AUTO_TEMP (automático por temperatura) y SCHEDULE (control por horarios basado en registros).

Hardware:

- Ventilador: Controlado mediante PWM en GPIO 26 (LEDC Channel 2, Timer 1)
- Sensor PIR: El ventilador solo funciona si detecta presencia (excepto modo MANUAL)
- Sensor NTC: Utilizado para control automático por temperatura

Características:

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 25kHz (adecuada para motores)
- Múltiples modos de operación con tareas independientes
- Integración con sistema de registros para control por horarios
- Verificación de presencia mediante sensor PIR (excepto modo MANUAL)

Modos de operación:

- FAN_MODE_OFF: Ventilador apagado (0% PWM)

- FAN_MODE_MANUAL: Control manual por porcentaje (0-100%), ignora PIR
- FAN_MODE_AUTO_TEMP: Control automático basado en temperatura (15-25 °C)
- FAN_MODE_SCHEDULE: Control por horarios usando registros guardados

ÍNDICE DE SECCIONES:

Sección 1: CONFIGURACIÓN DE PINES Y PWM se encuentra en las líneas 45 a 50 Sección 2: CONFIGURACIÓN DE TEMPERATURAS se encuentra en las líneas 52 a 54 Sección 3: ESTRUCTURAS DE DATOS se encuentra en las líneas 56 a 61 Sección 4: MODOS DE OPERACIÓN se encuentra en las líneas 63 a 68

Sección 5: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 70 a 108

4.3.5 Documentación de «define»

4.3.5.1 FAN_DUTY_RES

```
#define FAN_DUTY_RES LEDC_TIMER_8_BIT
```

4.3.5.2 FAN_FREQUENCY

```
#define FAN_FREQUENCY (25000)
```

4.3.5.3 FAN_LEDC_CHANNEL

```
#define FAN_LEDC_CHANNEL LEDC_CHANNEL_2
```

4.3.5.4 FAN_LEDC_TIMER

```
#define FAN_LEDC_TIMER LEDC_TIMER_1
```

4.3.5.5 FAN_PWM_PIN

```
#define FAN_PWM_PIN 26
```

4.3.5.6 FAN_TEMP_HYSTERESIS

```
#define FAN_TEMP_HYSTERESIS 1.0f
```

4.3.5.7 FAN_TEMP_MAX_C

```
#define FAN_TEMP_MAX_C 25.0f
```

4.3.5.8 FAN_TEMP_MIN_C

```
#define FAN_TEMP_MIN_C 15.0f
```

4.3.6 Documentación de enumeraciones

4.3.6.1 fan_mode_t

```
enum fan_mode_t
```

Modos de operación del ventilador.

Valores de enumeraciones

FAN_MODE_OFF	
FAN_MODE_MANUAL	
FAN_MODE_AUTO_TEMP	
FAN_MODE_SCHEDULE	

4.3.7 Documentación de funciones

4.3.7.1 fan_get_current_percent()

```
uint8_t fan_get_current_percent (
    void )
```

Obtiene el porcentaje de velocidad PWM actual.

Devuelve

Porcentaje de velocidad actual (0-100)

4.3.7.2 fan_get_mode()

```
fan_mode_t fan_get_mode (
    void )
```

Obtiene el modo de operación actual.

Devuelve

Modo actual del ventilador

4.3.7.3 fan_init()

```
void fan_init (
    void )
```

Inicializa el PWM para el control del ventilador.

Configura el timer LEDC y el canal PWM para controlar el ventilador. Esta función debe llamarse una vez durante la inicialización del sistema antes de usar cualquier otra función de este módulo.

Proceso de inicialización:

1. Configura el timer LEDC con resolución de 8 bits y frecuencia de 25kHz
2. Configura el canal LEDC para el GPIO del ventilador
3. Inicia el ventilador en modo OFF (0% PWM)

4.3.7.4 fan_set_manual_percent()

```
void fan_set_manual_percent (
    uint8_t percent )
```

Establece el porcentaje de velocidad en modo manual.

Solo tiene efecto si el ventilador está en modo FAN_MODE_MANUAL. En modo MANUAL, el ventilador ignora el sensor PIR y funciona directamente.

Parámetros

<i>percent</i>	Porcentaje de velocidad (0-100)
----------------	---------------------------------

4.3.7.5 fan_set_mode()

```
void fan_set_mode (
    fan_mode_t mode )
```

Establece el modo de operación del ventilador.

Cambia el modo de operación del ventilador y aplica la configuración correspondiente. Al cambiar de modo, se actualiza inmediatamente la velocidad del ventilador.

Parámetros

<i>mode</i>	Modo a establecer (FAN_MODE_OFF, FAN_MODE_MANUAL, FAN_MODE_AUTO_TEMP, FAN_MODE_SCHEDULE)
-------------	--

4.3.7.6 fan_set_schedule()

```
void fan_set_schedule (
    const fan_schedule_t * schedule )
```

Establece la configuración para el modo programado (legacy)

Nota

Esta función se mantiene por compatibilidad, pero el modo SCHEDULE ahora funciona basado en registros guardados en SPIFFS

Parámetros

<i>schedule</i>	Configuración de horario y temperatura mínima
-----------------	---

4.3.7.7 fan_start_auto_temp_task()

```
void fan_start_auto_temp_task (
    void )
```

Inicia la tarea de control automático por temperatura.

Crea una tarea de FreeRTOS que lee la temperatura periódicamente (cada 1 segundo) y actualiza el ventilador automáticamente cuando está en modo FAN_MODE_AUTO_TEMP.

Requisitos:

- [ntc_sensor_init\(\)](#) debe haber sido llamado
- [ntc_start_reading_task\(\)](#) debe haber sido llamado

La tarea se ejecuta de forma independiente y monitorea la temperatura continuamente.

Esta función crea una tarea de FreeRTOS que monitorea la temperatura y actualiza el ventilador automáticamente cuando está en modo AUTO_TEMP.

4.3.7.8 fan_start_schedule_task()

```
void fan_start_schedule_task (
    void )
```

Inicia la tarea de control por horarios (registros)

Crea una tarea de FreeRTOS que verifica periódicamente los registros guardados (cada 10 segundos) y activa el ventilador cuando coincide el día y hora actual con algún registro guardado en SPIFFS.

Requisitos:

- [time_sync_init\(\)](#) debe haber sido llamado
- La hora debe estar sincronizada ([hora_sincronizada\(\)](#) == true)
- Los registros deben estar guardados en /spiffs/registros.json

Cuando está en modo FAN_MODE_SCHEDULE, el ventilador solo funciona según los registros.

Inicia la tarea de control por horarios (registros)

Esta función crea una tarea de FreeRTOS que verifica periódicamente los registros guardados y activa el ventilador cuando coincide el día y hora actual con algún registro. Cuando está en modo FAN_MODE_SCHEDULE, el ventilador solo funciona según los registros.

4.3.7.9 fan_update_auto_temp()

```
void fan_update_auto_temp (
    float current_temp )
```

Actualiza la velocidad del ventilador en modo automático por temperatura.

Calcula el porcentaje de velocidad basado en la temperatura actual usando un mapeo lineal entre FAN_TEMP_↔ MIN_C y FAN_TEMP_MAX_C. Solo tiene efecto si el ventilador está en modo FAN_MODE_AUTO_TEMP.

Parámetros

<i>current_temp</i>	Temperatura actual en Celsius
---------------------	-------------------------------

4.3.7.10 fan_update_from_registros()

```
void fan_update_from_registros (
    void )
```

Actualiza el ventilador basado en registros activos.

Verifica si hay un registro activo para el día y hora actuales y ajusta la velocidad del ventilador según el registro encontrado.

Solo tiene efecto si el ventilador está en modo FAN_MODE_SCHEDULE. Si no hay registro activo, el ventilador se apaga (0% PWM).

Requisitos:

- La hora debe estar sincronizada
- Debe poder obtener día y hora actual

Verifica si hay un registro activo para el momento actual y ajusta la velocidad. Solo funciona cuando el ventilador está en modo FAN_MODE_SCHEDULE. Durante el modo horario, el ventilador solo funciona según los registros, ignorando otros modos (manual, temperatura).

4.3.7.11 fan_update_schedule()

```
void fan_update_schedule (
    float current_temp,
    uint8_t current_hour )
```

Actualiza la velocidad del ventilador en modo programado (legacy)

Nota

Esta función se mantiene por compatibilidad, pero el modo SCHEDULE ahora funciona basado en registros. Usar [fan_update_from_registros\(\)](#).

Parámetros

<i>current_temp</i>	Temperatura actual en Celsius
<i>current_hour</i>	Hora actual (0-23)

4.4 fan_control.h

[Ir a la documentación de este archivo.](#)

```
00001
00069 #ifndef FAN_CONTROL_H
00070 #define FAN_CONTROL_H
00071
00072 #include <stdint.h>
00073 #include <stdbool.h>
00074
00075 // ===== CONFIGURACIÓN DE PINES Y PWM =====
```

```

00076 // Configuración del hardware para control PWM del ventilador
00077 #define FAN_PWM_PIN      26          // Pin GPIO para el control PWM del ventilador (gate del
      MOSFET)
00078 #define FAN_LEDC_TIMER    LEDC_TIMER_1 // Timer LEDC a usar (diferente al del LED RGB)
00079 #define FAN_LEDC_CHANNEL  LEDC_CHANNEL_2 // Canal LEDC para el ventilador
00080 #define FAN_DUTY_RES      LEDC_TIMER_8_BIT // Resolución de 8 bits (0-255)
00081 #define FAN_FREQUENCY     (25000)      // Frecuencia en Hz, 25kHz es común para motores
00082
00083 // ===== CONFIGURACIÓN DE TEMPERATURAS =====
00084 // Rangos de temperatura para control automático
00085 #define FAN_TEMP_MIN_C    15.0f        // Temperatura mínima para ENCENDER el ventilador (0% PWM)
00086 #define FAN_TEMP_MAX_C    25.0f        // Temperatura máxima para 100% PWM
00087 #define FAN_TEMP_HYSTERESIS 1.0f        // Histéresis para el punto de apagado (opcional)
00088
00089 // ===== ESTRUCTURAS DE DATOS =====
00090 typedef struct {
00091     uint8_t start_hour; // Hora de inicio (0-23)
00092     uint8_t end_hour;   // Hora de fin (0-23)
00093     float min_temp;     // Temperatura mínima para encender dentro del horario
00094 } fan_schedule_t;
00095
00096 // ===== MODOS DE OPERACIÓN =====
00097 typedef enum {
00098     FAN_MODE_OFF,        // Apagado forzado (0% PWM)
00099     FAN_MODE_MANUAL,     // Control manual por porcentaje (0-100%), ignora PIR
00100     FAN_MODE_AUTO_TEMP,  // Control automático por temperatura (15-25°C), requiere PIR
00101     FAN_MODE_SCHEDULE    // Control por horario usando registros guardados, requiere PIR
00102 } fan_mode_t;
00103
00104 // ===== PROTOTIPOS DE FUNCIONES =====
00105
00106 void fan_init(void);
00107
00108 void fan_set_mode(fan_mode_t mode);
00109
00110 fan_mode_t fan_get_mode(void);
00111
00112 void fan_set_manual_percent(uint8_t percent);
00113
00114 void fan_set_schedule(const fan_schedule_t *schedule);
00115
00116 void fan_update_auto_temp(float current_temp);
00117
00118 void fan_update_schedule(float current_temp, uint8_t current_hour);
00119
00120 uint8_t fan_get_current_percent(void);
00121
00122 void fan_start_auto_temp_task(void);
00123
00124 void fan_start_schedule_task(void);
00125
00126 void fan_update_from_registros(void);
00127
00128 #endif // FAN_CONTROL_H

```

4.5 Referencia del archivo main/main.c

Punto de entrada principal de la aplicación ESP32 - Terminal Web Retro.

```

#include <nvs_flash.h>
#include <esp_log.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "fan_control.h"
#include "ntc_sensor.h"
#include "pir_driver.h"
#include "rgb_led.h"
#include "time_sync.h"
#include "web_server.h"
#include "wifi_app.h"

```

Funciones

- void [app_main](#) (void)

Función principal de la aplicación ESP32.

Variables

- static const char * TAG = "MAIN"

4.5.1 Descripción detallada

Punto de entrada principal de la aplicación ESP32 - Terminal Web Retro.

Autor

Proyecto Final RTOS

Fecha

2024

Este es el punto de entrada principal de la aplicación ESP32. Se encarga de inicializar todos los componentes del sistema en el orden correcto:

- Sistema de almacenamiento no volátil (NVS) para WiFi
- Controladores de hardware (LED RGB, sensor NTC de temperatura, sensor PIR, ventilador)
- Configuración de WiFi en modo AP+STA (Access Point + Station)
- Sincronización de tiempo mediante SNTP
- Servidor web HTTP para la interfaz de usuario

El sistema crea una red WiFi propia a la que los usuarios pueden conectarse y acceder a una terminal web retro para controlar LEDs, ventilador y monitorear temperatura. También se conecta a una red WiFi externa para tener Internet y sincronizar la hora mediante SNTP.

4.5.2 Arquitectura del sistema

- Hardware: LED RGB (GPIO 27), Sensor NTC (GPIO 32), Sensor PIR (GPIO 12), Ventilador (GPIO 26)
- Red: WiFi AP+STA (red local + conexión a Internet)
- Servidor: HTTP con SPIFFS para archivos estáticos
- Control: Terminal web, API REST, control automático por temperatura y horarios

4.5.3 Tareas FreeRTOS

Las tareas se crean en los módulos correspondientes:

- [ntc_start_reading_task\(\)](#): Tarea de lectura periódica del sensor NTC
- [fan_start_auto_temp_task\(\)](#): Tarea de control automático por temperatura
- [fan_start_schedule_task\(\)](#): Tarea de control por horarios (registros)
- [start_webserver\(\)](#): Crea tareas de procesamiento de comandos y gestión de sesiones

4.5.4 Colas FreeRTOS

Las colas se crean en [web_server.c](#):

- [gpio_command_queue](#): Para comandos desde handler HTTP a tarea de procesamiento
- [gpio_response_queue](#): Para respuestas desde tarea de procesamiento a handler HTTP

4.5.5 Semáforos FreeRTOS

Los semáforos se crean en [web_server.c](#):

- `session_mutex`: Protege acceso a sesiones de usuarios
- `command_id_mutex`: Protege contador de IDs de comandos

ARCHIVO: [main.c](#)

RESUMEN: Este es el punto de entrada principal de la aplicación ESP32. Se encarga de inicializar todos los componentes del sistema en el orden correcto:

- Sistema de almacenamiento no volátil (NVS) para WiFi
- Controladores de hardware (LED RGB, sensor NTC de temperatura, sensor PIR, ventilador)
- Configuración de WiFi en modo AP+STA (Access Point + Station)
- Sincronización de tiempo mediante SNTP
- Servidor web HTTP para la interfaz de usuario

El sistema crea una red WiFi propia a la que los usuarios pueden conectarse y acceder a una terminal web retro para controlar LEDs, ventilador y monitorear temperatura. También se conecta a una red WiFi externa para tener Internet y sincronizar la hora mediante SNTP.

Arquitectura del sistema:

- Hardware: LED RGB (GPIO 27), Sensor NTC (GPIO 32), Sensor PIR (GPIO 12), Ventilador (GPIO 26)
- Red: WiFi AP+STA (red local + conexión a Internet)
- Servidor: HTTP con SPIFFS para archivos estáticos
- Control: Terminal web, API REST, control automático por temperatura y horarios

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 39 a 53 Sección 2: DEFINICIONES se encuentra en las líneas 55 a 56 Sección 3: FUNCIÓN PRINCIPAL (`app_main`) se encuentra en las líneas 58 a 142

- Subsección 3.1: INICIALIZACIÓN DE NVS se encuentra en las líneas 72 a 80
- Subsección 3.2: INICIALIZACIÓN DE HARDWARE se encuentra en las líneas 82 a 106
- Subsección 3.3: INICIALIZACIÓN DE RED se encuentra en las líneas 108 a 131

- Subsección 3.4: INICIO DEL SERVIDOR WEB se encuentra en las líneas 133 a 141

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

Las tareas se crean en los módulos correspondientes:

- [ntc_start_reading_task\(\)](#): Tarea de lectura periódica del sensor NTC
- [fan_start_auto_temp_task\(\)](#): Tarea de control automático por temperatura
- [fan_start_schedule_task\(\)](#): Tarea de control por horarios (registros)
- [start_webserver\(\)](#): Crea tareas de procesamiento de comandos y gestión de sesiones

=== COLAS (QUEUES) ===

Las colas se crean en [web_server.c](#):

- `gpio_command_queue`: Para comandos desde handler HTTP a tarea de procesamiento

- `gpio_response_queue`: Para respuestas desde tarea de procesamiento a handler HTTP

=== SEMÁFOROS (MUTEXES) ===

Los semáforos se crean en [web_server.c](#):

- `session_mutex`: Protege acceso a sesiones de usuarios
- `command_id_mutex`: Protege contador de IDs de comandos

4.5.6 Documentación de funciones

4.5.6.1 `app_main()`

```
void app_main (
    void )
```

Función principal de la aplicación ESP32.

Se ejecuta automáticamente al iniciar el dispositivo. Inicializa todos los componentes del sistema en el orden correcto.

Nota

Orden de inicialización:

1. NVS (almacenamiento no volátil para configuración WiFi)
2. Hardware (LED RGB y sensor NTC de temperatura)
3. WiFi (configuración como Access Point)
4. Servidor Web (HTTP con SPIFFS y rutas)

Función principal de la aplicación ESP32 Se ejecuta automáticamente al iniciar el dispositivo

Orden de inicialización:

1. NVS (almacenamiento no volátil para configuración WiFi)
2. Hardware (LED RGB y sensor NTC de temperatura)
3. WiFi (configuración como Access Point)
4. Servidor Web (HTTP con SPIFFS y rutas)

4.5.7 Documentación de variables

4.5.7.1 TAG

```
const char* TAG = "MAIN" [static]
```

4.6 Referencia del archivo `main/ntc_sensor.c`

Implementación del driver para sensor de temperatura NTC.

```
#include "ntc_sensor.h"
#include "esp_log.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"
#include "hal/adc_types.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include <math.h>
```

defines

- #define `TEMP_ERROR_VALUE` -999.0
- #define `ADC_MAX_VALUE` 4096
- #define `ADC_MIN_VALUE` 0
- #define `RESISTANCE_MAX_OHMS` 1000000.0
- #define `SENSOR_READ_INTERVAL_MS` 1000
- #define `ADC_STABILIZATION_DELAY_MS` 100
- #define `NTC_TASK_STACK_SIZE` 4096
- #define `NTC_TASK_PRIORITY` 5

Funciones

- static bool `adc_calibration_init` (adc_unit_t unit, adc_atten_t atten, adc_cali_handle_t *out_handle)
Inicializa la calibración del ADC para obtener lecturas más precisas Intenta usar curve fitting primero, luego line fitting como fallback.
- void `ntc_sensor_init` (void)
Inicializa el ADC1 y configura el canal para leer el sensor NTC.
- `ntc_data_t ntc_read_temperature` (void)
Lee el valor del ADC, calcula la resistencia del NTC y convierte a temperatura.
- static void `ntc_reading_task` (void *arg)
Tarea de FreeRTOS que lee la temperatura periódicamente cada segundo.
- `ntc_data_t ntc_get_current_temperature` (void)
Obtiene la temperatura actual almacenada por la tarea de lectura Esta función es thread-safe y puede llamarse desde cualquier tarea.
- void `ntc_start_reading_task` (void)
Crea e inicia la tarea de FreeRTOS para lectura periódica de temperatura La tarea lee la temperatura cada segundo y actualiza los datos globales.

Variables

- static const char * `TAG` = "NTC_TEMP_CONTROL"
- static adc_oneshot_unit_handle_t `adc1_handle`
- static adc_cali_handle_t `adc1_cali_handle` = NULL
- static `ntc_data_t current_ntc_data` = {.temperature_c = `TEMP_ERROR_VALUE`, .resistance = 0.0, .raw_↵
adc_value = 0}
- static bool `data_ready` = false
- static SemaphoreHandle_t `data_mutex` = NULL

4.6.1 Descripción detallada

Implementación del driver para sensor de temperatura NTC.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del módulo de sensor de temperatura NTC (Negative Temperature Coefficient). Este módulo gestiona la lectura de temperatura usando un termistor NTC 10k conectado a través de un divisor de voltaje al ADC del ESP32.

4.6.2 Funcionalidades

- Inicializa el ADC1 del ESP32 para leer el voltaje del divisor de voltaje
- Calcula la resistencia del NTC basándose en el valor ADC
- Convierte la resistencia a temperatura usando la ecuación de Steinhart-Hart
- Ejecuta una tarea periódica que lee la temperatura cada segundo
- Proporciona acceso thread-safe a los datos de temperatura mediante mutex

4.6.3 Hardware

Circuito: VCC —[10k Resistor]—[NTC 10k]—GND | GPIO32 (ADC1_CH4)

Nota: ADC2 no funciona cuando WiFi está activo, por eso usamos ADC1

4.6.4 Características

- Calibración del ADC para lecturas más precisas (curve fitting o line fitting)
- Validación de rangos de ADC y resistencia
- Protección thread-safe con mutex para acceso concurrente
- Tarea independiente para lectura periódica (cada 1 segundo)

ARCHIVO: [ntc_sensor.c](#)

RESUMEN: Implementación del módulo de sensor de temperatura NTC (Negative Temperature Coefficient). Este módulo gestiona la lectura de temperatura usando un termistor NTC 10k conectado a través de un divisor de voltaje al ADC del ESP32.

Funcionalidades:

- Inicializa el ADC1 del ESP32 para leer el voltaje del divisor de voltaje
- Calcula la resistencia del NTC basándose en el valor ADC
- Convierte la resistencia a temperatura usando la ecuación de Steinhart-Hart
- Ejecuta una tarea periódica que lee la temperatura cada segundo
- Proporciona acceso thread-safe a los datos de temperatura mediante mutex

Hardware: Circuito: VCC —[10k Resistor]—[NTC 10k]—GND | GPIO32 (ADC1_CH4)

Nota: ADC2 no funciona cuando WiFi está activo, por eso usamos ADC1

Características:

- Calibración del ADC para lecturas más precisas (curve fitting o line fitting)
- Validación de rangos de ADC y resistencia
- Protección thread-safe con mutex para acceso concurrente
- Tarea independiente para lectura periódica (cada 1 segundo)

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 43 a 60 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 62 a 77 Sección 3: VARIABLES GLOBALES se encuentra en las líneas 79 a 87 Sección 4: CALIBRACIÓN DEL ADC se encuentra en las líneas 89 a 119 Sección 5: INICIALIZACIÓN se encuentra en las líneas 121 a 148 Sección 6: LECTURA Y CÁLCULO DE TEMPERATURA se encuentra en las líneas 150 a 203 Sección 7: TAREA DE LECTURA PERIÓDICA se encuentra en las líneas 205 a 249

Sección 8: FUNCIONES GETTER se encuentra en las líneas 251 a 289

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

1. ntc_reading_task (Sección 7)

- Nombre: "ntc_reader"
- Stack: 4096 bytes
- Prioridad: 5 (alta)
- Función: Lee la temperatura periódicamente cada segundo
- Propósito: Mantener datos de temperatura actualizados de forma continua
- Estado: Loop infinito, lee cada 1 segundo
- Flujo:
 - (a) Lee temperatura del sensor
 - (b) Valida que los datos sean válidos
 - (c) Almacena datos de forma thread-safe usando mutex
 - (d) Marca datos como "ready" cuando son válidos

=== COLAS (QUEUES) ===

Ninguna en este módulo.

=== SEMÁFOROS (MUTEXES) ===

1. data_mutex

- Tipo: SemaphoreHandle_t (FreeRTOS mutex)
- Protege: current_ntc_data y data_ready
- Uso: Protege acceso concurrente a los datos del sensor
- Operaciones: xSemaphoreTake() antes de leer/escribir, xSemaphoreGive() después
- Timeout: Variable (portMAX_DELAY en tarea, 100ms en getter)
- Funciones que lo usan:
 - [ntc_reading_task\(\)](#) - Escribe datos periódicamente
 - [ntc_get_current_temperature\(\)](#) - Lee datos de forma thread-safe

4.6.5 Documentación de «define»

4.6.5.1 ADC_MAX_VALUE

```
#define ADC_MAX_VALUE 4096
```

4.6.5.2 ADC_MIN_VALUE

```
#define ADC_MIN_VALUE 0
```

4.6.5.3 ADC_STABILIZATION_DELAY_MS

```
#define ADC_STABILIZATION_DELAY_MS 100
```

4.6.5.4 NTC_TASK_PRIORITY

```
#define NTC_TASK_PRIORITY 5
```

4.6.5.5 NTC_TASK_STACK_SIZE

```
#define NTC_TASK_STACK_SIZE 4096
```

4.6.5.6 RESISTANCE_MAX_OHMS

```
#define RESISTANCE_MAX_OHMS 1000000.0
```

4.6.5.7 SENSOR_READ_INTERVAL_MS

```
#define SENSOR_READ_INTERVAL_MS 1000
```

4.6.5.8 TEMP_ERROR_VALUE

```
#define TEMP_ERROR_VALUE -999.0
```

4.6.6 Documentación de funciones

4.6.6.1 adc_calibration_init()

```
static bool adc_calibration_init (
    adc_unit_t unit,
    adc_atten_t atten,
    adc_cali_handle_t * out_handle ) [static]
```

Inicializa la calibración del ADC para obtener lecturas más precisas. Intenta usar curve fitting primero, luego line fitting como fallback.

Parámetros

<i>unit</i>	Unidad ADC (ADC_UNIT_1 o ADC_UNIT_2)
<i>atten</i>	Atenuación del ADC
<i>out_handle</i>	Puntero donde se guardará el handle de calibración

Devuelve

true si la calibración fue exitosa, false en caso contrario

4.6.6.2 ntc_get_current_temperature()

```
ntc_data_t ntc_get_current_temperature (
    void )
```

Obtiene la temperatura actual almacenada por la tarea de lectura. Esta función es thread-safe y puede llamarse desde cualquier tarea.

Devuelve

Estructura `ntc_data_t` con los últimos datos leídos

4.6.6.3 ntc_read_temperature()

```
ntc_data_t ntc_read_temperature (
    void )
```

Lee el valor del ADC, calcula la resistencia del NTC y convierte a temperatura.

Proceso:

1. Lee el valor crudo del ADC (0-4095)
2. Calcula la resistencia del NTC usando la fórmula del divisor de voltaje: $R_{ntc} = R_{series} * (4095 / ADC_value - 1)$
3. Convierte resistencia a temperatura usando ecuación de Steinhart-Hart: $1/T = 1/T_0 + (1/B) * \ln(R/R_0)$

Devuelve

Estructura `ntc_data_t` con temperatura, resistencia y valor ADC

4.6.6.4 ntc_reading_task()

```
static void ntc_reading_task (
    void * arg ) [static]
```

Tarea de FreeRTOS que lee la temperatura periódicamente cada segundo.

- Lee la temperatura del sensor
- Valida que los datos estén en rangos razonables
- Almacena los datos de forma thread-safe usando mutex
- Marca los datos como "ready" cuando son válidos

Parámetros

<i>arg</i>	Argumentos de la tarea (no usado)
------------	-----------------------------------

4.6.6.5 ntc_sensor_init()

```
void ntc_sensor_init (
    void )
```

Inicializa el ADC1 y configura el canal para leer el sensor NTC.

- Crea el mutex para proteger acceso a datos
- Configura ADC1 con atenuación de 12dB (rango 0-3.3V)
- Inicializa la calibración del ADC si está disponible

Nota: Usamos ADC1 porque ADC2 no funciona cuando WiFi está activo

4.6.6.6 ntc_start_reading_task()

```
void ntc_start_reading_task (
    void )
```

Crea e inicia la tarea de FreeRTOS para lectura periódica de temperatura La tarea lee la temperatura cada segundo y actualiza los datos globales.

4.6.7 Documentación de variables

4.6.7.1 adc1_cali_handle

```
adc_cali_handle_t adc1_cali_handle = NULL [static]
```

4.6.7.2 adc1_handle

```
adc_one_shot_unit_handle_t adc1_handle [static]
```

4.6.7.3 current_ntc_data

```
ntc_data_t current_ntc_data = {.temperature_c = TEMP_ERROR_VALUE, .resistance = 0.0, .raw_↵
adc_value = 0} [static]
```

4.6.7.4 data_mutex

```
SemaphoreHandle_t data_mutex = NULL [static]
```

4.6.7.5 data_ready

```
bool data_ready = false [static]
```

4.6.7.6 TAG

```
const char* TAG = "NTC_TEMP_CONTROL" [static]
```

4.7 Referencia del archivo main/ntc_sensor.h

Driver para sensor de temperatura NTC (Negative Temperature Coefficient)

```
#include <stdint.h>
```

Estructuras de datos

- struct [ntc_data_t](#)

Estructura que contiene los datos del sensor NTC.

defines

- #define [NTC_PIN](#) ADC_CHANNEL_4
- #define [ADC_UNIT](#) ADC_UNIT_1
- #define [NOMINAL_RESISTANCE](#) 10000.0
- #define [NOMINAL_TEMPERATURE](#) 25.0
- #define [B_COEFFICIENT](#) 3380.0
- #define [SERIES_RESISTOR](#) 10000.0

Funciones

- void [ntc_sensor_init](#) (void)
Inicializa el ADC1 y configura el canal para leer el sensor NTC.
- [ntc_data_t ntc_read_temperature](#) (void)
Lee el valor del ADC, calcula la resistencia del NTC y convierte a temperatura.
- [ntc_data_t ntc_get_current_temperature](#) (void)
Obtiene la temperatura actual almacenada por la tarea de lectura Esta función es thread-safe y puede llamarse desde cualquier tarea.
- void [ntc_start_reading_task](#) (void)
Crea e inicia la tarea de FreeRTOS para lectura periódica de temperatura La tarea lee la temperatura cada segundo y actualiza los datos globales.

4.7.1 Descripción detallada

Driver para sensor de temperatura NTC (Negative Temperature Coefficient)

Autor

Proyecto Final RTOS

Fecha

2024

Header file para el módulo de sensor de temperatura NTC (Negative Temperature Coefficient). Este módulo gestiona la lectura de temperatura usando un termistor NTC 10k conectado a través de un divisor de voltaje al ADC del ESP32.

4.7.2 Hardware

- Sensor: NTC 10k termistor
- Pin: GPIO32 (ADC1 Channel 4)
- Nota: ADC2 no funciona cuando WiFi está activo, por eso usamos ADC1

4.7.3 Características

- Lectura periódica en tarea separada de FreeRTOS
- Cálculo de temperatura usando ecuación de Steinhart-Hart
- Protección thread-safe con mutex para acceso a datos

ARCHIVO: [ntc_sensor.h](#)

RESUMEN: Header file para el módulo de sensor de temperatura NTC (Negative Temperature Coefficient). Este módulo gestiona la lectura de temperatura usando un termistor NTC 10k conectado a través de un divisor de voltaje al ADC del ESP32.

Hardware:

- Sensor: NTC 10k termistor
- Pin: GPIO32 (ADC1 Channel 4)
- Nota: ADC2 no funciona cuando WiFi está activo, por eso usamos ADC1

Características:

- Lectura periódica en tarea separada de FreeRTOS
 - Cálculo de temperatura usando ecuación de Steinhart-Hart
 - Protección thread-safe con mutex para acceso a datos
-

ÍNDICE DE SECCIONES:

Sección 1: CONFIGURACIÓN DE PINES Y ADC se encuentra en las líneas 28 a 32 Sección 2: CONSTANTES DEL TERMISTOR NTC 10k se encuentra en las líneas 34 a 38 Sección 3: ESTRUCTURAS DE DATOS se encuentra en las líneas 40 a 48

Sección 4: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 50 a 60

4.7.4 Documentación de «define»

4.7.4.1 ADC_UNIT

```
#define ADC_UNIT ADC_UNIT_1
```

4.7.4.2 B_COEFFICIENT

```
#define B_COEFFICIENT 3380.0
```

4.7.4.3 NOMINAL_RESISTANCE

```
#define NOMINAL_RESISTANCE 10000.0
```

4.7.4.4 NOMINAL_TEMPERATURE

```
#define NOMINAL_TEMPERATURE 25.0
```

4.7.4.5 NTC_PIN

```
#define NTC_PIN ADC_CHANNEL_4
```

4.7.4.6 SERIES_RESISTOR

```
#define SERIES_RESISTOR 10000.0
```

4.7.5 Documentación de funciones

4.7.5.1 ntc_get_current_temperature()

```
ntc_data_t ntc_get_current_temperature (
    void )
```

Obtiene la temperatura actual almacenada por la tarea de lectura. Esta función es thread-safe y puede llamarse desde cualquier tarea.

Devuelve

Estructura `ntc_data_t` con los últimos datos leídos

4.7.5.2 ntc_read_temperature()

```
ntc_data_t ntc_read_temperature (
    void )
```

Lee el valor del ADC, calcula la resistencia del NTC y convierte a temperatura.

Proceso:

1. Lee el valor crudo del ADC (0-4095)
2. Calcula la resistencia del NTC usando la fórmula del divisor de voltaje: $R_{ntc} = R_{series} * (4095 / ADC_value - 1)$
3. Convierte resistencia a temperatura usando ecuación de Steinhart-Hart: $1/T = 1/T_0 + (1/B) * \ln(R/R_0)$

Devuelve

Estructura `ntc_data_t` con temperatura, resistencia y valor ADC

4.7.5.3 ntc_sensor_init()

```
void ntc_sensor_init (
    void )
```

Inicializa el ADC1 y configura el canal para leer el sensor NTC.

- Crea el mutex para proteger acceso a datos
- Configura ADC1 con atenuación de 12dB (rango 0-3.3V)
- Inicializa la calibración del ADC si está disponible

Nota: Usamos ADC1 porque ADC2 no funciona cuando WiFi está activo

4.7.5.4 ntc_start_reading_task()

```
void ntc_start_reading_task (
    void )
```

Crea e inicia la tarea de FreeRTOS para lectura periódica de temperatura. La tarea lee la temperatura cada segundo y actualiza los datos globales.

4.8 ntc_sensor.h

[Ir a la documentación de este archivo.](#)

```
00001
00050 #ifndef NTC_SENSOR_H
00051 #define NTC_SENSOR_H
00052
00053 #include <stdint.h>
00054
00055 // ===== CONFIGURACIÓN DE PINES Y ADC =====
00056 // Nota: ADC2 no funciona con WiFi activo, por lo que usamos ADC1
00057 // GPIO32 es ADC_CHANNEL_4 (ADC1) y funciona correctamente con WiFi
```

```

00058 #define NTC_PIN          ADC_CHANNEL_4    // GPIO32 es ADC_CHANNEL_4 (ADC1)
00059 #define ADC_UNIT          ADC_UNIT_1
00060
00061 // ===== CONSTANTES DEL TERMISTOR NTC 10k =====
00062 #define NOMINAL_RESISTANCE 10000.0 // Resistencia nominal a 25°C (en ohmios)
00063 #define NOMINAL_TEMPERATURE 25.0 // Temperatura nominal en Celsius
00064 #define B_COEFFICIENT      3380.0 // Coeficiente Beta del termistor (ajustado para NTC 10k)
00065 #define SERIES_RESISTOR    10000.0 // Valor de la resistencia en serie (10k ohmios)
00066
00067 // ===== ESTRUCTURAS DE DATOS =====
00071 typedef struct {
00072     float temperature_c; // Temperatura calculada en grados Celsius
00073     float resistance;     // Resistencia calculada del NTC en ohmios
00074     int raw_adc_value;    // Valor crudo leído del ADC (0-4095)
00075 } ntc_data_t;
00076
00077 // ===== PROTOTIPOS DE FUNCIONES =====
00078
00079 // Inicialización
00080 void ntc_sensor_init(void);
00081
00082 // Lectura de temperatura
00083 ntc_data_t ntc_read_temperature(void);
00084 ntc_data_t ntc_get_current_temperature(void);
00085
00086 // Gestión de tarea de lectura periódica
00087 void ntc_start_reading_task(void);
00088
00089 #endif // NTC_SENSOR_H

```

4.9 Referencia del archivo main/pir_driver.c

Implementación del driver para sensor PIR.

```

#include "pir_driver.h"
#include "driver/gpio.h"
#include "esp_attr.h"
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"

```

Funciones

- static void IRAM_ATTR [pir_isr_handler](#) (void *arg)
ISR (Interrupt Service Routine) del sensor PIR.
- bool [pir_init](#) (gpio_num_t pir_gpio, QueueHandle_t pir_queue)
Inicializa el driver del sensor PIR.
- bool [pir_is_motion_active](#) (void)
Lee el estado actual del sensor PIR.

Variables

- static const char * [TAG](#) = "PIR_DRIVER"
- static gpio_num_t [s_pir_gpio](#) = GPIO_NUM_NC
- static QueueHandle_t [s_pir_evt_queue](#) = NULL
- static bool [s_pir_initialized](#) = false

4.9.1 Descripción detallada

Implementación del driver para sensor PIR.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del driver del sensor PIR (Passive Infrared Sensor). Este módulo gestiona la detección de movimiento mediante interrupciones GPIO.

4.9.2 Hardware

- Sensor PIR: GPIO 12 (configurable mediante `pir_init()`)
- El sensor PIR detecta movimiento mediante cambios en radiación infrarroja

4.9.3 Características

- Detección de movimiento mediante interrupciones GPIO
- Soporte para cola de eventos opcional para notificaciones asíncronas
- Lectura síncrona del estado actual del sensor
- ISR (Interrupt Service Routine) thread-safe usando colas desde ISR
- Configuración automática de pull-up/pull-down (deshabilitados, el módulo PIR ya los tiene)

4.9.4 Arquitectura del sistema

- Interrupciones GPIO en ambos flancos (subida y bajada) para detectar cambios
- ISR envía eventos a cola de FreeRTOS (si está configurada) usando `xQueueSendFromISR()`
- Función síncrona para leer estado actual sin esperar eventos
- El ventilador utiliza este sensor para verificar presencia antes de activarse

4.9.5 Uso en el sistema

- El ventilador verifica presencia mediante `pir_is_motion_active()`
- En modo MANUAL, el ventilador ignora el PIR
- En otros modos, el ventilador solo funciona si hay presencia detectada

ARCHIVO: `pir_driver.c`

RESUMEN: Implementación del driver del sensor PIR (Passive Infrared Sensor). Este módulo gestiona la detección de movimiento mediante interrupciones GPIO.

Hardware:

- Sensor PIR: GPIO 12 (configurable mediante `pir_init()`)
- El sensor PIR detecta movimiento mediante cambios en radiación infrarroja

Características:

- Detección de movimiento mediante interrupciones GPIO
- Soporte para cola de eventos opcional para notificaciones asíncronas
- Lectura síncrona del estado actual del sensor
- ISR (Interrupt Service Routine) thread-safe usando colas desde ISR
- Configuración automática de pull-up/pull-down (deshabilitados, el módulo PIR ya los tiene)

Arquitectura:

- Interrupciones GPIO en ambos flancos (subida y bajada) para detectar cambios
- ISR envía eventos a cola de FreeRTOS (si está configurada) usando `xQueueSendFromISR()`
- Función síncrona para leer estado actual sin esperar eventos
- El ventilador utiliza este sensor para verificar presencia antes de activarse

Uso en el sistema:

- El ventilador verifica presencia mediante `pir_is_motion_active()`
- En modo MANUAL, el ventilador ignora el PIR
- En otros modos, el ventilador solo funciona si hay presencia detectada

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 37 a 47 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 49 a 50 Sección 3: VARIABLES ESTÁTICAS se encuentra en las líneas 52 a 55 Sección 4: FUNCIONES INTERNAS se encuentra en las líneas 57 a 94

Sección 5: FUNCIONES PÚBLICAS se encuentra en las líneas 96 a 219

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

Ninguna en este módulo. Las funciones se llaman desde otras tareas.

=== COLAS (QUEUES) ===

1. `s_pir_evt_queue` (opcional, configurada externamente)
 - Tipo: `QueueHandle_t` (FreeRTOS queue)
 - Tamaño: Configurado externamente
 - Elemento: `pir_event_t` (estructura con campo motion)
 - Dirección: ISR → Tarea externa
 - Operación: `xQueueSendFromISR()` desde ISR
 - Uso: Notificaciones asíncronas de cambios de movimiento (opcional)

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. `gpio_get_level()` es thread-safe.

4.9.6 Documentación de funciones

4.9.6.1 `pir_init()`

```
bool pir_init (
    gpio_num_t pir_gpio,
    QueueHandle_t pir_queue )
```

Inicializa el driver del sensor PIR.

Configura el pin GPIO como entrada con interrupciones en ambos flancos (subida y bajada) para detectar cambios en el estado del sensor.

Si se proporciona una cola de eventos, los eventos de movimiento se enviarán automáticamente a través de la cola cuando se detecten cambios. Esto permite procesamiento asíncrono de eventos de movimiento en una tarea separada.

Proceso de inicialización:

1. Valida que el pin GPIO sea válido
2. Configura el pin GPIO como entrada sin pull-up/pull-down
3. Configura interrupciones en ambos flancos (GPIO_INTR_ANYEDGE)
4. Instala el servicio de ISR de GPIO (si no está instalado)
5. Registra el handler de interrupción para el pin

Parámetros

<i>pir_gpio</i>	Número del pin GPIO donde está conectado el sensor PIR
<i>pir_queue</i>	Cola de eventos donde se enviarán las notificaciones de movimiento. Puede ser NULL si no se necesita notificación asíncrona. La cola debe ser creada previamente con <code>xQueueCreate()</code> .

Devuelve

true si la inicialización fue exitosa, false en caso contrario

Nota

La cola de eventos debe ser creada antes de llamar a esta función. Ejemplo: `QueueHandle_t pir_queue = xQueueCreate(10, sizeof(pir_event_t));`

Muchos módulos PIR ya incluyen electrónica interna (pull-up, filtrado), por lo que no se configuran pull-up/pull-down en el GPIO.

4.9.6.2 pir_is_motion_active()

```
bool pir_is_motion_active (
    void )
```

Lee el estado actual del sensor PIR.

Lee directamente el nivel del pin GPIO para determinar si hay movimiento detectado. Esta función es síncrona y puede ser llamada desde cualquier tarea.

Devuelve

true si hay movimiento detectado (GPIO en nivel alto), false si no hay movimiento (GPIO en nivel bajo) o si el driver no está inicializado

Nota

Esta función lee el estado actual del GPIO, no el último evento. Para recibir notificaciones de cambios, usar la cola de eventos en [pir_init\(\)](#).

Esta función es thread-safe porque `gpio_get_level()` es thread-safe.

4.9.6.3 pir_isr_handler()

```
static void IRAM_ATTR pir_isr_handler (
    void * arg ) [static]
```

ISR (Interrupt Service Routine) del sensor PIR.

Esta función se ejecuta cuando se detecta un cambio en el pin GPIO del sensor PIR (flanco de subida o bajada). Lee el nivel actual del GPIO y envía un evento a la cola de FreeRTOS si está configurada.

Características importantes:

- Debe estar marcada con `IRAM_ATTR` para ejecutarse desde IRAM (memoria rápida)
- Usa `xQueueSendFromISR()` para enviar eventos de forma thread-safe desde ISR
- Hace yield si una tarea de mayor prioridad fue despertada

Parámetros

<i>arg</i>	Argumento pasado al registrar el ISR (no usado)
------------	---

4.9.7 Documentación de variables

4.9.7.1 s_pir_evt_queue

```
QueueHandle_t s_pir_evt_queue = NULL [static]
```

4.9.7.2 s_pir_gpio

```
gpio_num_t s_pir_gpio = GPIO_NUM_NC [static]
```

4.9.7.3 s_pir_initialized

```
bool s_pir_initialized = false [static]
```

4.9.7.4 TAG

```
const char* TAG = "PIR_DRIVER" [static]
```

4.10 Referencia del archivo main/pir_driver.h

Driver para sensor PIR (Passive Infrared Sensor)

```
#include <stdint.h>
#include <stdbool.h>
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
```

Estructuras de datos

- struct [pir_event_t](#)

Estructura de evento del sensor PIR.

defines

- #define [PIR_GPIO_PIN](#) 12

Pin GPIO donde está conectado el sensor PIR Cambia este valor según tu conexión física.

Funciones

- bool [pir_init](#) (gpio_num_t pir_gpio, QueueHandle_t pir_queue)

Inicializa el driver del sensor PIR.

- bool [pir_is_motion_active](#) (void)

Lee el estado actual del sensor PIR.

4.10.1 Descripción detallada

Driver para sensor PIR (Passive Infrared Sensor)

Autor

Proyecto Final RTOS

Fecha

2024

Header file para el driver del sensor PIR (Passive Infrared Sensor). Este módulo gestiona la detección de movimiento mediante interrupciones GPIO.

4.10.2 Hardware

- Sensor PIR: GPIO 12 (configurable mediante PIR_GPIO_PIN)
- El sensor PIR detecta movimiento mediante cambios en radiación infrarroja

4.10.3 Características

- Detección de movimiento mediante interrupciones GPIO (flanco de subida/bajada)
- Soporte para cola de eventos opcional para notificaciones asíncronas
- Lectura síncrona del estado actual del sensor
- ISR (Interrupt Service Routine) thread-safe usando colas desde ISR
- Configuración automática de pull-up/pull-down (deshabilitados, el módulo PIR ya los tiene)

4.10.4 Uso en el sistema

- El ventilador verifica presencia mediante [pir_is_motion_active\(\)](#)
- En modo MANUAL, el ventilador ignora el PIR
- En otros modos, el ventilador solo funciona si hay presencia detectada

ARCHIVO: [pir_driver.h](#)

RESUMEN: Header file para el driver del sensor PIR (Passive Infrared Sensor). Este módulo gestiona la detección de movimiento mediante interrupciones GPIO.

Hardware:

- Sensor PIR: GPIO 12 (configurable mediante PIR_GPIO_PIN)
- El sensor PIR detecta movimiento mediante cambios en radiación infrarroja

Características:

- Detección de movimiento mediante interrupciones GPIO (flanco de subida/bajada)
- Soporte para cola de eventos opcional para notificaciones asíncronas
- Lectura síncrona del estado actual del sensor
- ISR (Interrupt Service Routine) thread-safe usando colas desde ISR
- Configuración automática de pull-up/pull-down (deshabilitados, el módulo PIR ya los tiene)

Uso en el sistema:

- El ventilador verifica presencia mediante [pir_is_motion_active\(\)](#)
- En modo MANUAL, el ventilador ignora el PIR
- En otros modos, el ventilador solo funciona si hay presencia detectada

ÍNDICE DE SECCIONES:

Sección 1: CONFIGURACIÓN DE PIN GPIO se encuentra en las líneas 35 a 40 Sección 2: ESTRUCTURAS DE DATOS se encuentra en las líneas 42 a 51

Sección 3: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 53 a 94

4.10.5 Documentación de «define»

4.10.5.1 PIR_GPIO_PIN

```
#define PIR_GPIO_PIN 12
```

Pin GPIO donde está conectado el sensor PIR Cambia este valor según tu conexión física.

4.10.6 Documentación de funciones

4.10.6.1 pir_init()

```
bool pir_init (
    gpio_num_t pir_gpio,
    QueueHandle_t pir_queue )
```

Inicializa el driver del sensor PIR.

Configura el pin GPIO como entrada con interrupciones en ambos flancos (subida y bajada) para detectar cambios en el estado del sensor.

Si se proporciona una cola de eventos, los eventos de movimiento se enviarán automáticamente a través de la cola cuando se detecten cambios. Esto permite procesamiento asíncrono de eventos de movimiento en una tarea separada.

Proceso de inicialización:

1. Configura el pin GPIO como entrada
2. Instala el servicio de ISR de GPIO (si no está instalado)
3. Registra el handler de interrupción para el pin

Parámetros

<i>pir_gpio</i>	Número del pin GPIO donde está conectado el sensor PIR
<i>pir_queue</i>	Cola de eventos donde se enviarán las notificaciones de movimiento. Puede ser NULL si no se necesita notificación asíncrona. La cola debe ser creada previamente con xQueueCreate().

Devuelve

true si la inicialización fue exitosa, false en caso contrario

Nota

La cola de eventos debe ser creada antes de llamar a esta función. Ejemplo: QueueHandle_t pir_queue = xQueueCreate(10, sizeof(pir_event_t));

Configura el pin GPIO como entrada con interrupciones en ambos flancos (subida y bajada) para detectar cambios en el estado del sensor.

Si se proporciona una cola de eventos, los eventos de movimiento se enviarán automáticamente a través de la cola cuando se detecten cambios. Esto permite procesamiento asíncrono de eventos de movimiento en una tarea separada.

Proceso de inicialización:

1. Valida que el pin GPIO sea válido
2. Configura el pin GPIO como entrada sin pull-up/pull-down
3. Configura interrupciones en ambos flancos (GPIO_INTR_ANYEDGE)
4. Instala el servicio de ISR de GPIO (si no está instalado)
5. Registra el handler de interrupción para el pin

Parámetros

<i>pir_gpio</i>	Número del pin GPIO donde está conectado el sensor PIR
<i>pir_queue</i>	Cola de eventos donde se enviarán las notificaciones de movimiento. Puede ser NULL si no se necesita notificación asíncrona. La cola debe ser creada previamente con <code>xQueueCreate()</code> .

Devuelve

true si la inicialización fue exitosa, false en caso contrario

Nota

La cola de eventos debe ser creada antes de llamar a esta función. Ejemplo: `QueueHandle_t pir_queue = xQueueCreate(10, sizeof(pir_event_t));`

Muchos módulos PIR ya incluyen electrónica interna (pull-up, filtrado), por lo que no se configuran pull-up/pull-down en el GPIO.

4.10.6.2 pir_is_motion_active()

```
bool pir_is_motion_active (
    void )
```

Lee el estado actual del sensor PIR.

Lee directamente el nivel del pin GPIO para determinar si hay movimiento detectado. Esta función es síncrona y puede ser llamada desde cualquier tarea.

Devuelve

true si hay movimiento detectado (GPIO en nivel alto), false si no hay movimiento (GPIO en nivel bajo)

Nota

Esta función lee el estado actual del GPIO, no el último evento. Para recibir notificaciones de cambios, usar la cola de eventos en [pir_init\(\)](#).

Lee directamente el nivel del pin GPIO para determinar si hay movimiento detectado. Esta función es síncrona y puede ser llamada desde cualquier tarea.

Devuelve

true si hay movimiento detectado (GPIO en nivel alto), false si no hay movimiento (GPIO en nivel bajo) o si el driver no está inicializado

Nota

Esta función lee el estado actual del GPIO, no el último evento. Para recibir notificaciones de cambios, usar la cola de eventos en [pir_init\(\)](#).

Esta función es thread-safe porque `gpio_get_level()` es thread-safe.

4.11 pir_driver.h

[Ir a la documentación de este archivo.](#)

```
00001
00059 #ifndef PIR_DRIVER_H
00060 #define PIR_DRIVER_H
00061
00062 #include <stdint.h>
00063 #include <stdbool.h>
00064 #include "driver/gpio.h"
00065 #include "freertos/FreeRTOS.h"
00066 #include "freertos/queue.h"
00067
00068 #ifdef __cplusplus
00069 extern "C" {
```

```

00070 #endif
00071
00072 // ===== CONFIGURACIÓN DE PIN GPIO =====
00073 #define PIR_GPIO_PIN 12 // GPIO 12 - Ajusta según tu hardware
00074
00075 // ===== ESTRUCTURAS DE DATOS =====
00076 typedef struct {
00077     bool motion;
00078 } pir_event_t;
00079
00080 // ===== PROTOTIPOS DE FUNCIONES =====
00081
00082 bool pir_init(gpio_num_t pir_gpio, QueueHandle_t pir_queue);
00083
00084 bool pir_is_motion_active(void);
00085
00086 #ifdef __cplusplus
00087 }
00088 #endif
00089
00090 #endif // PIR_DRIVER_H

```

4.12 Referencia del archivo main/registros.c

Implementación de la gestión de registros de horarios.

```

#include "registros.h"
#include <esp_log.h>
#include <cJSON.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdint.h>
#include <inttypes.h>

```

Funciones

- static int64_t `get_time_ms` (void)
Obtiene el tiempo actual en milisegundos desde el inicio del sistema.
- void `crear_archivo_si_no_existe` (void)
Crea el archivo registros.json en SPIFFS si no existe Inicializa el archivo con un array JSON vacío [] Esta función debe llamarse durante la inicialización del servidor web para asegurar que el archivo existe antes de intentar leer/escribir registros.
- bool `agregar_registro` (const char *dia, const char *hora, int velocidad)
Agrega un registro al archivo registros.json en SPIFFS Lee el archivo completo, agrega el nuevo registro al array JSON, y guarda el archivo actualizado de forma persistente.
- char * `leer_registros_json` (void)
Lee todos los registros del archivo registros.json desde SPIFFS Retorna un string JSON con el array completo de registros almacenados.
- static void `normalizar_dia` (const char *dia, char *output, size_t output_size)
Convierte el nombre del día en español a minúsculas normalizado.
- `registro_activo_t verificar_registro_activo` (const char *dia_actual, const char *hora_actual)
Verifica si hay un registro activo para el día y hora actuales Lee todos los registros desde SPIFFS y busca coincidencias exactas de día y hora.

Variables

- static const char * `TAG` = "REGISTROS"

4.12.1 Descripción detallada

Implementación de la gestión de registros de horarios.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación de la gestión de registros de horarios del ventilador. Este módulo proporciona funciones para almacenar y leer registros de forma persistente en la partición SPIFFS del ESP32.

4.12.2 Funcionalidades

- Creación automática del archivo registros.json si no existe
- Guardado de registros en formato JSON en /spiffs/registros.json
- Lectura de todos los registros almacenados
- Verificación de registros activos para el día y hora actual
- Persistencia de datos tras reinicio del ESP32

4.12.3 Uso en el sistema

Los registros se utilizan desde el módulo [web_server.c](#) a través de los endpoints HTTP GET /registros y POST /registros, y desde [fan_control.c](#) para el modo SCHEDULE del ventilador.

4.12.4 Formato de registro

```
{
  "dia": "lunes",
  "hora": "14:30",
  "velocidad": 50,
  "id": "1234567890"
}
```

ARCHIVO: [registros.c](#)

RESUMEN: Implementación de la gestión de registros de horarios del ventilador. Este módulo proporciona funciones para almacenar y leer registros de forma persistente en la partición SPIFFS del ESP32.

Funcionalidades:

- Creación automática del archivo registros.json si no existe
- Guardado de registros en formato JSON en /spiffs/registros.json
- Lectura de todos los registros almacenados
- Verificación de registros activos para el día y hora actual
- Persistencia de datos tras reinicio del ESP32

Los registros se utilizan desde el módulo [web_server.c](#) a través de los endpoints HTTP GET /registros y POST /registros, y desde [fan_control.c](#) para el modo SCHEDULE del ventilador.

Formato de registro: { "dia": "lunes", "hora": "14:30", "velocidad": 50, "id": "1234567890" }

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 44 a 60 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 62 a 63 Sección 3: FUNCIONES INTERNAS se encuentra en las líneas 65 a 76

Sección 4: FUNCIONES PÚBLICAS se encuentra en las líneas 78 a 325

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

Ninguna en este módulo. Las funciones se llaman desde otras tareas.

=== COLAS (QUEUES) ===

Ninguna en este módulo.

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. El acceso a archivos SPIFFS es thread-safe mediante las funciones estándar de C (fopen, fread, fwrite).

4.12.5 Documentación de funciones

4.12.5.1 agregar_registro()

```
bool agregar_registro (
    const char * dia,
    const char * hora,
    int velocidad )
```

Agrega un registro al archivo registros.json en SPIFFS Lee el archivo completo, agrega el nuevo registro al array JSON, y guarda el archivo actualizado de forma persistente.

Agrega un registro al archivo registros.json.

Parámetros

<i>dia</i>	Día de la semana (ej: "lunes", "martes", etc.)
<i>hora</i>	Hora en formato HH:MM (ej: "14:30")
<i>velocidad</i>	Velocidad del ventilador (0-100)

Devuelve

true si éxito, false si error (archivo no existe, error de escritura, etc.)

Nota: El registro incluye un ID único basado en timestamp para compatibilidad con el frontend que espera un campo "id" en cada registro.

4.12.5.2 crear_archivo_si_no_existe()

```
void crear_archivo_si_no_existe (
    void )
```

Crea el archivo registros.json en SPIFFS si no existe Inicializa el archivo con un array JSON vacío [] Esta función debe llamarse durante la inicialización del servidor web para asegurar que el archivo existe antes de intentar leer/escribir registros.

Crea el archivo registros.json si no existe Inicializa con un array JSON vacío [].

4.12.5.3 get_time_ms()

```
static int64_t get_time_ms (
    void ) [static]
```

Obtiene el tiempo actual en milisegundos desde el inicio del sistema.

Devuelve

Tiempo en milisegundos

4.12.5.4 leer_registros_json()

```
char * leer_registros_json (
    void )
```

Lee todos los registros del archivo registros.json desde SPIFFS Retorna un string JSON con el array completo de registros almacenados.

Lee todos los registros del archivo registros.json.

Devuelve

String JSON con todos los registros (debe ser liberado con free()) Retorna "[]" (array vacío) si hay error, el archivo no existe o está vacío

Formato del JSON retornado: [{"dia": "lunes", "hora": "14:30", "velocidad": 50, "id": "1234567890"}, {"dia": "martes", "hora": "08:00", "velocidad": 75, "id": "1234567891"}, ...]

4.12.5.5 normalizar_dia()

```
static void normalizar_dia (
    const char * dia,
    char * output,
    size_t output_size ) [static]
```

Convierte el nombre del día en español a minúsculas normalizado.

Parámetros

<i>dia</i>	Día en español (puede tener mayúsculas)
<i>output</i>	Buffer donde se almacenará el día normalizado
<i>output_size</i>	Tamaño del buffer

4.12.5.6 verificar_registro_activo()

```
registro_activo_t verificar_registro_activo (
    const char * dia_actual,
    const char * hora_actual )
```

Verifica si hay un registro activo para el día y hora actuales Lee todos los registros desde SPIFFS y busca coincidencias exactas de día y hora.

Verifica si hay un registro activo para el día y hora actuales.

Si encuentra una coincidencia, retorna la velocidad del registro.

Parámetros

<i>dia_actual</i>	Día actual en español (ej: "lunes", "martes", etc.)
<i>hora_actual</i>	Hora actual en formato HH:MM (ej: "14:30")

Devuelve

`registro_activo_t` con la velocidad si hay registro activo, o velocidad=0 y activo=false si no hay coincidencia

4.12.6 Documentación de variables

4.12.6.1 TAG

```
const char* TAG = "REGISTROS" [static]
```

4.13 Referencia del archivo main/registros.h

Gestión de registros de horarios del ventilador.

```
#include <stdbool.h>
```

Estructuras de datos

- struct [registro_activo_t](#)

Estructura para representar un registro activo.

Funciones

- void [crear_archivo_si_no_existe](#) (void)
Crea el archivo registros.json si no existe Inicializa con un array JSON vacío [].
- bool [agregar_registro](#) (const char *dia, const char *hora, int velocidad)
Agrega un registro al archivo registros.json.
- char * [leer_registros_json](#) (void)
Lee todos los registros del archivo registros.json.
- [registro_activo_t](#) [verificar_registro_activo](#) (const char *dia_actual, const char *hora_actual)
Verifica si hay un registro activo para el día y hora actuales.

4.13.1 Descripción detallada

Gestión de registros de horarios del ventilador.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para la gestión de registros de horarios del ventilador. Proporciona funciones para guardar y leer registros desde SPIFFS de forma persistente. Los registros se almacenan en /spiffs/registros.json.

4.13.2 Uso en el sistema

Este módulo es utilizado por:

- [web_server.c](#): Para manejar los endpoints HTTP GET /registros y POST /registros
- [fan_control.c](#): Para verificar registros activos en modo SCHEDULE

4.13.3 Formato de registro

```
{  
  "dia": "lunes",  
  "hora": "14:30",  
  "velocidad": 50,  
  "id": "1234567890"  
}
```

ARCHIVO: [registros.h](#)

RESUMEN: Header file para la gestión de registros de horarios del ventilador. Proporciona funciones para guardar y leer registros desde SPIFFS de forma persistente. Los registros se almacenan en /spiffs/registros.json.

Este módulo es utilizado por:

- [web_server.c](#): Para manejar los endpoints HTTP GET /registros y POST /registros
- [fan_control.c](#): Para verificar registros activos en modo SCHEDULE

Formato de registro: { "dia": "lunes", "hora": "14:30", "velocidad": 50, "id": "1234567890" }

ÍNDICE DE SECCIONES:

Sección 1: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 25 a 58

4.13.4 Documentación de funciones

4.13.4.1 agregar_registro()

```
bool agregar_registro (
    const char * dia,
    const char * hora,
    int velocidad )
```

Agrega un registro al archivo registros.json.

Parámetros

<i>dia</i>	Día de la semana (ej: "lunes")
<i>hora</i>	Hora en formato HH:MM (ej: "14:30")
<i>velocidad</i>	Velocidad del ventilador (0-100)

Devuelve

true si éxito, false si error

Agrega un registro al archivo registros.json.

Parámetros

<i>dia</i>	Día de la semana (ej: "lunes", "martes", etc.)
<i>hora</i>	Hora en formato HH:MM (ej: "14:30")
<i>velocidad</i>	Velocidad del ventilador (0-100)

Devuelve

true si éxito, false si error (archivo no existe, error de escritura, etc.)

Nota: El registro incluye un ID único basado en timestamp para compatibilidad con el frontend que espera un campo "id" en cada registro.

4.13.4.2 crear_archivo_si_no_existe()

```
void crear_archivo_si_no_existe (
    void )
```

Crea el archivo registros.json si no existe Inicializa con un array JSON vacío [].

Crea el archivo registros.json si no existe Inicializa con un array JSON vacío [].

4.13.4.3 leer_registros_json()

```
char * leer_registros_json (
    void )
```

Lee todos los registros del archivo registros.json.

Devuelve

String JSON con todos los registros (debe ser liberado con free()) Retorna "[]" si hay error o el archivo está vacío

Lee todos los registros del archivo registros.json.

Devuelve

String JSON con todos los registros (debe ser liberado con free()) Retorna "[]" (array vacío) si hay error, el archivo no existe o está vacío

Formato del JSON retornado: [{"dia": "lunes", "hora": "14:30", "velocidad": 50, "id": "1234567890"}, {"dia": "martes", "hora": "08:00", "velocidad": 75, "id": "1234567891"}, ...]

4.13.4.4 verificar_registro_activo()

```
registro_activo_t verificar_registro_activo (
    const char * dia_actual,
    const char * hora_actual )
```

Verifica si hay un registro activo para el día y hora actuales.

Parámetros

<i>dia_actual</i>	Día actual en español (ej: "lunes", "martes", etc.)
<i>hora_actual</i>	Hora actual en formato HH:MM (ej: "14:30")

Devuelve

registro_activo_t con la velocidad si hay registro activo, o velocidad=0 y activo=false si no

Verifica si hay un registro activo para el día y hora actuales.
Si encuentra una coincidencia, retorna la velocidad del registro.

Parámetros

<i>dia_actual</i>	Día actual en español (ej: "lunes", "martes", etc.)
<i>hora_actual</i>	Hora actual en formato HH:MM (ej: "14:30")

Devuelve

registro_activo_t con la velocidad si hay registro activo, o velocidad=0 y activo=false si no hay coincidencia

4.14 registros.h

[Ir a la documentación de este archivo.](#)

```
00001
00054 #ifndef REGISTROS_H
00055 #define REGISTROS_H
00056
00057 #include <stdbool.h>
00058
00063 void crear_archivo_si_no_existe(void);
00064
00072 bool agregar_registro(const char *dia, const char *hora, int velocidad);
00073
00079 char *leer_registros_json(void);
00080
00084 typedef struct {
00085     int velocidad; // Velocidad del ventilador (0-100)
00086     bool activo; // Si hay un registro activo para el momento actual
00087 } registro_activo_t;
00088
00095 registro_activo_t verificar_registro_activo(const char *dia_actual, const char *hora_actual);
00096
00097 #endif // REGISTROS_H
```

4.15 Referencia del archivo main/rgb_led.c

Implementación del controlador de LED RGB.

```
#include "rgb_led.h"
#include "driver/ledc.h"
#include "esp_err.h"
#include "esp_log.h"
```

defines

- #define `GPIO_GREEN` 27
- #define `LEDC_TIMER` LEDC_TIMER_0
- #define `LEDC_MODE` LEDC_LOW_SPEED_MODE
- #define `LEDC_CHANNEL_G` LEDC_CHANNEL_1
- #define `LEDC_DUTY_RES` LEDC_TIMER_8_BIT
- #define `LEDC_FREQUENCY` 5000

Funciones

- bool `rgb_led_init` (void)
Inicializa el controlador PWM para el LED RGB verde.
- void `rgb_set_green_percent` (uint8_t percent)
Establece el brillo del LED verde como porcentaje.

Variables

- static const char * `TAG` = "RGB_LED"
- static bool `rgb_led_initialized` = false

4.15.1 Descripción detallada

Implementación del controlador de LED RGB.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del controlador de LED RGB (actualmente solo canal verde). Este módulo gestiona el control PWM del LED RGB mediante el periférico LEDC del ESP32.

4.15.2 Hardware

- LED Verde: GPIO 27 (PWM mediante LEDC Channel 1, Timer 0)

4.15.3 Características

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 5kHz (adecuada para LEDs sin parpadeo visible)
- Interfaz simple con porcentaje (0-100%)
- Validación de parámetros y manejo de errores

ARCHIVO: [rgb_led.c](#)

RESUMEN: Implementación del controlador de LED RGB (actualmente solo canal verde). Este módulo gestiona el control PWM del LED RGB mediante el periférico LEDC del ESP32.

Hardware:

- LED Verde: GPIO 27 (PWM mediante LEDC Channel 1, Timer 0)

Características:

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 5kHz (adecuada para LEDs sin parpadeo visible)
- Interfaz simple con porcentaje (0-100%)
- Validación de parámetros y manejo de errores

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 33 a 40 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 42 a 50 Sección 3: VARIABLES ESTÁTICAS se encuentra en las líneas 52 a 53 Sección 4: FUNCIONES DE INICIALIZACIÓN se encuentra en las líneas 55 a 104

Sección 5: FUNCIONES DE CONTROL DEL LED se encuentra en las líneas 106 a 145

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

Ninguna en este módulo. Las funciones se llaman directamente desde otras tareas.

=== COLAS (QUEUES) ===

Ninguna en este módulo.

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. El acceso a las funciones es thread-safe porque las funciones de LEDC son thread-safe.

4.15.4 Documentación de «define»**4.15.4.1 GPIO_GREEN**

```
#define GPIO_GREEN 27
```

4.15.4.2 LEDC_CHANNEL_G

```
#define LEDC_CHANNEL_G LEDC_CHANNEL_1
```

4.15.4.3 LEDC_DUTY_RES

```
#define LEDC_DUTY_RES LEDC_TIMER_8_BIT
```

4.15.4.4 LEDC_FREQUENCY

```
#define LEDC_FREQUENCY 5000
```

4.15.4.5 LEDC_MODE

```
#define LEDC_MODE LEDC_LOW_SPEED_MODE
```

4.15.4.6 LEDC_TIMER

```
#define LEDC_TIMER LEDC_TIMER_0
```

4.15.5 Documentación de funciones

4.15.5.1 `rgb_led_init()`

```
bool rgb_led_init (
    void )
```

Inicializa el controlador PWM para el LED RGB verde.

Configura el timer LEDC y el canal PWM para controlar el LED verde. Esta función debe llamarse una vez durante la inicialización del sistema antes de usar cualquier otra función de este módulo.

Proceso de inicialización:

1. Configura el timer LEDC con resolución de 8 bits y frecuencia de 5kHz
2. Configura el canal LEDC para el GPIO del LED verde
3. Inicia el LED con brillo 0% (apagado)

Devuelve

true si la inicialización fue exitosa, false en caso contrario

4.15.5.2 `rgb_set_green_percent()`

```
void rgb_set_green_percent (
    uint8_t percent )
```

Establece el brillo del LED verde como porcentaje.

Convierte el porcentaje (0-100) a valor PWM (0-255) y actualiza el LED. Valores fuera de rango se limitan automáticamente a 0-100.

Nota: Esta función requiere que `rgb_led_init()` haya sido llamada previamente. Si no está inicializado, la función no realizará ninguna acción.

Parámetros

<code>percent</code>	Porcentaje de brillo (0-100). Valores >100 se limitan a 100.
----------------------	--

4.15.6 Documentación de variables

4.15.6.1 `rgb_led_initialized`

```
bool rgb_led_initialized = false [static]
```

4.15.6.2 `TAG`

```
const char* TAG = "RGB_LED" [static]
```

4.16 Referencia del archivo `main/rgb_led.h`

Controlador de LED RGB mediante PWM.

```
#include <stdint.h>
#include <stdbool.h>
```

Funciones

- bool `rgb_led_init` (void)
Inicializa el controlador PWM para el LED RGB verde.
- void `rgb_set_green_percent` (uint8_t percent)
Establece el brillo del LED verde como porcentaje.

4.16.1 Descripción detallada

Controlador de LED RGB mediante PWM.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para el controlador de LED RGB (actualmente solo canal verde). Este módulo gestiona el control PWM del LED RGB conectado al ESP32 mediante el periférico LEDC (LED Controller).

4.16.2 Hardware

- LED Verde: GPIO 27 (PWM mediante LEDC Channel 1, Timer 0)

4.16.3 Características

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 5kHz (adecuada para LEDs sin parpadeo visible)
- Interfaz simple con porcentaje (0-100%)
- Validación automática de parámetros

ARCHIVO: [rgb_led.h](#)

RESUMEN: Header file para el controlador de LED RGB (actualmente solo canal verde). Este módulo gestiona el control PWM del LED RGB conectado al ESP32 mediante el periférico LEDC (LED Controller).

Hardware:

- LED Verde: GPIO 27 (PWM mediante LEDC Channel 1, Timer 0)

Características:

- Control PWM de 8 bits (0-255 niveles)
- Frecuencia: 5kHz (adecuada para LEDs sin parpadeo visible)
- Interfaz simple con porcentaje (0-100%)
- Validación automática de parámetros

ÍNDICE DE SECCIONES:

Sección 1: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 40 a 51

4.16.4 Documentación de funciones

4.16.4.1 `rgb_led_init()`

```
bool rgb_led_init (  
    void )
```

Inicializa el controlador PWM para el LED RGB verde.

Configura el timer LEDC y el canal PWM para controlar el LED verde. Debe llamarse una vez durante la inicialización del sistema.

Devuelve

true si la inicialización fue exitosa, false en caso contrario

Configura el timer LEDC y el canal PWM para controlar el LED verde. Esta función debe llamarse una vez durante la inicialización del sistema antes de usar cualquier otra función de este módulo.

Proceso de inicialización:

1. Configura el timer LEDC con resolución de 8 bits y frecuencia de 5kHz
2. Configura el canal LEDC para el GPIO del LED verde
3. Inicia el LED con brillo 0% (apagado)

Devuelve

true si la inicialización fue exitosa, false en caso contrario

4.16.4.2 rgb_set_green_percent()

```
void rgb_set_green_percent (
    uint8_t percent )
```

Establece el brillo del LED verde como porcentaje.

Convierte el porcentaje (0-100) a valor PWM (0-255) y actualiza el LED. Valores fuera de rango se limitan automáticamente.

Parámetros

<i>percent</i>	Porcentaje de brillo (0-100)
----------------	------------------------------

Convierte el porcentaje (0-100) a valor PWM (0-255) y actualiza el LED. Valores fuera de rango se limitan automáticamente a 0-100.

Nota: Esta función requiere que [rgb_led_init\(\)](#) haya sido llamada previamente. Si no está inicializado, la función no realizará ninguna acción.

Parámetros

<i>percent</i>	Porcentaje de brillo (0-100). Valores >100 se limitan a 100.
----------------	--

4.17 rgb_led.h

[Ir a la documentación de este archivo.](#)

```
00001
00045 #ifndef RGB_LED_H
00046 #define RGB_LED_H
00047
00048 #include <stdint.h>
00049 #include <stdbool.h>
00050
00051 #ifdef __cplusplus
00052 extern "C" {
00053 #endif
00054
00055 // ===== PROTOTIPOS DE FUNCIONES =====
00056
00065 bool rgb_led_init(void);
00066
00075 void rgb_set_green_percent(uint8_t percent);
00076
00077 #ifdef __cplusplus
00078 }
00079 #endif
00080
00081 #endif // RGB_LED_H
```

4.18 Referencia del archivo main/terminal_commands.c

Implementación del procesamiento de comandos de terminal.

```
#include "terminal_commands.h"
#include "rgb_led.h"
#include "fan_control.h"
#include <esp_log.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include <string.h>
#include <stdlib.h>
```

Estructuras de datos

- struct [terminal_context_t](#)

Funciones

- void [process_terminal_command](#) ([gpio_command_t](#) *cmd)
Procesa un comando de terminal y genera la respuesta correspondiente.
- void [terminal_command_task](#) (void *pvParameters)
Tarea de FreeRTOS que procesa comandos GPIO recibidos desde la web Lee comandos de la cola, los ejecuta y envía respuestas.

Variables

- static const char * [TAG](#) = "TERMINAL_CMD"

4.18.1 Descripción detallada

Implementación del procesamiento de comandos de terminal.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del módulo de procesamiento de comandos de terminal. Este módulo gestiona el procesamiento de comandos recibidos desde la terminal web, incluyendo comandos para control de LEDs, ventilador y comandos del sistema.

4.18.2 Funcionalidades

- Procesa comandos de la terminal (led, fan, status, help, clear, etc.)
- Ejecuta acciones correspondientes (control de GPIO, PWM para LED y ventilador)
- Genera respuestas apropiadas para cada comando

4.18.3 Arquitectura del sistema

Utiliza el patrón Productor-Consumidor con colas de FreeRTOS para comunicación asíncrona entre el handler HTTP y la tarea de procesamiento.

ARCHIVO: [terminal_commands.c](#)

RESUMEN: Implementación del módulo de procesamiento de comandos de terminal. Este módulo gestiona el procesamiento de comandos recibidos desde la terminal web, incluyendo comandos para control de LEDs, ventilador y comandos del sistema.

Funcionalidad:

- Procesa comandos de la terminal (led, fan, status, help, clear, etc.)
- Ejecuta acciones correspondientes (control de GPIO, PWM para LED y ventilador)
- Genera respuestas apropiadas para cada comando

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 30 a 45 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 47 a 50 Sección 3: FUNCIÓN DE PROCESAMIENTO DE COMANDOS se encuentra en las líneas 52 a 100

Sección 4: TAREA DE PROCESAMIENTO DE COMANDOS se encuentra en las líneas 102 a 130**RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:**

=== TAREAS (TASKS) ===

1. `terminal_command_task` (Sección 4)

- Nombre: "gpio_cmd_task" (configurado en [web_server.c](#))
- Stack: 4096 bytes
- Prioridad: 5 (alta)
- Función: Procesa comandos GPIO recibidos desde la terminal web
- Propósito: Ejecutar comandos de forma asíncrona sin bloquear el servidor HTTP
- Estado: Loop infinito, bloquea esperando comandos (`portMAX_DELAY`)
- Flujo:
 - (a) Recibe comando de `gpio_command_queue` (`xQueueReceive`, bloqueante)
 - (b) Procesa comando con [process_terminal_command\(\)](#)
 - (c) Envía respuesta a `gpio_response_queue` (`xQueueSend`)

=== COLAS (QUEUES) ===

1. `gpio_command_queue` (recibida en contexto)

- Tipo: `QueueHandle_t`
- Tamaño: `GPIO_QUEUE_SIZE` (10 elementos)
- Elemento: [gpio_command_t](#)
- Dirección: Handler HTTP → Esta tarea
- Operación: `xQueueReceive()` con `portMAX_DELAY` (bloqueante)
- Uso: Esta tarea recibe comandos de la cola para procesarlos

2. `gpio_response_queue` (recibida en contexto)

- Tipo: `QueueHandle_t`
- Tamaño: `GPIO_QUEUE_SIZE` (10 elementos)
- Elemento: [gpio_command_t](#) (con respuesta generada)
- Dirección: Esta tarea → Handler HTTP
- Operación: `xQueueSend()` con timeout de 100ms
- Uso: Esta tarea envía respuestas procesadas a la cola

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. Los semáforos se gestionan en [web_server.c](#)

ARQUITECTURA:

Esta tarea implementa el patrón Producer-Consumer:

- Producer: Handler HTTP /cmd (envía comandos a gpio_command_queue)
- Consumer: terminal_command_task (recibe y procesa comandos)
- Producer: terminal_command_task (envía respuestas a gpio_response_queue)
- Consumer: Handler HTTP /cmd (recibe respuestas)

Ventajas:

- Servidor HTTP no se bloquea esperando procesamiento
- Múltiples comandos pueden estar en cola simultáneamente
- Thread-safe gracias a las colas de FreeRTOS
- Permite manejar picos de tráfico sin perder comandos

4.18.4 Documentación de funciones

4.18.4.1 process_terminal_command()

```
void process_terminal_command (
    gpio_command_t * cmd )
```

Procesa un comando de terminal y genera la respuesta correspondiente.

Parámetros

<i>cmd</i>	Estructura con el comando a procesar (se modifica in-place con la respuesta)
------------	--

Comandos soportados:

Control de LED RGB:

- led on : Enciende el LED RGB verde (100% brillo)
- led off : Apaga el LED RGB verde (0% brillo)
- led <0-100> : Establece el brillo del LED RGB verde (0-100%)

Control de Ventilador:

- fan on : Enciende el ventilador al 50% de velocidad (modo manual)
- fan off : Apaga el ventilador (modo OFF)
- fan <0-100> : Establece la velocidad del ventilador manualmente (0-100%, modo manual)

Sistema:

- status : Estado del sistema (LED RGB y ventilador)
- help : Lista de comandos disponibles
- clear : Limpiar pantalla (manejado en frontend)

4.18.4.2 terminal_command_task()

```
void terminal_command_task (
    void * pvParameters )
```

Tarea de FreeRTOS que procesa comandos GPIO recibidos desde la web Lee comandos de la cola, los ejecuta y envía respuestas.

Parámetros

<i>pvParameters</i>	Puntero al contexto del servidor web (terminal_context_t)
---------------------	---

===== EXPLICACIÓN DE LA TAREA ===== Esta es una tarea de FreeRTOS que se ejecuta de forma independiente para procesar comandos de la terminal web de manera asíncrona.

Arquitectura:

1. El handler HTTP (/cmd) recibe un comando del cliente web
2. El handler envía el comando a `gpio_command_queue` (no bloquea)
3. Esta tarea recibe el comando de la cola (bloqueante)
4. Procesa el comando usando [process_terminal_command\(\)](#)
5. Envía la respuesta a `gpio_response_queue`
6. El handler HTTP recibe la respuesta y la envía al cliente

Ventajas de esta arquitectura:

- El servidor HTTP no se bloquea esperando procesamiento
- Múltiples comandos pueden estar en cola simultáneamente
- El procesamiento es thread-safe gracias a las colas
- Permite manejar picos de tráfico sin perder comandos

===== USO DE COLAS ===== `gpio_command_queue` (entrada):

- `xQueueReceive()` espera indefinidamente (`portMAX_DELAY`) hasta recibir un comando
- Cuando recibe un comando, lo copia a la variable local 'cmd'
- La cola es thread-safe: múltiples handlers HTTP pueden enviar simultáneamente

`gpio_response_queue` (salida):

- `xQueueSend()` envía la respuesta con el mismo `command_id`
- Timeout de 100ms: si la cola está llena, espera máximo 100ms
- Si falla, registra un warning pero continúa (el handler HTTP manejará el timeout)

===== FLUJO DE DATOS ===== Cliente Web -> Handler HTTP -> `gpio_command_queue` -> Esta Tarea -> [process_terminal_command\(\)](#) -> `gpio_response_queue` -> Handler HTTP -> Cliente Web

Prioridad: 5 (configurada en [web_server.c](#)) Stack: 4096 bytes (suficiente para procesamiento de comandos)

4.18.5 Documentación de variables

4.18.5.1 TAG

```
const char* TAG = "TERMINAL_CMD" [static]
```

4.19 Referencia del archivo main/terminal_commands.h

Procesamiento de comandos de terminal web.

```
#include <stdint.h>
#include <stdbool.h>
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
```

Estructuras de datos

- struct [gpio_command_t](#)

Funciones

- void [process_terminal_command](#) ([gpio_command_t](#) *cmd)
Procesa un comando de terminal y genera la respuesta correspondiente.
- void [terminal_command_task](#) (void *pvParameters)
Tarea de FreeRTOS que procesa comandos GPIO recibidos desde la web Lee comandos de la cola, los ejecuta y envía respuestas.

4.19.1 Descripción detallada

Procesamiento de comandos de terminal web.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para el módulo de procesamiento de comandos de terminal. Este módulo gestiona el procesamiento de comandos recibidos desde la terminal web, incluyendo comandos para control de LEDs, ventilador y comandos del sistema.

4.19.2 Funcionalidades

- Procesa comandos de la terminal (led, fan, status, help, clear, etc.)
- Ejecuta acciones correspondientes (control de GPIO, PWM para LED y ventilador)
- Genera respuestas apropiadas para cada comando

4.19.3 Arquitectura del sistema

Utiliza colas de FreeRTOS para comunicación asíncrona entre el handler HTTP y la tarea de procesamiento de comandos.

ARCHIVO: [terminal_commands.h](#)

RESUMEN: Header file para el módulo de procesamiento de comandos de terminal. Este módulo gestiona el procesamiento de comandos recibidos desde la terminal web, incluyendo comandos para control de LEDs, ventilador y comandos del sistema.

Funcionalidad:

- Procesa comandos de la terminal (led, fan, status, help, clear, etc.)
- Ejecuta acciones correspondientes (control de GPIO, PWM para LED y ventilador)
- Genera respuestas apropiadas para cada comando

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES Y DEFINICIONES se encuentra en las líneas 30 a 40

Sección 2: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 42 a 50

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

1. [terminal_command_task\(\)](#)

- Definida en: [terminal_commands.c](#)
- Creada en: [web_server.c](#) ([start_webserver\(\)](#))
- Nombre: "gpio_cmd_task"
- Stack: 4096 bytes
- Prioridad: 5
- Función: Procesa comandos de terminal de forma asíncrona

=== COLAS (QUEUES) ===

1. [gpio_command_queue](#)

- Tipo: [QueueHandle_t](#)
- Tamaño: 10 elementos ([GPIO_QUEUE_SIZE](#))
- Elemento: [gpio_command_t](#)
- Uso: Recibe comandos desde handler HTTP

2. [gpio_response_queue](#)

- Tipo: [QueueHandle_t](#)
- Tamaño: 10 elementos ([GPIO_QUEUE_SIZE](#))
- Elemento: [gpio_command_t](#)
- Uso: Envía respuestas al handler HTTP

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. Los semáforos se gestionan en [web_server.c](#)

4.19.4 Documentación de funciones

4.19.4.1 [process_terminal_command\(\)](#)

```
void process_terminal_command (
    gpio\_command\_t * cmd )
```

Procesa un comando de terminal y genera la respuesta correspondiente.

Parámetros

<i>cmd</i>	Estructura con el comando a procesar (se modifica in-place con la respuesta)
------------	--

Comandos soportados:

Control de LED RGB:

- led on : Enciende el LED RGB verde (100% brillo)
- led off : Apaga el LED RGB verde (0% brillo)
- led <0-100> : Establece el brillo del LED RGB verde (0-100%)

Control de Ventilador:

- fan on : Enciende el ventilador al 50% de velocidad (modo manual)
- fan off : Apaga el ventilador (modo OFF)

- fan <0-100> : Establece la velocidad del ventilador manualmente (0-100%, modo manual)

Sistema:

- status : Estado del sistema (LED RGB y ventilador)
- help : Lista de comandos disponibles
- clear : Limpiar pantalla (manejado en frontend)

4.19.4.2 terminal_command_task()

```
void terminal_command_task (
    void * pvParameters )
```

Tarea de FreeRTOS que procesa comandos GPIO recibidos desde la web Lee comandos de la cola, los ejecuta y envía respuestas.

Parámetros

<i>pvParameters</i>	Puntero al contexto del servidor web (terminal_context_t) Debe contener gpio_command_queue y gpio_response_queue
---------------------	--

===== ARQUITECTURA DE TAREAS Y COLAS ===== Esta función implementa una tarea de FreeRTOS que procesa comandos de forma asíncrona.

Flujo de comunicación mediante colas:

1. Handler HTTP recibe comando del cliente web
2. Handler envía comando a gpio_command_queue (xQueueSend)
3. Esta tarea recibe comando de gpio_command_queue (xQueueReceive, bloqueante)
4. Procesa comando con [process_terminal_command\(\)](#)
5. Envía respuesta a gpio_response_queue (xQueueSend)
6. Handler HTTP recibe respuesta de gpio_response_queue (xQueueReceive)
7. Handler envía respuesta al cliente web

Ventajas:

- El servidor HTTP no se bloquea esperando procesamiento
- Múltiples comandos pueden estar en cola simultáneamente
- Thread-safe gracias a las colas de FreeRTOS
- Permite manejar picos de tráfico sin perder comandos

Configuración (en [web_server.c](#)):

- Stack: 4096 bytes
- Prioridad: 5 (alta, para procesamiento rápido)
- Nombre: "gpio_cmd_task"

Parámetros

<i>pvParameters</i>	Puntero al contexto del servidor web (terminal_context_t)
---------------------	---

===== EXPLICACIÓN DE LA TAREA ===== Esta es una tarea de FreeRTOS que se ejecuta de forma

independiente para procesar comandos de la terminal web de manera asíncrona.
Arquitectura:

1. El handler HTTP (/cmd) recibe un comando del cliente web
2. El handler envía el comando a gpio_command_queue (no bloquea)
3. Esta tarea recibe el comando de la cola (bloqueante)
4. Procesa el comando usando [process_terminal_command\(\)](#)
5. Envía la respuesta a gpio_response_queue
6. El handler HTTP recibe la respuesta y la envía al cliente

Ventajas de esta arquitectura:

- El servidor HTTP no se bloquea esperando procesamiento
- Múltiples comandos pueden estar en cola simultáneamente
- El procesamiento es thread-safe gracias a las colas
- Permite manejar picos de tráfico sin perder comandos

===== USO DE COLAS ===== gpio_command_queue (entrada):

- xQueueReceive() espera indefinidamente (portMAX_DELAY) hasta recibir un comando
- Cuando recibe un comando, lo copia a la variable local 'cmd'
- La cola es thread-safe: múltiples handlers HTTP pueden enviar simultáneamente

gpio_response_queue (salida):

- xQueueSend() envía la respuesta con el mismo command_id
- Timeout de 100ms: si la cola está llena, espera máximo 100ms
- Si falla, registra un warning pero continúa (el handler HTTP manejará el timeout)

===== FLUJO DE DATOS ===== Cliente Web -> Handler HTTP -> gpio_command_queue -> Esta Tarea -> [process_terminal_command\(\)](#) -> gpio_response_queue -> Handler HTTP -> Cliente Web

Prioridad: 5 (configurada en [web_server.c](#)) Stack: 4096 bytes (suficiente para procesamiento de comandos)

4.20 terminal_commands.h

[Ir a la documentación de este archivo.](#)

```
00001
00076 #pragma once
00077
00078 #ifndef __cplusplus
00079 extern "C" {
00080 #endif
00081
00082 // ===== INCLUDES =====
00083 #include <stdint.h>
00084 #include <stdbool.h>
00085 #include "freertos/FreeRTOS.h"
00086 #include "freertos/queue.h"
00087
00088 // ===== DEFINICIONES =====
00089 // Estructura para comandos GPIO
00090 // Esta estructura se usa para comunicar comandos entre el handler HTTP y la tarea de procesamiento
00091 // mediante colas (queues) de FreeRTOS
00092 typedef struct {
00093     uint32_t command_id; // ID único para emparejar comando-respuesta
00094                               // Permite que múltiples comandos estén en proceso simultáneamente
00095     char command[100]; // y que cada handler HTTP reciba la respuesta correcta
00096     char response[512]; // Texto del comando recibido (ej: "led on", "fan 50", "status", "help")
00097                               // Respuesta generada por process_terminal_command()
00098                               // Se llena después de procesar el comando
00099 } gpio_command_t;
00100
```

```

00101 // ===== PROTOTIPOS DE FUNCIONES =====
00124 void process_terminal_command(gpio_command_t *cmd);
00125
00156 void terminal_command_task(void *pvParameters);
00157
00158 #ifdef __cplusplus
00159 }
00160 #endif

```

4.21 Referencia del archivo main/time_sync.c

Implementación de la sincronización de tiempo mediante SNTP.

```

#include "time_sync.h"
#include "esp_log.h"
#include "esp_netif_sntp.h"
#include "lwip/apps/sntp.h"
#include "nvs.h"
#include "esp_system.h"
#include "esp_timer.h"
#include <time.h>
#include <string.h>
#include <sys/time.h>

```

Funciones

- static void `guardar_hora_en_nvs` (time_t timestamp)
Guarda la hora actual en NVS para restaurarla después de un reinicio.
- static bool `cargar_hora_desde_nvs` (void)
Carga la hora guardada en NVS y la restaura ajustada por el tiempo transcurrido.
- static void `sntp_sync_time_cb` (struct timeval *tv)
Callback que se ejecuta cuando SNTP sincroniza la hora.
- void `time_sync_init` (void)
Inicializa el servicio SNTP para sincronizar la hora del sistema Configura servidores NTP y espera a que la hora se sincronice.
- bool `hora_sincronizada` (void)
Verifica si la hora del sistema está sincronizada.
- bool `obtener_dia_actual` (char *dia_buffer, size_t buffer_size)
Obtiene el día de la semana actual en español.
- bool `obtener_hora_actual` (char *hora_buffer, size_t buffer_size)
Obtiene la hora actual en formato HH:MM.
- bool `establecer_hora_manual` (int year, int month, int day, int hour, int minute, int second)
Establece la hora manualmente (útil cuando SNTP no puede sincronizar) Útil cuando el ESP32 está en modo SoftAP sin conexión a Internet.

Variables

- static const char * `TAG` = "TIME_SYNC"
- static const char * `NVS_NAMESPACE` = "time_sync"
- static const char * `NVS_KEY_TIME` = "saved_time"
- static bool `sntp_initialized` = false
- static bool `time_synced` = false
- static const char * `dias_semana` []

4.21.1 Descripción detallada

Implementación de la sincronización de tiempo mediante SNTP.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación de la sincronización de tiempo usando SNTP (Simple Network Time Protocol). Este módulo permite obtener la hora y fecha actual del sistema, necesaria para el sistema de horarios del ventilador.

4.21.2 Funcionalidades

- Sincronización automática con servidores NTP (co.pool.ntp.org)
- Configuración de zona horaria para Colombia (UTC-5)
- Persistencia de hora en NVS para restaurar después de reinicio
- Establecimiento manual de hora cuando SNTP no está disponible
- Obtención de día de la semana y hora actual en formato español

4.21.3 Características

- Restauración automática de hora desde NVS al reiniciar
- Fallback a hora manual si SNTP no puede sincronizar
- Soporte para zona horaria de Colombia (COT5)
- Validación de sincronización antes de usar funciones de tiempo

ARCHIVO: [time_sync.c](#)

RESUMEN: Implementación de la sincronización de tiempo usando SNTP (Simple Network Time Protocol). Este módulo permite obtener la hora y fecha actual del sistema, necesaria para el sistema de horarios del ventilador.
Funcionalidades:

- Sincronización automática con servidores NTP (co.pool.ntp.org)
- Configuración de zona horaria para Colombia (UTC-5)
- Persistencia de hora en NVS para restaurar después de reinicio
- Establecimiento manual de hora cuando SNTP no está disponible
- Obtención de día de la semana y hora actual en formato español

Características:

- Restauración automática de hora desde NVS al reiniciar
 - Fallback a hora manual si SNTP no puede sincronizar
 - Soporte para zona horaria de Colombia (COT5)
 - Validación de sincronización antes de usar funciones de tiempo
-

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 24 a 36 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 38 a 48 Sección 3: FUNCIONES INTERNAS se encuentra en las líneas 50 a 158

Sección 4: FUNCIONES PÚBLICAS se encuentra en las líneas 160 a 337**RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:**

=== TAREAS (TASKS) ===

Ninguna en este módulo. SNTP funciona mediante callbacks y eventos.

=== COLAS (QUEUES) ===

Ninguna en este módulo.

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. Las funciones de tiempo de C estándar son thread-safe.

4.21.4 Documentación de funciones**4.21.4.1 cargar_hora_desde_nvs()**

```
static bool cargar_hora_desde_nvs (
    void ) [static]
```

Carga la hora guardada en NVS y la restaura ajustada por el tiempo transcurrido.

Devuelve

true si se restauró la hora, false si no había hora guardada

4.21.4.2 establecer_hora_manual()

```
bool establecer_hora_manual (
    int year,
    int month,
    int day,
    int hour,
    int minute,
    int second )
```

Establece la hora manualmente (útil cuando SNTP no puede sincronizar) Útil cuando el ESP32 está en modo SoftAP sin conexión a Internet.

Establece la hora manualmente (útil cuando SNTP no puede sincronizar)

4.21.4.3 guardar_hora_en_nvs()

```
static void guardar_hora_en_nvs (
    time_t timestamp ) [static]
```

Guarda la hora actual en NVS para restaurarla después de un reinicio.

4.21.4.4 hora_sincronizada()

```
bool hora_sincronizada (
    void )
```

Verifica si la hora del sistema está sincronizada.

Devuelve

true si está sincronizada, false en caso contrario

4.21.4.5 obtener_dia_actual()

```
bool obtener_dia_actual (
    char * dia_buffer,
    size_t buffer_size )
```

Obtiene el día de la semana actual en español.

Parámetros

<i>dia_buffer</i>	Buffer donde se almacenará el día (ej: "lunes", "martes")
<i>buffer_size</i>	Tamaño del buffer

Devuelve

true si éxito, false si la hora no está sincronizada

4.21.4.6 obtener_hora_actual()

```
bool obtener_hora_actual (
    char * hora_buffer,
    size_t buffer_size )
```

Obtiene la hora actual en formato HH:MM.

Parámetros

<i>hora_buffer</i>	Buffer donde se almacenará la hora (ej: "14:30")
<i>buffer_size</i>	Tamaño del buffer

Devuelve

true si éxito, false si la hora no está sincronizada

4.21.4.7 sntp_sync_time_cb()

```
static void sntp_sync_time_cb (
    struct timeval * tv ) [static]
```

Callback que se ejecuta cuando SNTP sincroniza la hora.

4.21.4.8 time_sync_init()

```
void time_sync_init (
    void )
```

Inicializa el servicio SNTP para sincronizar la hora del sistema Configura servidores NTP y espera a que la hora se sincronice.

Inicializa el servicio SNTP para sincronizar la hora del sistema Debe llamarse después de que WiFi esté conectado.

4.21.5 Documentación de variables**4.21.5.1 dias_semana**

```
const char* dias_semana[] [static]
```

Valor inicial:

```
= {
    "domingo", "lunes", "martes", "miercoles", "jueves", "viernes", "sabado"
}
```

4.21.5.2 NVS_KEY_TIME

```
const char* NVS_KEY_TIME = "saved_time" [static]
```

4.21.5.3 NVS_NAMESPACE

```
const char* NVS_NAMESPACE = "time_sync" [static]
```

4.21.5.4 `sntp_initialized`

```
bool sntp_initialized = false [static]
```

4.21.5.5 `TAG`

```
const char* TAG = "TIME_SYNC" [static]
```

4.21.5.6 `time_synced`

```
bool time_synced = false [static]
```

4.22 Referencia del archivo `main/time_sync.h`

Sincronización de tiempo mediante SNTP.

```
#include <stdbool.h>
```

```
#include <stddef.h>
```

Funciones

- void `time_sync_init` (void)
Inicializa el servicio SNTP para sincronizar la hora del sistema Debe llamarse después de que WiFi esté conectado.
- bool `obtener_dia_actual` (char *dia_buffer, size_t buffer_size)
Obtiene el día de la semana actual en español.
- bool `obtener_hora_actual` (char *hora_buffer, size_t buffer_size)
Obtiene la hora actual en formato HH:MM.
- bool `hora_sincronizada` (void)
Verifica si la hora del sistema está sincronizada.
- bool `establecer_hora_manual` (int year, int month, int day, int hour, int minute, int second)
Establece la hora manualmente (útil cuando SNTP no puede sincronizar)

4.22.1 Descripción detallada

Sincronización de tiempo mediante SNTP.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para la sincronización de tiempo usando SNTP (Simple Network Time Protocol). Este módulo permite obtener la hora y fecha actual del sistema, necesaria para el sistema de horarios del ventilador.

4.22.2 Características

- Sincronización automática con servidores NTP
- Configuración de zona horaria para Colombia (UTC-5)
- Persistencia de hora en NVS para restaurar después de reinicio
- Establecimiento manual de hora cuando SNTP no está disponible
- Obtención de día de la semana y hora actual en formato español

ARCHIVO: [time_sync.h](#)

RESUMEN: Header file para la sincronización de tiempo usando SNTP (Simple Network Time Protocol). Este módulo permite obtener la hora y fecha actual del sistema, necesaria para el sistema de horarios del ventilador. Funcionalidades:

- Sincronización automática con servidores NTP
- Configuración de zona horaria para Colombia (UTC-5)
- Persistencia de hora en NVS para restaurar después de reinicio
- Establecimiento manual de hora cuando SNTP no está disponible
- Obtención de día de la semana y hora actual en formato español

ÍNDICE DE SECCIONES:

Sección 1: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 24 a 58

4.22.3 Documentación de funciones**4.22.3.1 establecer_hora_manual()**

```
bool establecer_hora_manual (
    int year,
    int month,
    int day,
    int hour,
    int minute,
    int second )
```

Establece la hora manualmente (útil cuando SNTP no puede sincronizar)

Parámetros

<i>year</i>	Año (ej: 2024)
<i>month</i>	Mes (1-12)
<i>day</i>	Día (1-31)
<i>hour</i>	Hora (0-23)
<i>minute</i>	Minuto (0-59)
<i>second</i>	Segundo (0-59)

Devuelve

true si éxito, false si error

Establece la hora manualmente (útil cuando SNTP no puede sincronizar)

4.22.3.2 hora_sincronizada()

```
bool hora_sincronizada (
    void )
```

Verifica si la hora del sistema está sincronizada.

Devuelve

true si está sincronizada, false en caso contrario

4.22.3.3 obtener_dia_actual()

```
bool obtener_dia_actual (
    char * dia_buffer,
    size_t buffer_size )
```

Obtiene el día de la semana actual en español.

Parámetros

<i>dia_buffer</i>	Buffer donde se almacenará el día (ej: "lunes", "martes")
<i>buffer_size</i>	Tamaño del buffer

Devuelve

true si éxito, false si la hora no está sincronizada

4.22.3.4 obtener_hora_actual()

```
bool obtener_hora_actual (
    char * hora_buffer,
    size_t buffer_size )
```

Obtiene la hora actual en formato HH:MM.

Parámetros

<i>hora_buffer</i>	Buffer donde se almacenará la hora (ej: "14:30")
<i>buffer_size</i>	Tamaño del buffer

Devuelve

true si éxito, false si la hora no está sincronizada

4.22.3.5 time_sync_init()

```
void time_sync_init (
    void )
```

Inicializa el servicio SNTP para sincronizar la hora del sistema Debe llamarse después de que WiFi esté conectado.
Inicializa el servicio SNTP para sincronizar la hora del sistema Debe llamarse después de que WiFi esté conectado.

4.23 time_sync.h

[Ir a la documentación de este archivo.](#)

```
00001
00041 #ifndef TIME_SYNC_H
00042 #define TIME_SYNC_H
00043
00044 #include <stdbool.h>
00045 #include <stddef.h>
00046
00051 void time_sync_init(void);
00052
00059 bool obtener_dia_actual(char *dia_buffer, size_t buffer_size);
00060
00067 bool obtener_hora_actual(char *hora_buffer, size_t buffer_size);
00068
00073 bool hora_sincronizada(void);
00074
```

```
00085 bool establecer_hora_manual(int year, int month, int day, int hour, int minute, int second);
00086
00087 #endif // TIME_SYNC_H
00088
```

4.24 Referencia del archivo main/web_server.c

Implementación del servidor web HTTP del ESP32.

```
#include "web_server.h"
#include "rgb_led.h"
#include "ntc_sensor.h"
#include "registros.h"
#include "time_sync.h"
#include "terminal_commands.h"
#include "fan_control.h"
#include "pir_driver.h"
#include <esp_http_server.h>
#include <esp_spiffs.h>
#include <esp_log.h>
#include <cJSON.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "lwip/inet.h"
#include "lwip/sockets.h"
#include <sys/stat.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdint.h>
#include <inttypes.h>
#include <errno.h>
#include <math.h>
```

Estructuras de datos

- struct [session_t](#)
- struct [webserver_context_t](#)

defines

- #define [MAX_SESSIONS](#) 5
- #define [SESSION_TIMEOUT_MS](#) (3 * 60 * 1000)
- #define [VALID_USER](#) "root"
- #define [VALID_PASS](#) "matrix123"
- #define [GPIO_QUEUE_SIZE](#) 10
- #define [CMD_RESPONSE_TIMEOUT_MS](#) 500
- #define [CMD_RESPONSE_MAX_ATTEMPTS](#) 10
- #define [CHECK_ESP_OK](#)(err, msg)
- #define [CHECK_AUTH](#)(ctx, req)
- #define [SEND_JSON_RESPONSE_STR](#)(req, json_str)
- #define [SEND_JSON_RESPONSE](#)(req, json_buf)
- #define [REDIRECT](#)(req, location)

Funciones

- static int64_t [get_time_ms](#) (void)
Obtiene el tiempo actual en milisegundos desde el inicio del sistema.
- static void [gpio_command_task_wrapper](#) (void *pvParameters)
Wrapper para la tarea de procesamiento de comandos Esta función adapta el contexto del servidor web al contexto de terminal y llama a la función del módulo terminal_commands.
- static void [session_management_task](#) (void *pvParameters)
Tarea que verifica periódicamente las sesiones y expira las inactivas Apaga los LEDs cuando una sesión expira por timeout.
- static void [get_client_ip](#) (httpd_req_t *req, char *ip_str, size_t len)
Obtiene la dirección IP del cliente desde la request HTTP.
- static [session_t](#) * [find_or_create_session](#) ([webserver_context_t](#) *ctx, const char *ip)
Busca una sesión existente o crea una nueva para una IP.
- static bool [is_authenticated](#) ([webserver_context_t](#) *ctx, httpd_req_t *req)
Verifica si el cliente está autenticado y actualiza la última actividad.
- esp_err_t [send_file_from_spiffs](#) (httpd_req_t *req, const char *filepath, const char *type)
Sirve un archivo estático desde SPIFFS al cliente HTTP Lee el archivo en chunks y lo envía de forma eficiente.
- static esp_err_t [root_get_handler](#) (httpd_req_t *req)
Handler para GET / (raíz) Si el usuario está autenticado, redirige al dashboard Si no, muestra la página de login.
- static esp_err_t [dashboard_get_handler](#) (httpd_req_t *req)
Handler para GET /dashboard Requiere autenticación, redirige a login si no está autenticado.
- static esp_err_t [terminal_get_handler](#) (httpd_req_t *req)
Handler para GET /terminal Requiere autenticación, redirige a login si no está autenticado.
- static esp_err_t [slider_get_handler](#) (httpd_req_t *req)
Handler para GET /slider Requiere autenticación, redirige a login si no está autenticado.
- static esp_err_t [style_get_handler](#) (httpd_req_t *req)
Handler para GET /style.css Sirve el archivo CSS desde SPIFFS.
- static esp_err_t [script_get_handler](#) (httpd_req_t *req)
Handler para GET /script.js Sirve el archivo JavaScript desde SPIFFS.
- static esp_err_t [favicon_get_handler](#) (httpd_req_t *req)
Handler para GET /favicon.ico Responde con 204 No Content para evitar errores 404.
- static esp_err_t [registros_get_handler](#) (httpd_req_t *req)
Handler para GET /registros Devuelve todos los registros almacenados en formato JSON desde SPIFFS Utiliza la función [leer_registros_json\(\)](#) del módulo [registros.c](#) Respuesta: Array JSON con todos los registros [{"dia": "...", "hora": "...", "velocidad": ...}, ...] Requiere autenticación.
- static esp_err_t [registros_post_handler](#) (httpd_req_t *req)
Handler para POST /registros Agrega un nuevo registro al archivo registros.json en SPIFFS Utiliza la función [agregar_registro\(\)](#) del módulo [registros.c](#) Body esperado: {"dia": "lunes", "hora": "14:30", "velocidad": 50} El registro se guarda de forma persistente en /spiffs/registros.json Requiere autenticación.
- static esp_err_t [temperature_get_handler](#) (httpd_req_t *req)
Handler para GET /temperature Devuelve la temperatura actual en formato JSON Respuesta: {"temperature": 25.5} o {"error": "No data available"} Requiere autenticación.
- static esp_err_t [fan_mode_post_handler](#) (httpd_req_t *req)
Handler para POST /fan/mode Establece el modo de operación del ventilador Body esperado: {"mode": "manual"} o {"mode": "off"} o {"mode": "temperature"} Requiere autenticación.
- static esp_err_t [fan_manual_post_handler](#) (httpd_req_t *req)
Handler para POST /fan/manual Establece el porcentaje de velocidad en modo manual Body esperado: {"percent": 50} (0-100) Requiere autenticación.
- static esp_err_t [fan_status_get_handler](#) (httpd_req_t *req)
Handler para GET /fan/status Devuelve el estado actual del ventilador en formato JSON Respuesta: {"mode": "manual", "percent": 50} Requiere autenticación.
- static esp_err_t [pir_status_get_handler](#) (httpd_req_t *req)

- Handler para GET /pir/status Devuelve el estado actual del sensor PIR en formato JSON Respuesta: {"motion": true} o {"motion": false} Requiere autenticación.*
- static void `url_decode` (char *str)
 - Decodifica una cadena URL-encoded (convierte %20 y + a espacios)*
- static void `str_to_lower` (char *str)
 - Convierte una cadena a minúsculas (modifica in-place)*
- static bool `extract_form_param` (const char *content, const char *param_name, char *output, size_t output_size)
 - Extrae un parámetro de una cadena form-urlencoded.*
- static esp_err_t `cmd_get_handler` (httpd_req_t *req)
 - Handler para GET /cmd?c=comando Procesa comandos desde la terminal web Envía el comando a la cola y espera la respuesta Requiere autenticación.*
- static esp_err_t `login_post_handler` (httpd_req_t *req)
 - Handler para POST /login Valida credenciales y crea una sesión autenticada Parámetros: user=xxx&pass=yyy (form-urlencoded)*
- static esp_err_t `logout_get_handler` (httpd_req_t *req)
 - Handler para GET /logout Cierra la sesión del usuario y apaga los LEDs.*
- static esp_err_t `fan_diagnostic_get_handler` (httpd_req_t *req)
 - Handler para GET /fan/diagnostic Endpoint de diagnóstico que muestra el estado completo del sistema de registros Útil para depurar problemas con el modo SCHEDULE Requiere autenticación.*
- static esp_err_t `time_set_post_handler` (httpd_req_t *req)
 - Handler para POST /time/set Establece la hora manualmente Body esperado: {"year": 2024, "month": 12, "day": 9, "hour": 14, "minute": 30, "second": 0} Requiere autenticación.*
- static int `verify_spiffs_files` (void)
 - Verifica que todos los archivos necesarios estén presentes en SPIFFS.*
- static esp_err_t `init_spiffs` (void)
 - Inicializa y monta el sistema de archivos SPIFFS.*
- static esp_err_t `register_http_routes` (webserver_context_t *ctx)
 - Registra todas las rutas HTTP del servidor web.*
- void `start_webserver` (void)
 - Función principal que inicia el servidor web.*

Variables

- static const char * `TAG` = "WEB_SERVER"

4.24.1 Descripción detallada

Implementación del servidor web HTTP del ESP32.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del servidor web HTTP del ESP32. Este módulo proporciona:

4.24.1.1 Sistema de archivos SPIFFS

- Monta partición SPIFFS para servir archivos estáticos (HTML, CSS, JS)
- Verifica que todos los archivos necesarios estén presentes

4.24.1.2 Servidor HTTP con múltiples rutas

- Páginas web: login, dashboard, terminal, slider
- API REST: /cmd (comandos), /temperature (datos JSON), /registros (gestión de registros)
- Autenticación: /login, /logout

4.24.1.3 Sistema de autenticación y sesiones

- Login con usuario/contraseña (root/matrix123)
- Gestión de sesiones basada en IP
- Timeout automático después de 3 minutos de inactividad

4.24.1.4 Procesamiento de comandos

- Sistema de colas para comandos GPIO (thread-safe)
- Tarea dedicada para procesar comandos
- Soporte para comandos: led on/off/<0-100>, status, help, clear

4.24.1.5 Gestión de temperatura

- Endpoint JSON para obtener temperatura actual
- Datos actualizados desde la tarea de lectura del sensor

4.24.1.6 Gestión de registros de horarios

- Endpoints HTTP para guardar y leer registros (/registros)
- Utiliza el módulo [registros.c](#) para almacenamiento persistente en SPIFFS
- Los registros se guardan en /spiffs/registros.json

ARCHIVO: [web_server.c](#)

RESUMEN: Implementación del servidor web HTTP del ESP32. Este módulo proporciona:

1. Sistema de archivos SPIFFS:
 - Monta partición SPIFFS para servir archivos estáticos (HTML, CSS, JS)
 - Verifica que todos los archivos necesarios estén presentes
2. Servidor HTTP con múltiples rutas:
 - Páginas web: login, dashboard, terminal, slider
 - API REST: /cmd (comandos), /temperature (datos JSON), /registros (gestión de registros)
 - Autenticación: /login, /logout
3. Sistema de autenticación y sesiones:
 - Login con usuario/contraseña (root/matrix123)
 - Gestión de sesiones basada en IP
 - Timeout automático después de 3 minutos de inactividad
4. Procesamiento de comandos:
 - Sistema de colas para comandos GPIO (thread-safe)
 - Tarea dedicada para procesar comandos

- Soporte para comandos: led on/off/<0-100>, status, help, clear
5. Gestión de temperatura:
 - Endpoint JSON para obtener temperatura actual
 - Datos actualizados desde la tarea de lectura del sensor
 6. Gestión de registros de horarios:
 - Endpoints HTTP para guardar y leer registros (/registros)
 - Utiliza el módulo [registros.c](#) para almacenamiento persistente en SPIFFS
 - Los registros se guardan en /spiffs/registros.json

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 34 a 62 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 64 a 76 Sección 3: ESTRUCTURAS DE DATOS se encuentra en las líneas 78 a 103 Sección 4: UTILIDADES GENERALES se encuentra en las líneas 105 a 112 Sección 5: TAREA DE PROCESAMIENTO DE COMANDOS se encuentra en las líneas 114 a 188 Sección 6: TAREA DE GESTIÓN DE SESIONES se encuentra en las líneas 190 a 218 Sección 7: FUNCIONES DE GESTIÓN DE SESIONES se encuentra en las líneas 220 a 318 Sección 8: UTILIDADES DE ARCHIVOS se encuentra en las líneas 323 a 358 Sección 9: HANDLERS HTTP se encuentra en las líneas 360 a 769 Sección 10: INICIALIZACIÓN DE SPIFFS se encuentra en las líneas 771 a 869 Sección 11: REGISTRO DE RUTAS HTTP se encuentra en las líneas 871 a 928

Sección 12: INICIALIZACIÓN DEL SERVIDOR se encuentra en las líneas 930 a 1010

RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:

=== TAREAS (TASKS) ===

1. gpio_command_task_wrapper (Sección 5)
 - Nombre: "gpio_cmd_task"
 - Stack: 4096 bytes
 - Prioridad: 5 (alta)
 - Función: Wrapper que adapta el contexto del servidor web y llama a [terminal_command_task\(\)](#) del módulo [terminal_commands.c](#)
 - Propósito: Procesar comandos de la terminal web de forma asíncrona
 - Estado: Loop infinito, bloquea esperando comandos de la cola
2. session_management_task (Sección 6)
 - Nombre: "session_mgmt"
 - Stack: 2048 bytes
 - Prioridad: 3 (media)
 - Función: Verifica periódicamente las sesiones y expira las inactivas
 - Propósito: Gestión automática del ciclo de vida de sesiones
 - Estado: Loop infinito, verifica cada 2 segundos
 - Acciones: Expira sesiones inactivas > 3 minutos, apaga LEDs automáticamente

=== COLAS (QUEUES) ===

1. gpio_command_queue
 - Tipo: QueueHandle_t (FreeRTOS queue)
 - Tamaño: GPIO_QUEUE_SIZE (10 elementos)
 - Elemento: [gpio_command_t](#) (estructura con ID, comando y respuesta)

- Dirección: Handler HTTP → Tarea de procesamiento
- Uso: El handler HTTP /cmd envía comandos aquí
- Operaciones: xQueueSend() desde handler, xQueueReceive() desde tarea
- Thread-safe: Sí, permite comunicación segura entre tareas

2. gpio_response_queue

- Tipo: QueueHandle_t (FreeRTOS queue)
- Tamaño: GPIO_QUEUE_SIZE (10 elementos)
- Elemento: [gpio_command_t](#) (mismo comando con respuesta generada)
- Dirección: Tarea de procesamiento → Handler HTTP
- Uso: La tarea terminal_command_task envía respuestas aquí
- Operaciones: xQueueSend() desde tarea, xQueueReceive() desde handler
- Thread-safe: Sí, permite comunicación asíncrona entre tareas

=== SEMÁFOROS (MUTEXES) ===

1. session_mutex

- Tipo: SemaphoreHandle_t (FreeRTOS mutex)
- Protege: Array ctx->sessions[] (MAX_SESSIONS = 5)
- Uso: Protege acceso concurrente a sesiones de múltiples requests HTTP
- Operaciones: xSemaphoreTake() antes de leer/escribir, xSemaphoreGive() después
- Timeout: Variable (100ms en verificaciones, portMAX_DELAY en creación)
- Funciones que lo usan:
 - [find_or_create_session\(\)](#) - Crear/buscar sesiones
 - [is_authenticated\(\)](#) - Verificar autenticación
 - [session_management_task\(\)](#) - Expirar sesiones inactivas

2. command_id_mutex

- Tipo: SemaphoreHandle_t (FreeRTOS mutex)
- Protege: ctx->command_id_counter (contador de IDs únicos)
- Uso: Garantiza generación atómica de IDs únicos para comandos
- Operaciones: xSemaphoreTake() antes de incrementar, xSemaphoreGive() después
- Timeout: portMAX_DELAY (espera indefinidamente)
- Funciones que lo usan:
 - [cmd_get_handler\(\)](#) - Genera ID único para cada comando
- Propósito: Permite emparejar comandos-respuestas cuando hay múltiples comandos simultáneos

FLUJO DE COMUNICACIÓN:

Cliente Web → Handler HTTP (/cmd) ↓ [xQueueSend] gpio_command_queue Tarea: terminal_command_task ↓ [[process_terminal_command\(\)](#)] ↓ [xQueueSend] gpio_response_queue Handler HTTP (/cmd) ↓ [xQueueReceive] Cliente Web ← Respuesta

4.24.2 Documentación de «define»

4.24.2.1 CHECK_AUTH

```
#define CHECK_AUTH(  
    ctx,  
    req )
```

Valor:

```
do { \
    if (!is_authenticated((ctx), (req))) { \
        httpd_resp_set_status((req), "401 Unauthorized"); \
        httpd_resp_send((req), "Unauthorized", HTTPD_RESP_USE_STRLEN); \
        return ESP_OK; \
    } \
} while(0)
```

4.24.2.2 CHECK_ESP_OK

```
#define CHECK_ESP_OK(
    err,
    msg )
```

Valor:

```
do { \
    if ((err) != ESP_OK) { \
        ESP_LOGE(TAG, "%s: %s (0x%x)", (msg), esp_err_to_name(err), (err)); \
        return ESP_FAIL; \
    } \
} while(0)
```

4.24.2.3 CMD_RESPONSE_MAX_ATTEMPTS

```
#define CMD_RESPONSE_MAX_ATTEMPTS 10
```

4.24.2.4 CMD_RESPONSE_TIMEOUT_MS

```
#define CMD_RESPONSE_TIMEOUT_MS 500
```

4.24.2.5 GPIO_QUEUE_SIZE

```
#define GPIO_QUEUE_SIZE 10
```

4.24.2.6 MAX_SESSIONS

```
#define MAX_SESSIONS 5
```

4.24.2.7 REDIRECT

```
#define REDIRECT(
    req,
    location )
```

Valor:

```
do { \
    httpd_resp_set_hdr((req), "Location", (location)); \
    httpd_resp_set_status((req), "302 Found"); \
    httpd_resp_send((req), NULL, 0); \
} while(0)
```

4.24.2.8 SEND_JSON_RESPONSE

```
#define SEND_JSON_RESPONSE(
    req,
    json_buf )
```

Valor:

```
do { \
    httpd_resp_set_type((req), "application/json"); \
    httpd_resp_set_hdr((req), "Cache-Control", "no-cache, no-store, must-revalidate"); \
    httpd_resp_set_hdr((req), "Pragma", "no-cache"); \
    httpd_resp_set_hdr((req), "Expires", "0"); \
    httpd_resp_send((req), (json_buf), strlen((json_buf))); \
} while(0)
```

4.24.2.9 SEND_JSON_RESPONSE_STR

```
#define SEND_JSON_RESPONSE_STR(
    req,
    json_str )
```

Valor:

```
do { \
    httpd_resp_set_type((req), "application/json"); \
    httpd_resp_set_hdr((req), "Cache-Control", "no-cache, no-store, must-revalidate"); \
    httpd_resp_set_hdr((req), "Pragma", "no-cache"); \
    httpd_resp_set_hdr((req), "Expires", "0"); \
    httpd_resp_sendstr((req), (json_str)); \
} while(0)
```

4.24.2.10 SESSION_TIMEOUT_MS

```
#define SESSION_TIMEOUT_MS (3 * 60 * 1000)
```

4.24.2.11 VALID_PASS

```
#define VALID_PASS "matrix123"
```

4.24.2.12 VALID_USER

```
#define VALID_USER "root"
```

4.24.3 Documentación de funciones

4.24.3.1 cmd_get_handler()

```
static esp_err_t cmd_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /cmd?c=comando Procesa comandos desde la terminal web Envía el comando a la cola y espera la respuesta Requiere autenticación.

4.24.3.2 dashboard_get_handler()

```
static esp_err_t dashboard_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /dashboard Requiere autenticación, redirige a login si no está autenticado.

4.24.3.3 extract_form_param()

```
static bool extract_form_param (
    const char * content,
    const char * param_name,
    char * output,
    size_t output_size ) [static]
```

Extrae un parámetro de una cadena form-urlencoded.

Parámetros

<i>content</i>	Cadena con los parámetros (ej: "user=root&pass=matrix123")
<i>param_name</i>	Nombre del parámetro a extraer (ej: "user")
<i>output</i>	Buffer donde se almacenará el valor
<i>output_size</i>	Tamaño del buffer

Devuelve

true si se encontró el parámetro, false en caso contrario

4.24.3.4 fan_diagnostic_get_handler()

```
static esp_err_t fan_diagnostic_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /fan/diagnostic Endpoint de diagnóstico que muestra el estado completo del sistema de registros Útil para depurar problemas con el modo SCHEDULE Requiere autenticación.

4.24.3.5 fan_manual_post_handler()

```
static esp_err_t fan_manual_post_handler (
    httpd_req_t * req ) [static]
```

Handler para POST /fan/manual Establece el porcentaje de velocidad en modo manual Body esperado: {"percent": 50} (0-100) Requiere autenticación.

4.24.3.6 fan_mode_post_handler()

```
static esp_err_t fan_mode_post_handler (
    httpd_req_t * req ) [static]
```

Handler para POST /fan/mode Establece el modo de operación del ventilador Body esperado: {"mode": "manual"} o {"mode": "off"} o {"mode": "temperature"} Requiere autenticación.

4.24.3.7 fan_status_get_handler()

```
static esp_err_t fan_status_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /fan/status Devuelve el estado actual del ventilador en formato JSON Respuesta: {"mode": "manual", "percent": 50} Requiere autenticación.

4.24.3.8 favicon_get_handler()

```
static esp_err_t favicon_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /favicon.ico Responde con 204 No Content para evitar errores 404.

4.24.3.9 find_or_create_session()

```
static session_t * find_or_create_session (
    webserver_context_t * ctx,
    const char * ip ) [static]
```

Busca una sesión existente o crea una nueva para una IP.

Parámetros

<i>ctx</i>	Contexto del servidor web
<i>ip</i>	Dirección IP del cliente

Devuelve

Puntero a la sesión o NULL si no hay slots disponibles

===== USO DE SEMÁFORO (MUTEX) ===== Esta función accede al array `ctx->sessions[]` que es compartido entre:

- Múltiples handlers HTTP (pueden ejecutarse simultáneamente)
- La tarea `session_management_task` (verifica timeouts periódicamente)

El mutex `session_mutex` garantiza acceso exclusivo:

- `xSemaphoreTake()` adquiere el mutex (`portMAX_DELAY` = espera indefinidamente)
- Solo una tarea puede acceder a `sessions[]` a la vez
- `xSemaphoreGive()` libera el mutex al finalizar

Sin el mutex, habría race conditions:

- Dos handlers HTTP podrían crear sesiones duplicadas
- La tarea de gestión podría expirar una sesión mientras se está usando
- Corrupción de datos en el array de sesiones

4.24.3.10 `get_client_ip()`

```
static void get_client_ip (
    httpd_req_t * req,
    char * ip_str,
    size_t len ) [static]
```

Obtiene la dirección IP del cliente desde la request HTTP.

Parámetros

<i>req</i>	Request HTTP
<i>ip_str</i>	Buffer donde se almacenará la IP
<i>len</i>	Tamaño del buffer

4.24.3.11 `get_time_ms()`

```
static int64_t get_time_ms (
    void ) [static]
```

Obtiene el tiempo actual en milisegundos desde el inicio del sistema.

Devuelve

Tiempo en milisegundos

4.24.3.12 `gpio_command_task_wrapper()`

```
static void gpio_command_task_wrapper (
    void * pvParameters ) [static]
```

Wrapper para la tarea de procesamiento de comandos Esta función adapta el contexto del servidor web al contexto de terminal y llama a la función del módulo `terminal_commands`.

===== EXPLICACIÓN DE LA TAREA ===== Esta función crea un wrapper que adapta el contexto del servidor web al formato esperado por [terminal_command_task\(\)](#). La tarea real se ejecuta en [terminal_commands.c](#), pero este wrapper permite mantener la modularidad del código.

La tarea real (`terminal_command_task`) se ejecuta en un loop infinito:

1. Espera comandos de `gpio_command_queue` (bloqueante, `portMAX_DELAY`)
2. Procesa el comando usando [process_terminal_command\(\)](#)
3. Envía la respuesta a `gpio_response_queue`

Esta arquitectura permite que el handler HTTP no se bloquee esperando el procesamiento del comando, mejorando la capacidad de respuesta del servidor.

4.24.3.13 `init_spiffs()`

```
static esp_err_t init_spiffs (
    void ) [static]
```

Inicializa y monta el sistema de archivos SPIFFS.

Devuelve

ESP_OK si éxito, ESP_FAIL si error

4.24.3.14 `is_authenticated()`

```
static bool is_authenticated (
    webserver_context_t * ctx,
    httpd_req_t * req ) [static]
```

Verifica si el cliente está autenticado y actualiza la última actividad.

Parámetros

<i>ctx</i>	Contexto del servidor web
<i>req</i>	Request HTTP

Devuelve

true si está autenticado, false en caso contrario

===== USO DE SEMÁFORO (MUTEX) ===== Esta función lee y modifica `session->authenticated` y `session->last_activity` que son compartidos con:

- Otros handlers HTTP (múltiples requests simultáneos)
- La tarea `session_management_task` (expira sesiones)

El mutex `session_mutex` protege estas operaciones:

- Lectura de `session->authenticated` (verificar estado)
- Escritura de `session->last_activity` (actualizar timestamp)
- Escritura de `session->authenticated` (expirar sesión si timeout)

Timeout de 100ms: si el mutex está ocupado, espera máximo 100ms Si no se puede adquirir, retorna false (sesión no autenticada por seguridad)

4.24.3.15 login_post_handler()

```
static esp_err_t login_post_handler (
    httpd_req_t * req ) [static]
```

Handler para POST /login Valida credenciales y crea una sesión autenticada Parámetros: user=xxx&pass=yyy (form-urlencoded)

4.24.3.16 logout_get_handler()

```
static esp_err_t logout_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /logout Cierra la sesión del usuario y apaga los LEDs.

4.24.3.17 pir_status_get_handler()

```
static esp_err_t pir_status_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /pir/status Devuelve el estado actual del sensor PIR en formato JSON Respuesta: {"motion": true} o {"motion": false} Requiere autenticación.

4.24.3.18 register_http_routes()

```
static esp_err_t register_http_routes (
    webserver_context_t * ctx ) [static]
```

Registra todas las rutas HTTP del servidor web.

Parámetros

<i>ctx</i>	Contexto del servidor web
------------	---------------------------

Devuelve

ESP_OK si todas las rutas se registraron correctamente, ESP_FAIL en caso contrario

4.24.3.19 registros_get_handler()

```
static esp_err_t registros_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /registros Devuelve todos los registros almacenados en formato JSON desde SPIFFS Utiliza la función [leer_registros_json\(\)](#) del módulo [registros.c](#) Respuesta: Array JSON con todos los registros [{"dia": "...", "hora": "...", "velocidad": ...}, ...] Requiere autenticación.

4.24.3.20 registros_post_handler()

```
static esp_err_t registros_post_handler (
    httpd_req_t * req ) [static]
```

Handler para POST /registros Agrega un nuevo registro al archivo registros.json en SPIFFS Utiliza la función [agregar_registro\(\)](#) del módulo [registros.c](#) Body esperado: {"dia": "lunes", "hora": "14:30", "velocidad": 50} El registro se guarda de forma persistente en /spiffs/registros.json Requiere autenticación.

4.24.3.21 root_get_handler()

```
static esp_err_t root_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET / (raíz) Si el usuario está autenticado, redirige al dashboard Si no, muestra la página de login.

4.24.3.22 script_get_handler()

```
static esp_err_t script_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /script.js Sirve el archivo JavaScript desde SPIFFS.

4.24.3.23 send_file_from_spiffs()

```
esp_err_t send_file_from_spiffs (
    httpd_req_t * req,
    const char * filepath,
    const char * type )
```

Sirve un archivo estático desde SPIFFS al cliente HTTP Lee el archivo en chunks y lo envía de forma eficiente.

Parámetros

<i>req</i>	Request HTTP
<i>filepath</i>	Ruta del archivo en SPIFFS (ej: "/spiffs/index.html")
<i>type</i>	Content-Type HTTP (ej: "text/html", "text/css")

Devuelve

ESP_OK si éxito, ESP_FAIL si error

4.24.3.24 session_management_task()

```
static void session_management_task (
    void * pvParameters ) [static]
```

Tarea que verifica periódicamente las sesiones y expira las inactivas Apaga los LEDs cuando una sesión expira por timeout.

===== EXPLICACIÓN DE LA TAREA ===== Esta es una tarea de FreeRTOS que se ejecuta de forma independiente en segundo plano para gestionar el ciclo de vida de las sesiones.

Funcionamiento:

1. Se ejecuta en un loop infinito (while(1))
2. Espera 2 segundos entre cada verificación (vTaskDelay)

3. Obtiene el tiempo actual y compara con last_activity de cada sesión
4. Si una sesión está inactiva más de SESSION_TIMEOUT_MS (3 minutos):
 - Marca la sesión como no autenticada
 - Apaga los LEDs por seguridad
 - Registra el evento en el log

Uso del semáforo (mutex):

- session_mutex protege el acceso al array ctx->sessions[]
- xSemaphoreTake() adquiere el mutex antes de leer/escribir
- xSemaphoreGive() libera el mutex después de la operación
- Timeout de 100ms para evitar bloqueo indefinido si hay problemas

Prioridad: 3 (configurada en [start_webserver\(\)](#)) Stack: 2048 bytes

Esta tarea es crítica para la seguridad del sistema, ya que:

- Previene sesiones huérfanas que consuman recursos
- Apaga los LEDs automáticamente cuando el usuario se desconecta
- Libera slots de sesión para nuevos usuarios

4.24.3.25 slider_get_handler()

```
static esp_err_t slider_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /slider Requiere autenticación, redirige a login si no está autenticado.

4.24.3.26 start_webserver()

```
void start_webserver (
    void )
```

Función principal que inicia el servidor web.

Inicia el servidor web HTTP.

Proceso de inicialización:

1. Crea estructura de contexto para encapsular el estado (sin variables globales)
2. Crea mutex y colas para comandos y sesiones
3. Crea tareas para procesamiento de comandos y gestión de sesiones (pasa contexto)
4. Monta el sistema de archivos SPIFFS
5. Verifica que todos los archivos necesarios estén presentes
6. Configura e inicia el servidor HTTP
7. Registra todas las rutas y handlers (pasa contexto a través de user_ctx)

4.24.3.27 str_to_lower()

```
static void str_to_lower (
    char * str ) [static]
```

Convierte una cadena a minúsculas (modifica in-place)

Parámetros

<i>str</i>	Cadena a convertir
------------	--------------------

4.24.3.28 style_get_handler()

```
static esp_err_t style_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /style.css Sirve el archivo CSS desde SPIFFS.

4.24.3.29 temperature_get_handler()

```
static esp_err_t temperature_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /temperature Devuelve la temperatura actual en formato JSON Respuesta: {"temperature": 25.5} o {"error": "No data available"} Requiere autenticación.

4.24.3.30 terminal_get_handler()

```
static esp_err_t terminal_get_handler (
    httpd_req_t * req ) [static]
```

Handler para GET /terminal Requiere autenticación, redirige a login si no está autenticado.

4.24.3.31 time_set_post_handler()

```
static esp_err_t time_set_post_handler (
    httpd_req_t * req ) [static]
```

Handler para POST /time/set Establece la hora manualmente Body esperado: {"year": 2024, "month": 12, "day": 9, "hour": 14, "minute": 30, "second": 0} Requiere autenticación.

4.24.3.32 url_decode()

```
static void url_decode (
    char * str ) [static]
```

Decodifica una cadena URL-encoded (convierte %20 y + a espacios)

Parámetros

<i>str</i>	Cadena a decodificar (se modifica in-place)
------------	---

4.24.3.33 verify_spiffs_files()

```
static int verify_spiffs_files (
    void ) [static]
```

Verifica que todos los archivos necesarios estén presentes en SPIFFS.

Devuelve

Número de archivos encontrados

4.24.4 Documentación de variables

4.24.4.1 TAG

```
const char* TAG = "WEB_SERVER" [static]
```

4.25 Referencia del archivo main/web_server.h

Servidor web HTTP del ESP32.

Funciones

- void [start_webserver](#) (void)

Inicia el servidor web HTTP.

4.25.1 Descripción detallada

Servidor web HTTP del ESP32.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para el servidor web HTTP del ESP32. Este módulo gestiona:

- Montaje del sistema de archivos SPIFFS (donde se almacenan los archivos HTML/CSS/JS)
- Inicialización del servidor HTTP con múltiples rutas
- Manejo de autenticación de usuarios (login/logout)
- Procesamiento de comandos desde la terminal web mediante colas
- Servicio de datos de temperatura en tiempo real
- Gestión de sesiones con timeout automático
- Control del ventilador mediante API REST
- Gestión de registros de horarios del ventilador
- Control del sensor PIR
- Configuración de hora manual

4.25.2 Rutas disponibles

4.25.2.1 Páginas web

- / : Página de login o redirección al dashboard
- /dashboard : Panel principal con opciones
- /terminal : Terminal web retro para comandos
- /slider : Panel de control con temperatura y ventilador

4.25.2.2 Sistema de autenticación y sesiones

- /login : Autenticación de usuarios (POST)
- /logout : Cerrar sesión (GET)

4.25.2.3 API REST

- /cmd : Endpoint para ejecutar comandos de terminal (GET ?c=comando)
- /temperature : API JSON para obtener temperatura actual (GET)
- /fan/mode : Establecer modo del ventilador (POST)
- /fan/manual : Establecer velocidad manual (POST)
- /fan/status : Obtener estado del ventilador (GET)
- /fan/diagnostic : Diagnóstico completo del sistema (GET)
- /registros : Obtener todos los registros (GET) / Agregar nuevo registro (POST)
- /pir/status : Obtener estado del sensor PIR (GET)
- /time/set : Establecer hora manualmente (POST)

ARCHIVO: [web_server.h](#)

RESUMEN: Header file para el servidor web HTTP del ESP32. Este módulo gestiona:

- Montaje del sistema de archivos SPIFFS (donde se almacenan los archivos HTML/CSS/JS)
- Inicialización del servidor HTTP con múltiples rutas
- Manejo de autenticación de usuarios (login/logout)
- Procesamiento de comandos desde la terminal web mediante colas
- Servicio de datos de temperatura en tiempo real
- Gestión de sesiones con timeout automático
- Control del ventilador mediante API REST
- Gestión de registros de horarios del ventilador
- Control del sensor PIR
- Configuración de hora manual

Rutas disponibles:

Páginas web:

- / : Página de login o redirección al dashboard
- /dashboard : Panel principal con opciones
- /terminal : Terminal web retro para comandos
- /slider : Panel de control con temperatura y ventilador

Autenticación:

- /login : Autenticación de usuarios (POST)
- /logout : Cerrar sesión (GET)

API de comandos:

- /cmd : Endpoint para ejecutar comandos de terminal (GET ?c=comando)

API de temperatura:

- /temperature : API JSON para obtener temperatura actual (GET)

API de ventilador:

- /fan/mode : Establecer modo del ventilador (POST)
- /fan/manual : Establecer velocidad manual (POST)
- /fan/status : Obtener estado del ventilador (GET)
- /fan/diagnostic : Diagnóstico completo del sistema (GET)

API de registros:

- /registros : Obtener todos los registros (GET)
- /registros : Agregar nuevo registro (POST)

API de sensor PIR:

- /pir/status : Obtener estado del sensor PIR (GET)

API de tiempo:

- /time/set : Establecer hora manualmente (POST)

ÍNDICE DE SECCIONES:

Sección 1: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 60 a 70

4.25.3 Documentación de funciones**4.25.3.1 start_webserver()**

```
void start_webserver (
    void )
```

Inicia el servidor web HTTP.

- Monta el sistema de archivos SPIFFS
- Registra todas las rutas y handlers
- Crea tareas para procesamiento de comandos y gestión de sesiones

Inicia el servidor web HTTP.

Proceso de inicialización:

1. Crea estructura de contexto para encapsular el estado (sin variables globales)
2. Crea mutex y colas para comandos y sesiones
3. Crea tareas para procesamiento de comandos y gestión de sesiones (pasa contexto)
4. Monta el sistema de archivos SPIFFS
5. Verifica que todos los archivos necesarios estén presentes
6. Configura e inicia el servidor HTTP
7. Registra todas las rutas y handlers (pasa contexto a través de user_ctx)

4.26 web_server.h

[Ir a la documentación de este archivo.](#)

```
00001
00100 #pragma once
00101
00102 #ifdef __cplusplus
00103 extern "C" {
00104 #endif
00105
00106 // ===== PROTOTIPOS DE FUNCIONES =====
00113 void start_webserver(void);
00114
00115 // Si necesitaras detenerlo desde otro lado, podrías añadir:
00116 // void stop_webserver(void);
00117
00118 #ifdef __cplusplus
00119 }
00120 #endif
```

4.27 Referencia del archivo main/wifi_app.c

Implementación del módulo de WiFi en modo AP+STA.

```
#include "wifi_app.h"
#include "esp_event.h"
#include "esp_log.h"
#include "esp_mac.h"
#include "esp_wifi.h"
#include "nvs_flash.h"
#include "esp_netif_ip_addr.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
```

```
#include "lwip/err.h"
#include "lwip/sys.h"
#include "lwip/netdb.h"
#include "esp_netif.h"
#include "esp_netif_types.h"
#include <string.h>
```

Funciones

- static void [wifi_event_handler](#) (void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data)
Manejador de eventos WiFi para logging de conexiones/desconexiones Registra cuando los clientes se conectan o desconectan del Access Point y maneja la conexión del ESP32 como Station a la red WiFi externa.
- void [wifi_init_softap](#) (void)
Inicializa el ESP32 en modo AP+STA (Access Point + Station)
- bool [wifi_is_connected](#) (void)
Verifica si el ESP32 está conectado a Internet (modo Station)

Variables

- static const char * [TAG](#) = "WIFI_APP"
- static bool [wifi_connected](#) = false
- static int [s_retry_num](#) = 0

4.27.1 Descripción detallada

Implementación del módulo de WiFi en modo AP+STA.

Autor

Proyecto Final RTOS

Fecha

2024

Implementación del módulo de WiFi en modo AP+STA (Access Point + Station). Este módulo configura el ESP32 para funcionar simultáneamente como:

- Access Point (SoftAP): Crea su propia red WiFi para acceso local
- Station (STA): Se conecta a una red WiFi externa para tener Internet

4.27.2 Funcionalidades

- Inicializa la pila de red TCP/IP
- Configura el ESP32 como punto de acceso WiFi (red local)
- Se conecta a una red WiFi externa para tener Internet (SNTP, etc.)
- Maneja eventos de conexión/desconexión de clientes y estaciones
- Reintentos automáticos de conexión a la red externa

4.27.3 Uso en el sistema

Los usuarios pueden conectarse a la red "ESP32_Server" y acceder al servidor web para controlar el sistema. El ESP32 también se conecta a la red "Mondongo" para tener acceso a Internet (SNTP, etc.).

ARCHIVO: [wifi_app.c](#)

RESUMEN: Implementación del módulo de WiFi en modo AP+STA (Access Point + Station). Este módulo configura el ESP32 para funcionar simultáneamente como:

- Access Point (SoftAP): Crea su propia red WiFi para acceso local
- Station (STA): Se conecta a una red WiFi externa para tener Internet

Funcionalidades:

- Inicializa la pila de red TCP/IP
- Configura el ESP32 como punto de acceso WiFi (red local)
- Se conecta a una red WiFi externa para tener Internet (SNTP, etc.)
- Maneja eventos de conexión/desconexión de clientes y estaciones
- Reintentos automáticos de conexión a la red externa

Los usuarios pueden conectarse a la red "ESP32_Server" y acceder al servidor web para controlar el sistema. El ESP32 también se conecta a la red "Mondongo" para tener acceso a Internet (SNTP, etc.).

ÍNDICE DE SECCIONES:

Sección 1: INCLUDES se encuentra en las líneas 36 a 60 Sección 2: DEFINICIONES Y CONSTANTES se encuentra en las líneas 62 a 64 Sección 3: MANEJO DE EVENTOS WIFI se encuentra en las líneas 66 a 103 Sección 4: INICIALIZACIÓN DE SOFTAP se encuentra en las líneas 105 a 197

Sección 5: FUNCIONES DE UTILIDAD se encuentra en las líneas 199 a 205**RESUMEN DE TAREAS, COLAS Y SEMÁFOROS IMPLEMENTADOS:**

=== TAREAS (TASKS) ===

Ninguna en este módulo. WiFi funciona mediante eventos y callbacks.

=== COLAS (QUEUES) ===

Ninguna en este módulo.

=== SEMÁFOROS (MUTEXES) ===

Ninguno en este módulo. El acceso a variables globales es thread-safe porque solo se modifican desde el handler de eventos que se ejecuta en el contexto del bucle de eventos.

4.27.4 Documentación de funciones**4.27.4.1 `wifi_event_handler()`**

```
static void wifi_event_handler (
    void * arg,
    esp_event_base_t event_base,
    int32_t event_id,
    void * event_data ) [static]
```

Manejador de eventos WiFi para logging de conexiones/desconexiones Registra cuando los clientes se conectan o desconectan del Access Point y maneja la conexión del ESP32 como Station a la red WiFi externa.

Parámetros

<i>arg</i>	Argumento del handler (no usado)
<i>event_base</i>	Base del evento (WIFI_EVENT o IP_EVENT)
<i>event_id</i>	ID del evento específico
<i>event_data</i>	Datos del evento

4.27.4.2 `wifi_init_softap()`

```
void wifi_init_softap (
    void )
```

Inicializa el ESP32 en modo AP+STA (Access Point + Station)

- Crea su propia red WiFi (SoftAP) para acceso local
- Se conecta a una red WiFi externa (Station) para tener Internet

Proceso de inicialización:

1. Inicializa la pila TCP/IP
2. Crea el bucle de eventos
3. Crea las interfaces de red WiFi (AP y STA)
4. Inicializa el driver WiFi
5. Registra los manejadores de eventos (WiFi e IP)
6. Configura credenciales del Access Point
7. Configura credenciales del Station (red externa)
8. Aplica configuración y inicia en modo AP+STA

4.27.4.3 `wifi_is_connected()`

```
bool wifi_is_connected (
    void )
```

Verifica si el ESP32 está conectado a Internet (modo Station)

Devuelve

true si está conectado, false en caso contrario

4.27.5 Documentación de variables

4.27.5.1 `s_retry_num`

```
int s_retry_num = 0 [static]
```

4.27.5.2 `TAG`

```
const char* TAG = "WIFI_APP" [static]
```

4.27.5.3 `wifi_connected`

```
bool wifi_connected = false [static]
```

4.28 Referencia del archivo `main/wifi_app.h`

Configuración y gestión de WiFi en modo AP+STA.

```
#include <stdbool.h>
```

defines

- #define ESP_WIFI_SSID "ESP32_Server"
- #define ESP_WIFI_PASS "12345678"
- #define ESP_WIFI_CHANNEL 1
- #define MAX_STA_CONN 4
- #define WIFI_STA_SSID "Mondongo"
- #define WIFI_STA_PASS "huevos12"
- #define WIFI_MAX_RETRY 5

Funciones

- void [wifi_init_softap](#) (void)
Inicializa el ESP32 en modo AP+STA (Access Point + Station)
- bool [wifi_is_connected](#) (void)
Verifica si el ESP32 está conectado a Internet (modo Station)

4.28.1 Descripción detallada

Configuración y gestión de WiFi en modo AP+STA.

Autor

Proyecto Final RTOS

Fecha

2024

Header file para la configuración y gestión de WiFi en modo AP+STA (Access Point + Station). Define las credenciales de las redes WiFi y las funciones para inicializar el punto de acceso y la conexión Station.

4.28.2 Configuración Access Point (SoftAP)

- SSID: "ESP32_Server"
- Contraseña: "12345678"
- Canal: 1
- Máximo de estaciones conectadas: 4
- Autenticación: WPA2 Personal

4.28.3 Configuración Station (STA)

- SSID: "Mondongo"
- Contraseña: "huevos12"
- Máximo de reintentos: 5

ARCHIVO: [wifi_app.h](#)

RESUMEN: Header file para la configuración y gestión de WiFi en modo AP+STA (Access Point + Station). Define las credenciales de las redes WiFi y las funciones para inicializar el punto de acceso y la conexión Station.

Configuración Access Point (SoftAP):

- SSID: "ESP32_Server"
- Contraseña: "12345678"

- Canal: 1
- Máximo de estaciones conectadas: 4
- Autenticación: WPA2 Personal

Configuración Station (STA):

- SSID: "Mondongo"
- Contraseña: "huevos12"
- Máximo de reintentos: 5

ÍNDICE DE SECCIONES:

Sección 1: CONFIGURACIÓN DE WIFI SOFTAP se encuentra en las líneas 33 a 34 Sección 2: CONFIGURACIÓN DE WIFI STATION se encuentra en las líneas 36 a 39

Sección 3: PROTOTIPOS DE FUNCIONES se encuentra en las líneas 41 a 61

4.28.4 Documentación de «define»

4.28.4.1 ESP_WIFI_CHANNEL

```
#define ESP_WIFI_CHANNEL 1
```

4.28.4.2 ESP_WIFI_PASS

```
#define ESP_WIFI_PASS "12345678"
```

4.28.4.3 ESP_WIFI_SSID

```
#define ESP_WIFI_SSID "ESP32_Server"
```

4.28.4.4 MAX_STA_CONN

```
#define MAX_STA_CONN 4
```

4.28.4.5 WIFI_MAX_RETRY

```
#define WIFI_MAX_RETRY 5
```

4.28.4.6 WIFI_STA_PASS

```
#define WIFI_STA_PASS "huevos12"
```

4.28.4.7 WIFI_STA_SSID

```
#define WIFI_STA_SSID "Mondongo"
```

4.28.5 Documentación de funciones

4.28.5.1 wifi_init_softap()

```
void wifi_init_softap (
    void )
```

Inicializa el ESP32 en modo AP+STA (Access Point + Station)

- Crea su propia red WiFi (SoftAP) para acceso local
- Se conecta a una red WiFi externa (Station) para tener Internet

Proceso:

- Inicializa la pila TCP/IP
- Crea el bucle de eventos
- Configura el driver WiFi en modo AP+STA
- Inicia el Access Point con credenciales locales
- Se conecta a la red WiFi externa para Internet
- Crea su propia red WiFi (SoftAP) para acceso local
- Se conecta a una red WiFi externa (Station) para tener Internet

Proceso de inicialización:

1. Inicializa la pila TCP/IP
2. Crea el bucle de eventos
3. Crea las interfaces de red WiFi (AP y STA)
4. Inicializa el driver WiFi
5. Registra los manejadores de eventos (WiFi e IP)
6. Configura credenciales del Access Point
7. Configura credenciales del Station (red externa)
8. Aplica configuración y inicia en modo AP+STA

4.28.5.2 wifi_is_connected()

```
bool wifi_is_connected (
    void )
```

Verifica si el ESP32 está conectado a Internet (modo Station)

Devuelve

true si está conectado, false en caso contrario

4.29 wifi_app.h

[Ir a la documentación de este archivo.](#)

```
00001
00053 #pragma once
00054
00055 #include <stdbool.h>
00056
00057 // ===== CONFIGURACIÓN DE WIFI SOFTAP =====
00058 // Credenciales de la red WiFi que crea el ESP32
00059 #define ESP_WIFI_SSID      "ESP32_Server" // Nombre de la red WiFi (SSID)
00060 #define ESP_WIFI_PASS      "12345678"     // Contraseña de la red (mínimo 8 caracteres para WPA2)
00061 #define ESP_WIFI_CHANNEL  1               // Canal WiFi (rango: 1-13)
00062 #define MAX_STA_CONN      4               // Máximo número de dispositivos que pueden conectarse
00063                                         simultáneamente
00064 // ===== CONFIGURACIÓN DE WIFI STATION (STA) =====
00065 // Credenciales de la red WiFi a la que se conectará el ESP32 para tener Internet
00066 #define WIFI_STA_SSID      "Mondongo"     // Nombre de la red WiFi a la que conectarse
00067 #define WIFI_STA_PASS      "huevos12"     // Contraseña de la red WiFi
00068 #define WIFI_MAX_RETRY     5               // Número máximo de intentos de conexión
00069
00070 // ===== PROTOTIPOS DE FUNCIONES =====
00083 void wifi_init_softap(void);
00084
00089 bool wifi_is_connected(void);
```


Index

- activo
 - registro_activo_t, 8
- adc1_cali_handle
 - ntc_sensor.c, 33
- adc1_handle
 - ntc_sensor.c, 33
- adc_calibration_init
 - ntc_sensor.c, 32
- ADC_MAX_VALUE
 - ntc_sensor.c, 31
- ADC_MIN_VALUE
 - ntc_sensor.c, 31
- ADC_STABILIZATION_DELAY_MS
 - ntc_sensor.c, 31
- ADC_UNIT
 - ntc_sensor.h, 35
- agregar_registro
 - registros.c, 47
 - registros.h, 50
- app_main
 - main.c, 28
- authenticated
 - session_t, 9
- B_COEFFICIENT
 - ntc_sensor.h, 35
- calculate_temp_percent
 - fan_control.c, 15
- cargar_hora_desde_nvs
 - time_sync.c, 67
- CHECK_AUTH
 - web_server.c, 78
- CHECK_ESP_OK
 - web_server.c, 78
- cmd_get_handler
 - web_server.c, 80
- CMD_RESPONSE_MAX_ATTEMPTS
 - web_server.c, 79
- CMD_RESPONSE_TIMEOUT_MS
 - web_server.c, 79
- command
 - gpio_command_t, 6
- command_id
 - gpio_command_t, 6
- command_id_counter
 - webserver_context_t, 10
- command_id_mutex
 - webserver_context_t, 10
- crear_archivo_si_no_existe
 - registros.c, 47
 - registros.h, 50
- current_mode
 - fan_control.c, 18
- current_ntc_data
 - ntc_sensor.c, 33
- current_pwm_percent
 - fan_control.c, 18
- dashboard_get_handler
 - web_server.c, 80
- data_mutex
 - ntc_sensor.c, 33
- data_ready
 - ntc_sensor.c, 33
- dias_semana
 - time_sync.c, 69
- end_hour
 - fan_schedule_t, 5
- ESP_WIFI_CHANNEL
 - wifi_app.h, 94
- ESP_WIFI_PASS
 - wifi_app.h, 94
- ESP_WIFI_SSID
 - wifi_app.h, 94
- establecer_hora_manual
 - time_sync.c, 67
 - time_sync.h, 71
- extract_form_param
 - web_server.c, 80
- fan_auto_temp_task
 - fan_control.c, 15
- FAN_AUTO_TEMP_TASK_PRIORITY
 - fan_control.c, 14
- FAN_AUTO_TEMP_TASK_STACK_SIZE
 - fan_control.c, 14
- FAN_AUTO_TEMP_UPDATE_INTERVAL_MS
 - fan_control.c, 15
- fan_control.c
 - calculate_temp_percent, 15
 - current_mode, 18
 - current_pwm_percent, 18
 - fan_auto_temp_task, 15
 - FAN_AUTO_TEMP_TASK_PRIORITY, 14
 - FAN_AUTO_TEMP_TASK_STACK_SIZE, 14
 - FAN_AUTO_TEMP_UPDATE_INTERVAL_MS, 15
 - fan_get_current_percent, 15
 - fan_get_mode, 15

- fan_init, [16](#)
- fan_schedule, [18](#)
- fan_schedule_task, [16](#)
- FAN_SCHEDULE_TASK_PRIORITY, [15](#)
- FAN_SCHEDULE_TASK_STACK_SIZE, [15](#)
- FAN_SCHEDULE_UPDATE_INTERVAL_MS, [15](#)
- fan_set_manual_percent, [16](#)
- fan_set_mode, [16](#)
- fan_set_schedule, [16](#)
- fan_start_auto_temp_task, [17](#)
- fan_start_schedule_task, [17](#)
- fan_update_auto_temp, [17](#)
- fan_update_from_registros, [17](#)
- fan_update_schedule, [17](#)
- manual_percent, [18](#)
- set_fan_pwm, [18](#)
- TAG, [18](#)
- fan_control.h
 - FAN_DUTY_RES, [21](#)
 - FAN_FREQUENCY, [21](#)
 - fan_get_current_percent, [22](#)
 - fan_get_mode, [22](#)
 - fan_init, [22](#)
 - FAN_LEDC_CHANNEL, [21](#)
 - FAN_LEDC_TIMER, [21](#)
 - FAN_MODE_AUTO_TEMP, [21](#)
 - FAN_MODE_MANUAL, [21](#)
 - FAN_MODE_OFF, [21](#)
 - FAN_MODE_SCHEDULE, [21](#)
 - fan_mode_t, [21](#)
 - FAN_PWM_PIN, [21](#)
 - fan_set_manual_percent, [22](#)
 - fan_set_mode, [22](#)
 - fan_set_schedule, [23](#)
 - fan_start_auto_temp_task, [23](#)
 - fan_start_schedule_task, [23](#)
 - FAN_TEMP_HYSTERESIS, [21](#)
 - FAN_TEMP_MAX_C, [21](#)
 - FAN_TEMP_MIN_C, [21](#)
 - fan_update_auto_temp, [23](#)
 - fan_update_from_registros, [24](#)
 - fan_update_schedule, [24](#)
- fan_diagnostic_get_handler
 - web_server.c, [80](#)
- FAN_DUTY_RES
 - fan_control.h, [21](#)
- FAN_FREQUENCY
 - fan_control.h, [21](#)
- fan_get_current_percent
 - fan_control.c, [15](#)
 - fan_control.h, [22](#)
- fan_get_mode
 - fan_control.c, [15](#)
 - fan_control.h, [22](#)
- fan_init
 - fan_control.c, [16](#)
 - fan_control.h, [22](#)
- FAN_LEDC_CHANNEL
 - fan_control.h, [21](#)
- FAN_LEDC_TIMER
 - fan_control.h, [21](#)
- fan_manual_post_handler
 - web_server.c, [80](#)
- FAN_MODE_AUTO_TEMP
 - fan_control.h, [21](#)
- FAN_MODE_MANUAL
 - fan_control.h, [21](#)
- FAN_MODE_OFF
 - fan_control.h, [21](#)
- fan_mode_post_handler
 - web_server.c, [81](#)
- FAN_MODE_SCHEDULE
 - fan_control.h, [21](#)
- fan_mode_t
 - fan_control.h, [21](#)
- FAN_PWM_PIN
 - fan_control.h, [21](#)
- fan_schedule
 - fan_control.c, [18](#)
- fan_schedule_t, [5](#)
 - end_hour, [5](#)
 - min_temp, [5](#)
 - start_hour, [5](#)
- fan_schedule_task
 - fan_control.c, [16](#)
- FAN_SCHEDULE_TASK_PRIORITY
 - fan_control.c, [15](#)
- FAN_SCHEDULE_TASK_STACK_SIZE
 - fan_control.c, [15](#)
- FAN_SCHEDULE_UPDATE_INTERVAL_MS
 - fan_control.c, [15](#)
- fan_set_manual_percent
 - fan_control.c, [16](#)
 - fan_control.h, [22](#)
- fan_set_mode
 - fan_control.c, [16](#)
 - fan_control.h, [22](#)
- fan_set_schedule
 - fan_control.c, [16](#)
 - fan_control.h, [23](#)
- fan_start_auto_temp_task
 - fan_control.c, [17](#)
 - fan_control.h, [23](#)
- fan_start_schedule_task
 - fan_control.c, [17](#)
 - fan_control.h, [23](#)
- fan_status_get_handler
 - web_server.c, [81](#)
- FAN_TEMP_HYSTERESIS
 - fan_control.h, [21](#)
- FAN_TEMP_MAX_C
 - fan_control.h, [21](#)
- FAN_TEMP_MIN_C
 - fan_control.h, [21](#)
- fan_update_auto_temp
 - fan_control.c, [17](#)

- fan_control.h, 23
- fan_update_from_registros
 - fan_control.c, 17
 - fan_control.h, 24
- fan_update_schedule
 - fan_control.c, 17
 - fan_control.h, 24
- favicon_get_handler
 - web_server.c, 81
- find_or_create_session
 - web_server.c, 81
- front/script.js, 11
- get_client_ip
 - web_server.c, 81
- get_time_ms
 - registros.c, 47
 - web_server.c, 82
- gpio_command_queue
 - terminal_context_t, 9
 - webserver_context_t, 10
- gpio_command_t, 6
 - command, 6
 - command_id, 6
 - response, 6
- gpio_command_task_wrapper
 - web_server.c, 82
- GPIO_GREEN
 - rgb_led.c, 53
- GPIO_QUEUE_SIZE
 - web_server.c, 79
- gpio_response_queue
 - terminal_context_t, 9
 - webserver_context_t, 10
- guardar_hora_en_nvs
 - time_sync.c, 67
- hora_sincronizada
 - time_sync.c, 67
 - time_sync.h, 71
- init_spiffs
 - web_server.c, 82
- ip
 - session_t, 9
- is_authenticated
 - web_server.c, 82
- last_activity
 - session_t, 9
- LEDC_CHANNEL_G
 - rgb_led.c, 53
- LEDC_DUTY_RES
 - rgb_led.c, 53
- LEDC_FREQUENCY
 - rgb_led.c, 53
- LEDC_MODE
 - rgb_led.c, 53
- LEDC_TIMER
 - rgb_led.c, 53
- leer_registros_json
 - registros.c, 47
 - registros.h, 50
- login_post_handler
 - web_server.c, 83
- logout_get_handler
 - web_server.c, 83
- main.c
 - app_main, 28
 - TAG, 28
- main/fan_control.c, 11
- main/fan_control.h, 18, 24
- main/main.c, 25
- main/ntc_sensor.c, 28
- main/ntc_sensor.h, 34, 36
- main/pir_driver.c, 37
- main/pir_driver.h, 41, 44
- main/registros.c, 45
- main/registros.h, 48, 51
- main/rgb_led.c, 51
- main/rgb_led.h, 54, 56
- main/terminal_commands.c, 57
- main/terminal_commands.h, 60, 64
- main/time_sync.c, 65
- main/time_sync.h, 70, 72
- main/web_server.c, 73
- main/web_server.h, 86, 89
- main/wifi_app.c, 89
- main/wifi_app.h, 92, 95
- manual_percent
 - fan_control.c, 18
- MAX_SESSIONS
 - web_server.c, 79
- MAX_STA_CONN
 - wifi_app.h, 94
- min_temp
 - fan_schedule_t, 5
- motion
 - pir_event_t, 8
- NOMINAL_RESISTANCE
 - ntc_sensor.h, 35
- NOMINAL_TEMPERATURE
 - ntc_sensor.h, 35
- normalizar_dia
 - registros.c, 48
- ntc_data_t, 6
 - raw_adc_value, 7
 - resistance, 7
 - temperature_c, 7
- ntc_get_current_temperature
 - ntc_sensor.c, 32
 - ntc_sensor.h, 36
- NTC_PIN
 - ntc_sensor.h, 35
- ntc_read_temperature
 - ntc_sensor.c, 32

- ntc_sensor.h, 36
- ntc_reading_task
 - ntc_sensor.c, 32
- ntc_sensor.c
 - adc1_cali_handle, 33
 - adc1_handle, 33
 - adc_calibration_init, 32
 - ADC_MAX_VALUE, 31
 - ADC_MIN_VALUE, 31
 - ADC_STABILIZATION_DELAY_MS, 31
 - current_ntc_data, 33
 - data_mutex, 33
 - data_ready, 33
 - ntc_get_current_temperature, 32
 - ntc_read_temperature, 32
 - ntc_reading_task, 32
 - ntc_sensor_init, 33
 - ntc_start_reading_task, 33
 - NTC_TASK_PRIORITY, 31
 - NTC_TASK_STACK_SIZE, 31
 - RESISTANCE_MAX_OHMS, 31
 - SENSOR_READ_INTERVAL_MS, 32
 - TAG, 33
 - TEMP_ERROR_VALUE, 32
- ntc_sensor.h
 - ADC_UNIT, 35
 - B_COEFFICIENT, 35
 - NOMINAL_RESISTANCE, 35
 - NOMINAL_TEMPERATURE, 35
 - ntc_get_current_temperature, 36
 - NTC_PIN, 35
 - ntc_read_temperature, 36
 - ntc_sensor_init, 36
 - ntc_start_reading_task, 36
 - SERIES_RESISTOR, 35
- ntc_sensor_init
 - ntc_sensor.c, 33
 - ntc_sensor.h, 36
- ntc_start_reading_task
 - ntc_sensor.c, 33
 - ntc_sensor.h, 36
- NTC_TASK_PRIORITY
 - ntc_sensor.c, 31
- NTC_TASK_STACK_SIZE
 - ntc_sensor.c, 31
- NVS_KEY_TIME
 - time_sync.c, 69
- NVS_NAMESPACE
 - time_sync.c, 69
- obtener_dia_actual
 - time_sync.c, 67
 - time_sync.h, 72
- obtener_hora_actual
 - time_sync.c, 69
 - time_sync.h, 72
- pir_driver.c
 - pir_init, 39
 - pir_is_motion_active, 40
 - pir_isr_handler, 40
 - s_pir_evt_queue, 41
 - s_pir_gpio, 41
 - s_pir_initialized, 41
 - TAG, 41
- pir_driver.h
 - PIR_GPIO_PIN, 43
 - pir_init, 43
 - pir_is_motion_active, 44
- pir_event_t, 7
 - motion, 8
- PIR_GPIO_PIN
 - pir_driver.h, 43
- pir_init
 - pir_driver.c, 39
 - pir_driver.h, 43
- pir_is_motion_active
 - pir_driver.c, 40
 - pir_driver.h, 44
- pir_isr_handler
 - pir_driver.c, 40
- pir_status_get_handler
 - web_server.c, 83
- process_terminal_command
 - terminal_commands.c, 59
 - terminal_commands.h, 62
- raw_adc_value
 - ntc_data_t, 7
- REDIRECT
 - web_server.c, 79
- register_http_routes
 - web_server.c, 83
- registro_activo_t, 8
 - activo, 8
 - velocidad, 8
- registros.c
 - agregar_registro, 47
 - crear_archivo_si_no_existe, 47
 - get_time_ms, 47
 - leer_registros_json, 47
 - normalizar_dia, 48
 - TAG, 48
 - verificar_registro_activo, 48
- registros.h
 - agregar_registro, 50
 - crear_archivo_si_no_existe, 50
 - leer_registros_json, 50
 - verificar_registro_activo, 51
- registros_get_handler
 - web_server.c, 83
- registros_post_handler
 - web_server.c, 84
- resistance
 - ntc_data_t, 7
- RESISTANCE_MAX_OHMS
 - ntc_sensor.c, 31
- response

- gpio_command_t, 6
- rgb_led.c
 - GPIO_GREEN, 53
 - LEDC_CHANNEL_G, 53
 - LEDC_DUTY_RES, 53
 - LEDC_FREQUENCY, 53
 - LEDC_MODE, 53
 - LEDC_TIMER, 53
 - rgb_led_init, 54
 - rgb_led_initialized, 54
 - rgb_set_green_percent, 54
 - TAG, 54
- rgb_led.h
 - rgb_led_init, 55
 - rgb_set_green_percent, 56
- rgb_led_init
 - rgb_led.c, 54
 - rgb_led.h, 55
- rgb_led_initialized
 - rgb_led.c, 54
- rgb_set_green_percent
 - rgb_led.c, 54
 - rgb_led.h, 56
- root_get_handler
 - web_server.c, 84
- s_pir_evt_queue
 - pir_driver.c, 41
- s_pir_gpio
 - pir_driver.c, 41
- s_pir_initialized
 - pir_driver.c, 41
- s_retry_num
 - wifi_app.c, 92
- script_get_handler
 - web_server.c, 84
- send_file_from_spiffs
 - web_server.c, 84
- SEND_JSON_RESPONSE
 - web_server.c, 79
- SEND_JSON_RESPONSE_STR
 - web_server.c, 79
- SENSOR_READ_INTERVAL_MS
 - ntc_sensor.c, 32
- SERIES_RESISTOR
 - ntc_sensor.h, 35
- server
 - webserver_context_t, 10
- session_management_task
 - web_server.c, 84
- session_mutex
 - webserver_context_t, 10
- session_t, 9
 - authenticated, 9
 - ip, 9
 - last_activity, 9
- SESSION_TIMEOUT_MS
 - web_server.c, 79
- sessions
 - webserver_context_t, 10
- set_fan_pwm
 - fan_control.c, 18
- slider_get_handler
 - web_server.c, 85
- sntp_initialized
 - time_sync.c, 69
- sntp_sync_time_cb
 - time_sync.c, 69
- start_hour
 - fan_schedule_t, 5
- start_webserver
 - web_server.c, 85
 - web_server.h, 89
- str_to_lower
 - web_server.c, 85
- style_get_handler
 - web_server.c, 86
- TAG
 - fan_control.c, 18
 - main.c, 28
 - ntc_sensor.c, 33
 - pir_driver.c, 41
 - registros.c, 48
 - rgb_led.c, 54
 - terminal_commands.c, 60
 - time_sync.c, 70
 - web_server.c, 86
 - wifi_app.c, 92
- TEMP_ERROR_VALUE
 - ntc_sensor.c, 32
- temperature_c
 - ntc_data_t, 7
- temperature_get_handler
 - web_server.c, 86
- terminal_command_task
 - terminal_commands.c, 59
 - terminal_commands.h, 63
- terminal_commands.c
 - process_terminal_command, 59
 - TAG, 60
 - terminal_command_task, 59
- terminal_commands.h
 - process_terminal_command, 62
 - terminal_command_task, 63
- terminal_context_t, 9
 - gpio_command_queue, 9
 - gpio_response_queue, 9
- terminal_get_handler
 - web_server.c, 86
- time_set_post_handler
 - web_server.c, 86
- time_sync.c
 - cargar_hora_desde_nvs, 67
 - dias_semana, 69
 - establecer_hora_manual, 67
 - guardar_hora_en_nvs, 67
 - hora_sincronizada, 67

- NVS_KEY_TIME, 69
- NVS_NAMESPACE, 69
- obtener_dia_actual, 67
- obtener_hora_actual, 69
- sntp_initialized, 69
- sntp_sync_time_cb, 69
- TAG, 70
- time_sync_init, 69
- time_synced, 70
- time_sync.h
 - establecer_hora_manual, 71
 - hora_sincronizada, 71
 - obtener_dia_actual, 72
 - obtener_hora_actual, 72
 - time_sync_init, 72
- time_sync_init
 - time_sync.c, 69
 - time_sync.h, 72
- time_synced
 - time_sync.c, 70
- url_decode
 - web_server.c, 86
- VALID_PASS
 - web_server.c, 80
- VALID_USER
 - web_server.c, 80
- velocidad
 - registro_activo_t, 8
- verificar_registro_activo
 - registros.c, 48
 - registros.h, 51
- verify_spiffs_files
 - web_server.c, 86
- web_server.c
 - CHECK_AUTH, 78
 - CHECK_ESP_OK, 78
 - cmd_get_handler, 80
 - CMD_RESPONSE_MAX_ATTEMPTS, 79
 - CMD_RESPONSE_TIMEOUT_MS, 79
 - dashboard_get_handler, 80
 - extract_form_param, 80
 - fan_diagnostic_get_handler, 80
 - fan_manual_post_handler, 80
 - fan_mode_post_handler, 81
 - fan_status_get_handler, 81
 - favicon_get_handler, 81
 - find_or_create_session, 81
 - get_client_ip, 81
 - get_time_ms, 82
 - gpio_command_task_wrapper, 82
 - GPIO_QUEUE_SIZE, 79
 - init_spiffs, 82
 - is_authenticated, 82
 - login_post_handler, 83
 - logout_get_handler, 83
 - MAX_SESSIONS, 79
 - pir_status_get_handler, 83
 - REDIRECT, 79
 - register_http_routes, 83
 - registros_get_handler, 83
 - registros_post_handler, 84
 - root_get_handler, 84
 - script_get_handler, 84
 - send_file_from_spiffs, 84
 - SEND_JSON_RESPONSE, 79
 - SEND_JSON_RESPONSE_STR, 79
 - session_management_task, 84
 - SESSION_TIMEOUT_MS, 79
 - slider_get_handler, 85
 - start_webserver, 85
 - str_to_lower, 85
 - style_get_handler, 86
 - TAG, 86
 - temperature_get_handler, 86
 - terminal_get_handler, 86
 - time_set_post_handler, 86
 - url_decode, 86
 - VALID_PASS, 80
 - VALID_USER, 80
 - verify_spiffs_files, 86
- web_server.h
 - start_webserver, 89
- webserver_context_t, 10
 - command_id_counter, 10
 - command_id_mutex, 10
 - gpio_command_queue, 10
 - gpio_response_queue, 10
 - server, 10
 - session_mutex, 10
 - sessions, 10
- wifi_app.c
 - s_retry_num, 92
 - TAG, 92
 - wifi_connected, 92
 - wifi_event_handler, 91
 - wifi_init_softap, 91
 - wifi_is_connected, 92
- wifi_app.h
 - ESP_WIFI_CHANNEL, 94
 - ESP_WIFI_PASS, 94
 - ESP_WIFI_SSID, 94
 - MAX_STA_CONN, 94
 - wifi_init_softap, 94
 - wifi_is_connected, 95
 - WIFI_MAX_RETRY, 94
 - WIFI_STA_PASS, 94
 - WIFI_STA_SSID, 94
- wifi_connected
 - wifi_app.c, 92
- wifi_event_handler
 - wifi_app.c, 91
- wifi_init_softap
 - wifi_app.c, 91
 - wifi_app.h, 94

wifi_is_connected
 wifi_app.c, [92](#)
 wifi_app.h, [95](#)
WIFI_MAX_RETRY
 wifi_app.h, [94](#)
WIFI_STA_PASS
 wifi_app.h, [94](#)
WIFI_STA_SSID
 wifi_app.h, [94](#)